Rust

K19G

2026-10-28

Table of contents

Rust **23**
A Complete Guide to Safe, Concurrent, High-Performance Programming 23
Introduction: Why Rust Matters 23
The Problem Rust Solves 23
How Rust Achieves This 24
What You Will Learn 24
How to Read This Book 24
The Current State of Rust 24
What Rust Is Not 25
Formats 25

Getting Started with Rust **26**

The Story of Rust: History and Philosophy **27**

The Story of Rust: History and Philosophy **28**
Core Philosophy 28

Installing Rust with rustup **29**

Installing Rust with rustup **30**
Installing rustup 30
Managing Toolchains 30

Your First Program: Hello World **31**

Your First Program: Hello World **32**
Direct Compilation with rustc 32
Deconstructing the Program 32

Understanding rustc and cargo **33**

Understanding rustc and cargo **34**
The Role of Cargo 34
Common Cargo Commands 34

Exploring a Cargo Project Structure **35**

Exploring a Cargo Project Structure **36**
Standard Directory Layout 36
The Cargo.toml Manifest 36

Summary 37

Summary 38
Chapter Key Takeaways 38

Language Basics: Variables, Types, and Expressions 39

Variables, Mutability, and Shadowing 40

Variables, Mutability, and Shadowing 41
Mutability with `mut` 41
Variable Shadowing 41

Primitive Data Types in Detail 42

Primitive Data Types in Detail 43
Scalar Types 43

Compound Types: Tuples and Arrays 44

Compound Types: Tuples and Arrays 45
Tuples 45
Arrays 45

Functions, Statements, and Expressions 46

Functions, Statements, and Expressions 47
Statements vs. Expressions 47

Control Flow: if, loop, while, and for 48

Control Flow: if, loop, while, and for 49
`if` as an Expression 49
Loops: `loop`, `while`, and `for` 49

NonZero Integer Types 50

NonZero Integer Types 51
Thesis 51
Mental model 51
Worked example 51
Trap 52
Boring rule 52
Try this 52
Sources 52

Summary 53

Summary 54
Chapter Key Takeaways 54

Ownership: The Heart of Rust **55**

What is Ownership and Why Does It Exist? **56**

What is Ownership and Why Does It Exist? **57**
The Three Rules of Ownership 57

Stack vs Heap Memory in Rust **58**

Stack vs Heap Memory in Rust **59**
The Stack 59
The Heap 59

Move Semantics Explained **60**

Move Semantics Explained **61**
Why Rust Moves Instead of Shallow Copying 61

Copy Types and Clone **62**

Copy Types and Clone **63**
The `Copy` Trait 63
The `Clone` Trait 63

The Drop Trait and Resource Cleanup **64**

The Drop Trait and Resource Cleanup **65**
Custom Drop Implementations 65

The Cost of Drop on Deep Ownership Chains **66**

The Cost of Drop on Deep Ownership Chains **67**
Thesis 67
Mental model 67
Worked examples 67
Trap 67
Boring rule 68
Try this 68
Sources 68

Summary **69**

Summary **70**
Chapter Key Takeaways 70

Borrowing, References, and Slices **71**

Shared and Mutable References **72**

Shared and Mutable References **73**
Shared References (`&T`) . . . 73
Mutable References (`&mut T`) . . . 73

The Borrowing Rules Demystified **74**

The Borrowing Rules Demystified **75**
Why the Rules Exist . . . 75

Dangling References Prevented at Compile Time **76**

Dangling References Prevented at Compile Time **77**

String Slices and &str **78**

String Slices and &str **79**
`String` vs `&str` . . . 79

Slice Types and Range Syntax **80**

Slice Types and Range Syntax **81**

Summary **82**

Summary **83**
Chapter Key Takeaways . . . 83

Structs, Enums, and Pattern Matching **84**

Defining and Using Structs **85**

Defining and Using Structs **86**

Enumerations Beyond Simple Tags **87**

Enumerations Beyond Simple Tags **88**

The match Expression in Depth **89**

The match Expression in Depth **90**

if let and while let for Partial Matching **91**

if let and while let for Partial Matching **92**

Real-World Data Modeling with Enums **93**

Real-World Data Modeling with Enums **94**

Summary **95**

Summary **96**
Chapter Key Takeaways 96

Modules, Crates, and Visibility **97**

Modules and the Filesystem **98**

Modules and the Filesystem **99**
The Modern Module Mapping 99

The use Statement and Path Resolution **100**

The use Statement and Path Resolution **101**

Public vs Private Visibility Rules **102**

Public vs Private Visibility Rules **103**

Crates, Packages, and Cargo Workspaces **104**

Crates, Packages, and Cargo Workspaces **105**

Re-exporting and Organizing Large Projects **106**

Re-exporting and Organizing Large Projects **107**

Summary **108**

Summary **109**
Chapter Key Takeaways 109

Collections and Strings **110**

Vec and Dynamic Arrays **111**

Vec and Dynamic Arrays **112**

String vs &str Deep Dive **113**

String vs &str Deep Dive **114**

HashMap and BTreeMap for Key-Value Storage **115**

HashMap and BTreeMap for Key-Value Storage **116**

Other Collections: HashSet, BinaryHeap, LinkedList **117**

Other Collections: HashSet, BinaryHeap, LinkedList **118**

Efficient Collection Operations **119**

Efficient Collection Operations 120

Summary 121

Summary 122
Chapter Key Takeaways 122

Project 1: Building a CLI Task Manager 123

Building a CLI Task Manager 124

Building a CLI Task Manager 125

Testing the Task Manager 127

Testing the Task Manager 128

Benchmarking the Task Manager 129

Benchmarking the Task Manager 130

Deploying the Task Manager 131

Deploying the Task Manager 132

Error Handling in Rust 133

Unrecoverable Errors with panic! 134

Unrecoverable Errors with panic! 135

Recoverable Errors with Result 136

Recoverable Errors with Result 137

The Option Type for Absent Values 138

The Option Type for Absent Values 139

Propagating Errors with the ? Operator 140

Propagating Errors with the ? Operator 141

Custom Error Types and thiserror 142

Custom Error Types and thiserror 143

The Never Type ! and Infallible 144

The Never Type `!` and `Infallible` **145**
Thesis . . . 145
Mental model . . . 145
Worked examples . . . 146
Case 1: `!` as a type — cannot-fail results, irrefutable `Ok` . . . 146
Case 2: the fallback change — code that relied on `()` by accident . . . 148
Case 3: the panicking closure . . . 150
Case 4: `Infallible` and `!` are one type now . . . 151
Where `!` belongs in error handling . . . 152
The trap . . . 152
The boring rule . . . 153
Try this . . . 153
Sources . . . 153

Summary **155**

Summary **156**
Chapter Key Takeaways . . . 156

Generics, Traits, and Associated Types **157**

Generic Functions and Data Structures **158**

Generic Functions and Data Structures **159**

Defining and Implementing Traits **160**

Defining and Implementing Traits **161**

Trait Bounds and Where Clauses **162**

Trait Bounds and Where Clauses **163**

Associated Types vs Generics **164**

Associated Types vs Generics **165**

Trait Objects and Dynamic Dispatch **166**

Trait Objects and Dynamic Dispatch **167**

Generic Const Arguments (GCA) **168**

Generic Const Arguments (GCA) **169**
Thesis . . . 169
Mental model . . . 169
Worked examples . . . 170
Case 0: stable still rejects `N + 1` . . . 170
Case 1: ADT const arguments with `gca_adts` . . . 171
Case 2: Associated constants with `gca_const_items` . . . 172

Case 3: Macroless (experimental) . . . 172
Case 4: What still hurts — arithmetic inside `gca!` . . . 173
The trap . . . 174
The boring rule . . . 174
Try this . . . 174
Sources . . . 174

AsRef, Borrow and Deref: Same Signature, Different Promises **175**

`AsRef`, `Borrow` and `Deref`: Same Signature, Different Promises **176**
Thesis . . . 176
Mental model . . . 176
Worked examples . . . 177
Case 1: How `HashMap<String, V>` answers a `&str` . . . 177
Case 2: `AsRef` as an "accept anything" bound . . . 179
Case 3: Break the `Borrow` contract and the map loses keys . . . 180
Case 4: The correct shape: borrow as a type with the same `Hash` and `Eq` . . . 182
Case 5: `Borrow` is reflexive, `AsRef` is not . . . 184
Case 6: `ToOwned` is `Borrow` run backwards, and `Cow` stands on both . . . 185
The trap . . . 186
The boring rule . . . 186
Try this . . . 187
Sources . . . 187

Cow — Borrow When You Can, Own When You Must **188**

Cow — Borrow When You Can, Own When You Must **189**
Thesis . . . 189
Mental model . . . 189
Worked examples . . . 189
Trap . . . 189
Boring rule . . . 190
Try this . . . 190
Sources . . . 190

Summary **191**

Summary **192**
Chapter Key Takeaways . . . 192

Lifetimes: Complete Understanding **193**

What Lifetimes Are (and Are Not) **194**

What Lifetimes Are (and Are Not) **195**
What Lifetimes Are Not . . . 195

The Lifetime Elision Rules **196**

The Lifetime Elision Rules **197**

Annotating Lifetimes in Functions and Structs **198**

Annotating Lifetimes in Functions and Structs **199**

Lifetime Bounds on Traits **200**

Lifetime Bounds on Traits **201**

Advanced Lifetime Patterns **202**

Advanced Lifetime Patterns **203**
The `'static` Lifetime . . . 203

Summary **204**

Summary **205**
Chapter Key Takeaways . . . 205

Smart Pointers and Interior Mutability **206**

Box for Heap Allocation **207**

Box for Heap Allocation **208**

Reference Counting with Rc and Arc **209**

Reference Counting with Rc and Arc **210**

Interior Mutability with RefCell **211**

Interior Mutability with RefCell **212**

Cell and Mutex for Shared State **213**

Cell and Mutex for Shared State **214**

Weak References and Breaking Cycles **215**

Weak References and Breaking Cycles **216**

LazyLock and OnceLock **217**

LazyLock and OnceLock **218**
Mental model . . . 218
Worked examples . . . 219
Case 1: `LazyLock` for a process-wide regex . . . 219
Case 2: `OnceLock` when init needs an argument . . . 219
Case 3: Fallible init without poisoning `LazyLock` . . . 220
Case 4: Single-threaded `LazyCell` . . . 221

Choosing among the options . . . 221
Pitfalls . . . 221
Try this . . . 222
Sources . . . 222

MaybeUninit and Uninitialized Memory 223

MaybeUninit and Uninitialized Memory 224
Mental model . . . 224
Worked examples . . . 225
Case 1: `write` then `assume_init` . . . 225
Case 2: Out-pointer style . . . 225
Case 3: Array element-by-element . . . 225
Case 4: Field-by-field struct init . . . 226
Case 5: `zeroed` pitfalls . . . 227
Choosing the API . . . 227
Pitfalls . . . 228
Try this . . . 228
Sources . . . 228

Custom Allocators and the Allocator API 229

Custom Allocators and the Allocator API 230
Thesis . . . 230
Mental model . . . 230
Worked examples . . . 231
Case 1: a counting allocator and a bump arena . . . 231
Case 2: what the allocator parameter costs . . . 235
Case 3: an allocator says no . . . 236
Case 4: what is *not* stable yet . . . 238
The trap . . . 240
A collection outlives its arena . . . 240
`Vec<T, A>` does not fit where `Vec<T>` is expected . . . 241
Implementing the trait is a safety contract . . . 242
Choosing an approach . . . 242
The boring rule . . . 242
Try this . . . 243
Sources . . . 243

Summary 244

Summary 245
Chapter Key Takeaways . . . 245

Iterators, Closures, and Functional Patterns 246

The Iterator Trait and Protocol 247

The Iterator Trait and Protocol 248

Creating Custom Iterators 249

Creating Custom Iterators 250

Closures and Capture Semantics 251

Closures and Capture Semantics 252

Higher-Order Functions and Combinators 253

Higher-Order Functions and Combinators 254

Lazy Evaluation and Performance 255

Lazy Evaluation and Performance 256

FusedIterator — Why Fuse Matters 257

FusedIterator — Why Fuse Matters 258
- Thesis . . . 258
- Mental model . . . 258
- Worked example . . . 258
- Trap . . . 259
- Boring rule . . . 259
- Try this . . . 259
- Sources . . . 259

Summary 260

Summary 261
- Chapter Key Takeaways . . . 261

Concurrency: Fearless Parallelism 262

Spawning Threads with std::thread 263

Spawning Threads with std::thread 264

Message Passing with Channels 265

Message Passing with Channels 266

Shared State with Mutex and RwLock 267

Shared State with Mutex and RwLock 268

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across .await 269

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across `.await` 270
Thesis . . . 270
Mental model . . . 270
Worked examples . . . 271
Case 1: A tiny read-only critical section . . . 273
Case 2: The same reads with real work inside . . . 274
Case 3: One write in a hundred . . . 275
Case 4: Stop sharing the written word: a big-reader lock . . . 276
Case 5: A `std::sync::MutexGuard` held across `.await` . . . 277
The trap . . . 281
The boring rule . . . 282
Try this . . . 282
Sources . . . 283

Atomic Operations for Lock-Free Code 284

Atomic Operations for Lock-Free Code 285

Send and Sync Traits Explained 286

Send and Sync Traits Explained 287

Summary 288

Summary 289
Chapter Key Takeaways . . . 289

Project 3: Parallel Data Processing Pipeline with rayon 290

Parallel Data Processing Pipeline with rayon 291

Parallel Data Processing Pipeline with rayon 292

Testing the Text Analyzer 293

Testing the Text Analyzer 294

Benchmarking the Text Analyzer 295

Benchmarking the Text Analyzer 296

Deploying the Text Analyzer 297

Deploying the Text Analyzer 298

Asynchronous Programming with async/await 299

The Future Trait and Async Basics 300

The Future Trait and Async Basics **301**

Running Async Code with Tokio **302**

Running Async Code with Tokio **303**

Pinning and Self-Referential Types **304**

Pinning and Self-Referential Types **305**

Async Streams and Error Handling **306**

Async Streams and Error Handling **307**

Building Production Async Applications **308**

Building Production Async Applications **309**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **310**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **311**
- Thesis . . . 311
- Mental model . . . 311
- Worked examples . . . 312
  - Case 1: Same throughput, very different latency . . . 312
  - Case 2: Under a CPU limit . . . 314
  - Case 3: Bound the work in flight . . . 315
- The trap . . . 315
- The boring rule . . . 316
- Try this . . . 317
- Sources . . . 317

Sans-IO Protocol Design: State Machines Without Sockets **318**

Sans-IO Protocol Design: State Machines Without Sockets **319**
- Thesis . . . 319
- Mental model . . . 319
- Worked examples . . . 320
  - Case 1: a length-prefixed framer with no sockets . . . 320
  - Case 2: the driver loop (sketch) . . . 323
  - Case 3: why this pairs with Rust . . . 324
- The trap . . . 324
- The boring rule . . . 325
- Try this . . . 325
- Sources . . . 325

Cancel Safety: Dropping a Future Is Cancelling It **326**

Cancel Safety: Dropping a Future Is Cancelling It **327**
- Thesis . . . 327

Mental model . . . 327
Worked examples . . . 328
Case 1: A future is a value, and dropping it cancels it . . . 328
Case 2: `send` under a timeout loses the message, `reserve` does not . . . 330
Case 3: A `select!` loop over `read_exact` corrupts a framed stream . . . 333
Case 3b: Or don't cancel at all: poll one future by `&mut` . . . 335
Case 4: Cancel correctness: an invariant broken across an `.await` . . . 337
The trap . . . 338
The boring rule . . . 340
Try this . . . 340
Sources . . . 340

A Tiny Async Runtime from Scratch **342**

A Tiny Async Runtime from Scratch **343**
Thesis . . . 343
Mental model . . . 343
Worked example . . . 343
Trap . . . 344
Boring rule . . . 344
Try this . . . 344
Sources . . . 345

Summary **346**

Summary **347**
Chapter Key Takeaways . . . 347

Macros, Unsafe Rust, and FFI **348**

Declarative Macros with macro_rules! **349**

Declarative Macros with macro_rules! **350**

Procedural Macros and Derive **351**

Procedural Macros and Derive **352**

Understanding unsafe Rust **353**

Understanding unsafe Rust **354**

Memory Layout and Representation **355**

Memory Layout and Representation **356**

Foreign Function Interfaces **357**

Foreign Function Interfaces **358**

C-Variadic Functions and VaList **359**

C-Variadic Functions and `VaList` **360**
Thesis . . . 360
Mental model . . . 360
Worked examples . . . 361
Case 1: count-prefixed list, defined and called in Rust . . . 361
Case 2: a printf-style logger implemented in Rust, called from C . . . 362
Case 3: the header catches caller mistakes before they become UB . . . 365
Case 4: what the compiler refuses . . . 366
The trap . . . 368
The boring rule . . . 369
Try this . . . 369
Sources . . . 370

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1) **371**

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1) **372**
Learning goals . . . 372
Thesis . . . 372
Mental model . . . 372
Case 1 — Minimal reproduction . . . 373
Genuine output on this box (rustc 1.99.0) . . . 374
Case 2 — Who was affected . . . 374
Case 3 — Mitigations . . . 374
The trap . . . 375
The boring rule . . . 375
Try this . . . 375
Sources . . . 375
Inspiration . . . 375
Verification . . . 376

Safe Rust UAF via covariant field assignment (rust#164250) **377**

Safe Rust UAF via covariant field assignment (rust#164250) **378**
Learning goals . . . 378
Thesis . . . 378
Mental model . . . 378
Worked examples . . . 379
Case 1: Minimal Drop smuggling (reproduced here) . . . 379
Case 2: `'static` bound bypass with a kept value . . . 380
Case 3: What to check in your code . . . 381
The trap . . . 381
The boring rule . . . 381
Try this . . . 381
Sources . . . 382

Summary **383**

Summary **384**
Chapter Key Takeaways . . . 384

Testing, Documentation, Performance, and Deployment **385**

Unit Tests, Integration Tests, and Doctests **386**

Unit Tests, Integration Tests, and Doctests **387**
Unit Tests . . . 387
Integration Tests . . . 387
Documentation Tests (Doctests) . . . 387

Benchmarking with Criterion **388**

Benchmarking with Criterion **389**

Writing Great Documentation **390**

Writing Great Documentation **391**

Profiling and Optimization Techniques **392**

Profiling and Optimization Techniques **393**

Building and Deploying Rust Applications **394**

Building and Deploying Rust Applications **395**

MSRV in practice: rust-version and the MSRV-aware resolver **396**

MSRV in practice: `rust-version` and the MSRV-aware resolver **397**
Learning goals . . . 397
Thesis . . . 397
Mental model . . . 397
Worked examples . . . 399
Case 1 — The break is two levels down . . . 399
Case 2 — Let the resolver keep the promise . . . 401
Case 3 — `cargo add` is MSRV-aware; your requirement wins over the resolver . . . 402
Case 4 — Your own `rust-version`, the escape hatch, and `--locked` . . . 403
Case 5 — No `rust-version`? Then your toolchain is the MSRV . . . 404
Case 6 — Check the MSRV in CI (configuration, not run here) . . . 405
The trap . . . 405
The boring rule . . . 406
Try this . . . 406
Sources . . . 406

Summary **407**

Summary **408**
Chapter Key Takeaways . . . 408

The Rust Ecosystem: Essential Crates for Production Development 409

serde: Serialization and Deserialization 410

serde: Serialization and Deserialization 411

clap: Command-Line Argument Parsing 412

clap: Command-Line Argument Parsing 413

reqwest: HTTP Client 414

reqwest: HTTP Client 415

axum: Web Framework 416

axum: Web Framework 417

actix-web: High-Performance Web Framework 418

actix-web: High-Performance Web Framework 419

Project 2: Async HTTP API with axum, sqlx, and tracing 420

Project 2: Async HTTP API with axum, sqlx, and tracing 421

Testing the User API 422

Testing the User API 423

Benchmarking the User API 424

Benchmarking the User API 425

Deploying the User API 426

Deploying the User API 427

sqlx: Async Database Access 428

sqlx: Async Database Access 429

rayon: Data Parallelism 430

rayon: Data Parallelism 431

tracing: Structured Logging and Observability 432

tracing: Structured Logging and Observability 433

chrono: Date and Time 434

chrono: Date and Time 435

uuid: Universally Unique Identifiers **436**

uuid: Universally Unique Identifiers **437**

rand: Random Number Generation **438**

rand: Random Number Generation **439**

regex: Regular Expressions **440**

regex: Regular Expressions **441**

bytes: Efficient Binary Buffers **442**

bytes: Efficient Binary Buffers **443**

futures: Async Primitives **444**

futures: Async Primitives **445**

Summary **446**

Summary **447**
Chapter Key Takeaways . . . 447

Servo 0.1: Embeddable Engine on crates.io **448**

Servo 0.1: Embeddable Engine on crates.io **449**
Thesis . . . 449
Mental model . . . 449
Worked examples . . . 450
Case 1: Read the milestone announcement, then pin deliberately . . . 450
Case 2: Embedding API shape (no full build in this chapter) . . . 451
Case 3: Building without `./mach` (embedder perspective) . . . 451
Case 4: Migrating off 0.1 LTS (preview) . . . 452
The trap . . . 452
The boring rule . . . 452
Try this . . . 452
Sources . . . 453

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns **454**

The Rust Compilation Pipeline **455**

The Rust Compilation Pipeline **456**

Debugging Rust Programs **457**

Debugging Rust Programs **458**

Security Considerations in Rust **459**

Security Considerations in Rust **460**

Common Anti-Patterns and How to Avoid Them **461**

Common Anti-Patterns and How to Avoid Them **462**

Summary **463**

Summary **464**
Chapter Key Takeaways 464

WebAssembly with Rust **465**

Why Rust for WebAssembly? **466**

Why Rust for WebAssembly? **467**

WebAssembly Fundamentals **468**

WebAssembly Fundamentals **469**

Setting Up: wasm-pack and wasm-bindgen **470**

Setting Up: wasm-pack and wasm-bindgen **471**

Building Your First WebAssembly Module **472**

Building Your First WebAssembly Module **473**

Interacting with the DOM **474**

Interacting with the DOM **475**

Handling Events and Async **476**

Handling Events and Async **477**

Performance Considerations **478**

Performance Considerations **479**

Server-Side WebAssembly **480**

Server-Side WebAssembly **481**

Publishing and Distribution **482**

Publishing and Distribution **483**

Summary **484**

Summary **485**
Chapter Key Takeaways 485

Embedded Rust and no_std Development **486**

Understanding no_std **487**

Understanding no_std **488**

Choosing a Target: Cortex-M **489**

Choosing a Target: Cortex-M **490**

Writing an Embedded Application: Blinking an LED **491**

Writing an Embedded Application: Blinking an LED **492**

Memory Management Without a Heap **493**

Memory Management Without a Heap **494**

Interrupts and Concurrency **495**

Interrupts and Concurrency **496**

Choosing an Executor or RTOS **497**

Choosing an Executor or RTOS **498**

Testing Embedded Code **499**

Testing Embedded Code **500**

Toolchain and Debugging **501**

Toolchain and Debugging **502**

Summary **503**

Summary **504**
Chapter Key Takeaways 504

Conclusion: The Rust Mindset **505**

Core Principles **506**

Core Principles **507**

Where Rust Shines **508**

Where Rust Shines **509**

Where Rust May Not Be the Best Fit **510**

Where Rust May Not Be the Best Fit **511**

Continuing Your Journey **512**

Continuing Your Journey **513**

Final Thoughts **514**

Final Thoughts **515**

Rust

A Complete Guide to Safe, Concurrent, High-Performance Programming

A systematic, production-grade guide to writing Rust the way it lasts in real systems: **safe, concurrent, and high-performance**. Rust eliminates entire categories of critical bugs—null pointer dereferences, data races, buffer overflows, and use-after-free vulnerabilities—without the runtime overhead of a garbage collector.

This book provides a direct, comprehensive path from first principles to advanced production systems.

Introduction: Why Rust Matters

For decades, systems programming forced engineers into an uncomfortable compromise:

1. **Control and Speed without Safety:** Languages like C and C++ provide bare-metal hardware access, deterministic resource destruction, and peak performance, but leave memory safety, thread safety, and undefined behavior to developer discipline. Decades of CVE audits prove that even the most disciplined teams cannot prevent memory corruption at scale.
2. **Safety without Bare-Metal Control:** Managed languages like Java, Go, Python, and C# enforce memory safety through automatic garbage collection and runtime checks. However, garbage collection introduces non-deterministic pause times, memory footprint overhead, and indirection that make them less suitable for low-latency systems, kernel drivers, game engines, and embedded microcontrollers.

Rust breaks this compromise by achieving **compile-time memory safety without a garbage collector**.

The Problem Rust Solves

Software reliability and security crises predominantly stem from memory safety bugs and concurrency defects: - **Spatial memory safety violations:** Buffer overruns, out-of-bounds array reads and writes. - **Temporal memory safety violations:** Use-after-free, double free, dangling pointers. - **Concurrency bugs:** Data races occurring when two threads access the same memory location concurrently with at least one writer and no synchronization.

Rust guarantees that properly written safe code cannot experience any of these defects. If your code compiles, these failure modes are mathematically disproven by the compiler's borrow checker.

How Rust Achieves This

Rust replaces runtime garbage collection and manual memory management with three core mechanisms: 1. **Ownership:** Every value has a single owner bound to lexical scope. When the owner goes out of scope, the memory is deterministically freed (`Drop`). 2. **Borrowing & References:** References allow inspection or mutation of data without taking ownership. Rust enforces aliasing XOR mutability at compile time: you may have either any number of immutable references (`&T`) *or* exactly one mutable reference (`&mut T`), but never both simultaneously. 3. **Lifetimes:** The compiler tracks the validity of every reference, preventing dangling pointers across function boundaries and data structures at compile time without runtime overhead.

What You Will Learn

Across 20 comprehensive chapters and three end-to-end production capstones, this book covers: - Core language syntax, primitive/compound types, and zero-cost control flow. - Deep mechanical mastery of ownership, stack vs. heap allocation, move semantics, and lifetimes. - Robust data modeling with algebraic data types (`struct`, `enum`) and expressive pattern matching. - Explicit, resilient error handling with `Result`, `Option`, and the `?` propagation operator. - Parametric polymorphism with generics, trait bounds, and dynamic dispatch via trait objects. - Fearless concurrency using threads, message-passing channels, and atomic primitives. - Asynchronous programming with `async`/`await`, `Future`, and the Tokio runtime. - Systems-level metaprogramming with declarative macros, procedural macros, and foreign function interfaces (FFI). - Production engineering: profiling, benchmarking with Criterion, Criterion, cargo packaging, WebAssembly (`wasm-pack`), and embedded `no_std` firmware.

How to Read This Book

1. **Linear Progression:** Chapters 1 through 13 form the core foundation. Read them sequentially; each chapter relies strictly on concepts verified in preceding sections.
2. **Complete, Runnable Code:** Every listing is self-contained. Copy the code into your editor, compile it with `cargo check` and `cargo run`, and observe compiler messages.
3. **Fight and Befriend the Borrow Checker:** When the compiler rejects your code, read the compiler diagnostic carefully. Rust diagnostics are world-class teaching tools explaining what invariant was violated and how to rectify it.
4. **Capstones & Projects:** Work through the three integrated projects (CLI Task Manager, Parallel Data Pipeline with Rayon, and Async HTTP API with Axum and SQLx) to solidify architecture and integration patterns.

The Current State of Rust

Rust is mature, battle-tested, and officially adopted across industry leaders for core infrastructure: - The Linux kernel has official support for Rust drivers and kernel modules. - Major cloud hypervisors and networking layers (AWS Firecracker, Cloudflare Workers, Android OS) are authored in Rust. - Standards organizations and government agencies (CISA, NSA) recommend transitioning memory-unsafe codebases to memory-safe languages like Rust.

What Rust Is Not

To set clear expectations, Rust is not: - **An object-oriented language with class hierarchies:** Rust uses composition, traits, and algebraic data types rather than inheritance. - **A rapid prototyping scripting language:** Rust demands explicit thinking up front about ownership, error conditions, and data lifetimes. - **Magic:** Zero-cost abstractions mean you don't pay runtime costs for high-level ergonomics; however, you must understand machine layouts and memory representation.

Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive layout with interactive syntax highlighting and dual light/dark themes. - **PDF** — typeset document suitable for offline reading and archiving. - **EPUB** — standardized eBook format for mobile devices and e-readers.

Getting Started with Rust

The Story of Rust: History and Philosophy

The Story of Rust: History and Philosophy

Rust began in 2006 as a personal project by Graydon Hoare, a language engineer at Mozilla. At the time, Mozilla was grappling with the massive complexity and vulnerability surface of the Gecko rendering engine in Firefox, written in C++. In multi-threaded C++, memory corruption vulnerabilities (such as use-after-free and buffer overflows) and concurrency bugs (data races) were extraordinarily difficult to prevent, even for seasoned systems developers.

Mozilla sponsored Rust in 2009 with a distinct mission: **create a modern systems language that guarantees memory safety and thread safety without sacrificing bare-metal speed or requiring a garbage collector**.

Core Philosophy

The design of Rust rests on three non-negotiable pillars: 1. **Safety:** Eliminate undefined behavior, dangling pointers, null dereferences, and data races at compile time. 2. **Speed:** Provide zero-cost abstractions—constructs that compile down to assembly indistinguishable from hand-written C. 3. **Concurrency:** "Fearless concurrency"—the compiler prevents data races across threads using the exact same ownership model used for memory safety.

```
// A safe, deterministic greeting
fn main() {
    let project = "Rust";
    let year = 2026;
    println!("Welcome to {}, built for safety and speed in {}.", project, year);
}
```

Installing Rust with rustup

Installing Rust with rustup

The primary and official method for installing and managing Rust toolchains across all platforms is `rustup`.

Installing rustup

On Linux and macOS, install `rustup` via your terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Follow the default prompts. This installs: - `rustc`: The Rust compiler. - `cargo`: The package manager and build system. - `rustup`: The toolchain multiplexer. - `rust-std`: Standard library binaries for your host architecture.

Managing Toolchains

Configure your shell environment:

```
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

To update your toolchain to the latest stable release:

```
rustup update stable
```

Your First Program: Hello World

Your First Program: Hello World

Let's begin with the canonical starting point: a minimal standalone Rust source file.

Direct Compilation with rustc

Create a file named `main.rs`:

```
// main.rs
fn main() {
    println!("Hello, world!");
}
```

Compile and run directly using `rustc`:

```
rustc main.rs
./main
```

Output:

```
Hello, world!
```

Deconstructing the Program

- `fn main() { ... }`: The `main` function is the entry point of every executable Rust program. It takes no arguments and returns unit `()`.
- `println!(...)`: In Rust, names ending in an exclamation mark `!` invoke **macros** rather than standard functions. `println!` parses format strings at compile time to ensure type safety.

Understanding rustc and cargo

Understanding rustc and cargo

While `rustc` is the underlying compiler, production Rust projects are almost exclusively managed with **Cargo**.

The Role of Cargo

Cargo handles four essential engineering responsibilities: 1. **Dependency Resolution:** Downloading, pinning, and compiling external crates from crates.io. 2. **Build Orchestration:** Invoking `rustc` with optimal optimization flags, link arguments, and target settings. 3. **Test & Benchmark Execution:** Discovering and executing unit tests, integration tests, and performance benchmarks. 4. **Documentation Generation:** Generating standard HTML documentation directly from doc-comments.

Common Cargo Commands

```
cargo check   # Fast syntax and type verification without binary emission
cargo build   # Compile debug build with symbols into target/debug
cargo run     # Build and execute the binary
cargo test    # Run automated test harness
cargo doc     # Build HTML documentation for current project and dependencies
```

Exploring a Cargo Project Structure

Exploring a Cargo Project Structure

Initialize a new project with Cargo:

```
cargo new desk_service --bin
cd desk_service
```

Standard Directory Layout

```
desk_service/
   Cargo.lock       # Exact resolved dependency graph
   Cargo.toml       # Manifest file: project metadata & dependencies
   src/
       main.rs      # Binary root entry point
   target/          # Build artifacts and compilation cache (gitignored)
```

The Cargo.toml Manifest

```
[package]
name = "desk_service"
version = "0.1.0"
edition = "2024"

[dependencies]
```

The `edition = "2024"` flag configures the language edition. Rust guarantees backwards compatibility across editions through epoch-based stability.

Summary

Summary

Chapter Key Takeaways

- Rust provides zero-cost memory safety without a garbage collector through static ownership analysis.
- `rustup` is the toolchain manager; keep it updated with `rustup update`.
- `rustc` compiles code, but day-to-day workflow centers on `cargo check`, `cargo build`, `cargo test`, and `cargo run`.
- Every Rust binary begins at `fn main()` and projects are configured through `Cargo.toml`.

Language Basics: Variables, Types, and Expressions

Variables, Mutability, and Shadowing

Variables, Mutability, and Shadowing

By default, variables in Rust are **immutable**. Once bound to a value, the binding cannot be modified.

Mutability with `mut`

To allow mutation, explicitly declare the binding with `mut`:

```
fn main() {
    let mut counter = 10;
    println!("Initial counter: {}", counter);
    counter += 5;
    println!("Updated counter: {}", counter);
}
```

Variable Shadowing

Rust allows redeclaring a variable with the same name in the same scope using `let`. This is **shadowing**:

```
fn main() {
    let spaces = "   ";
    let spaces = spaces.len(); // Shadows spaces: changes type from &str to usize
    println!("Number of spaces: {}", spaces);
}
```

Shadowing creates a new binding, allowing type transformation without requiring distinct variable names.

Primitive Data Types in Detail

Primitive Data Types in Detail

Rust is statically and strongly typed with full type inference.

Scalar Types

1. **Integers:**
   - Signed: `i8`, `i16`, `i32`, `i64`, `i128`, `isize` (pointer width)
   - Unsigned: `u8`, `u16`, `u32`, `u64`, `u128`, `usize` (pointer width, indexing)
2. **Floating-point:** `f32`, `f64` (IEEE 754-2008)
3. **Boolean:** `bool` (`true` or `false`, 1 byte representation)
4. **Character:** `char` (4-byte Unicode scalar value)

```
fn main() {
    let count: usize = 42;
    let temperature: f64 = 98.6;
    let is_active: bool = true;
    let emoji: char = ' ';

    println!("Scalar values: {}, {}, {}, {}", count, temperature, is_active, emoji);
}
```

Compound Types: Tuples and Arrays

Compound Types: Tuples and Arrays

Compound types group multiple values into a single type.

Tuples

A tuple has a fixed length and can contain elements of different types:

```
fn main() {
    let point: (i32, f64, &str) = (10, 3.14, "origin");
    let (x, y, label) = point; // Destructuring
    println!("Point {}: x = {}, y = {}", label, x, y);
    println!("Direct access: first = {}", point.0);
}
```

Arrays

An array has a fixed length, is stack-allocated, and elements must be of the **same type**:

```
fn main() {
    let buffer: [u8; 4] = [0xDE, 0xAD, 0xBE, 0xEF];
    println!("Array length: {}, first: 0x{:X}", buffer.len(), buffer[0]);

    let initialized = [0u32; 16]; // 16 elements initialized to 0
    println!("Initialized array length: {}", initialized.len());
}
```

Functions, Statements, and Expressions

Functions, Statements, and Expressions

Rust is an expression-oriented language. Almost every construct evaluates to a value.

Statements vs. Expressions

- **Statements:** Instructions that perform an action and do not return a value. Ending an expression with a semicolon `;` turns it into a statement returning unit `()`.
- **Expressions:** Evaluate to a resulting value. Omission of the final semicolon returns the expression value.

```
fn add(a: i32, b: i32) -> i32 {
    a + b // No semicolon: implicit return
}

fn compute_block(factor: i32) -> i32 {
    let result = {
        let base = 100;
        base * factor // Expression evaluated and bound to result
    };
    result
}

fn main() {
    println!("Add: {}", add(10, 20));
    println!("Compute block: {}", compute_block(3));
}
```

Control Flow: if, loop, while, and for

Control Flow: if, loop, while, and for

Control flow constructs in Rust are expressions that can return values.

`if` as an Expression

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };
    println!("Number: {}", number);
}
```

Loops: `loop`, `while`, and `for`

```
fn main() {
    // 1. loop with value return
    let mut counter = 0;
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2;
        }
    };
    println!("Loop result: {}", result);

    // 2. while loop
    let mut n = 3;
    while n > 0 {
        println!("{}!", n);
        n -= 1;
    }

    // 3. for loop over iterators
    for item in 0..3 {
        println!("Index: {}", item);
    }
}
```

NonZero Integer Types

NonZero Integer Types

Thesis

`NonZeroU32` (and friends) are integers that **cannot be zero**. Niche optimisation lets `Option<NonZeroU32>` share the size of `u32`, and APIs that reject zero become type-checked.

Mental model

```
NonZeroU32::new(0)  → None
NonZeroU32::new(7)  → Some(_)
Option<NonZeroU32> size == size_of::<u32>()   # niche
```

Worked example

```
// nonzero_demo.rs
use std::mem::size_of;
use std::num::NonZeroU32;

fn main() {
    println!("size u32={} Option<u32>={}", size_of::<u32>(), size_of::<Option<u32>>());
    println!(
        "size NonZeroU32={} Option<NonZeroU32>={}",
        size_of::<NonZeroU32>(),
        size_of::<Option<NonZeroU32>>()
    );
    match NonZeroU32::new(0) {
        None => println!("zero rejected"),
        Some(_) => unreachable!(),
    }
    let n = NonZeroU32::new(7).unwrap();
    println!("get={}", n.get());
}
```

```
get=7
zero rejected
div=42
```

Trap

Using `unwrap` on `NonZero*::new` with untrusted input. Prefer `new` + explicit error paths, or `NonZeroU32::try_from`. Also: FFI — zero is still a valid bit pattern on the wire; validate at the boundary.

Boring rule

Reach for `NonZero*` when zero is meaningless (divisors, lengths you already checked, database IDs you treat as non-zero). Document the invariant.

Try this

1. Compare `size_of::<Option<u32>>()` vs `Option<NonZeroU32>()`.
2. Use `NonZeroUsize` for a collection length after checking non-empty.
3. Read niche-optimisation notes in the nomicon / reference.

Sources

- `std::num::NonZeroU32`
- Rustonomicon: niche optimisations

Summary

Summary

Chapter Key Takeaways

- Variables are immutable by default; use `mut` to allow change.
- Shadowing (`let x = ...`) rebinds names and enables type changes.
- Rust primitives include scalar types (integers, floats, booleans, chars) and compound types (tuples, fixed-size arrays).
- Rust is expression-oriented: blocks and control flow evaluate directly to values.

Ownership: The Heart of Rust

What is Ownership and Why Does It Exist?

What is Ownership and Why Does It Exist?

Ownership is Rust's central memory management innovation. It governs how heap memory is allocated, accessed, and reclaimed without a garbage collector or manual `free()` calls.

The Three Rules of Ownership

1. **Each value in Rust has an owner.**
2. **There can only be one owner at a time.**
3. **When the owner goes out of scope, the value is dropped.**

```
fn main() {
    {
        let s = String::from("desk"); // s is valid from this point forward
        println!("Inside scope: {}", s);
    } // s goes out of scope; String memory is automatically freed here
}
```

Stack vs Heap Memory in Rust

Stack vs Heap Memory in Rust

Understanding where data lives is essential for writing efficient Rust code.

The Stack

- Fast, LIFO (Last In, First Out) allocation.
- All data stored on the stack must have a known, fixed size at compile time.
- Primitive types, fixed arrays, and function frames live on the stack.

The Heap

- Dynamic memory allocated at runtime.
- The operating system locates a contiguous region of memory, marks it in use, and returns a pointer.
- Slower than stack allocation; requires dereferencing pointers.
- Dynamically sized types like `String` and `Vec<T>` store metadata on the stack (pointer, length, capacity) pointing to heap buffers.

Move Semantics Explained

Move Semantics Explained

When a heap-backed value is assigned to another binding, passed to a function, or returned from a function, ownership is **moved**.

```
fn main() {
    let s1 = String::from("orders");
    let s2 = s1; // s1 is MOVED to s2

    // println!("{}", s1); // COMPILE ERROR: value borrowed here after move
    println!("s2 owns the heap buffer: {}", s2);
}
```

Why Rust Moves Instead of Shallow Copying

If `s1` and `s2` both owned the same heap pointer, when both went out of scope, they would attempt to free the same memory twice—a catastrophic **double free** bug. Rust prevents this by invalidating `s1` at compile time with zero runtime cost.

Copy Types and Clone

Copy Types and Clone

The `Copy` Trait

Types that live entirely on the stack and have fixed sizes implement the `Copy` trait. Assignment of `Copy` types performs a bitwise copy rather than a move:

```
fn main() {
    let x: i32 = 42;
    let y = x; // Bitwise copy
    println!("x = {}, y = {}", x, y); // Both valid!
}
```

The `Clone` Trait

For heap-allocated types where an explicit, potentially expensive deep copy is desired, use `.clone()`:

```
fn main() {
    let s1 = String::from("records");
    let s2 = s1.clone(); // Deep allocation on heap
    println!("s1 = {}, s2 = {}", s1, s2); // Both valid
}
```

The Drop Trait and Resource Cleanup

The Drop Trait and Resource Cleanup

Rust implements RAII (Resource Acquisition Is Initialization) via the `Drop` trait.

Custom Drop Implementations

```
struct ResourceHandle {
    name: String,
}

impl Drop for ResourceHandle {
    fn drop(&mut self) {
        println!("Cleaning up resource: {}", self.name);
    }
}

fn main() {
    let _r = ResourceHandle { name: String::from("NetworkSocket") };
    println!("Resource active");
    // _r drops automatically when main exits
}
```

Resources (file descriptors, sockets, database handles) cannot leak due to early returns or panics because `drop` runs deterministically on scope exit.

The Cost of Drop on Deep Ownership Chains

The Cost of Drop on Deep Ownership Chains

Thesis

`Drop` is not free. A long chain of `Box` nodes runs destructors one by one; `mem::forget` skips them and returns in nanoseconds — which is also how you leak. This chapter times dropping chains of 1 k / 10 k / 100 k nodes versus forgetting the large chain.

Mental model

```
drop(head) → Drop::drop → drop(child) → …  (recursive / iterative cleanup)
mem::forget(head) → no destructors; memory leaked unless into a global allocator you never
```

Worked examples

Verified with **rustc 1.99.1**, `cargo run --release`.

```
// chain of Box<Node>; time drop vs forget
```

Output:

```
drop chain of    1000 nodes: 21.888µs
drop chain of   10000 nodes: 267.357µs
drop chain of  100000 nodes: 4.211461ms
forget chain of  100000 nodes: 487ns
```

Drop scales with chain length (~4 ms for 100 k nodes here). `forget` on the same shape was ~0.5 µs — and leaked the graph.

Trap

Using `forget` to "speed up shutdown". You trade CPU for leaks and skipped safety hooks (`File` flush, lock poisoning cleanup, etc.).

Boring rule

Let `Drop` run. If shutdown profiles show destructor cost, batch into arenas or reuse pools — do not `forget` application state.

Try this

1. Replace the linked list with a `Vec<Node>` and compare drop time.
2. Implement `Drop` for a type that prints, and watch ordering.
3. Read `std::mem::ManuallyDrop`.

Sources

- Drop in the Rust book / std::ops::Drop
- `mem::forget`

Summary

Summary

Chapter Key Takeaways

- Ownership rules ensure single ownership per resource and deterministic cleanup.
- Stack memory holds fixed-size data; heap memory holds dynamic data referenced via stack pointers.
- Move semantics transfer ownership, invalidating previous bindings to prevent double-free errors.
- `Copy` types duplicate automatically via bitwise copy; heap types require explicit `.clone()`.
- The `Drop` trait executes custom cleanup when a value leaves scope.

Borrowing, References, and Slices

Shared and Mutable References

Shared and Mutable References

Instead of transferring ownership, a function can access data via **references**. This mechanism is called **borrowing**.

Shared References (`&T`)

Shared references allow read-only access to data without taking ownership:

```
fn calculate_length(s: &String) -> usize {
    s.len()
}

fn main() {
    let s = String::from("desk");
    let len = calculate_length(&s); // Pass reference
    println!("The length of '{}' is {}.", s, len); // s is still valid!
}
```

Mutable References (`&mut T`)

Mutable references permit modifying the borrowed value:

```
fn append_status(s: &mut String) {
    s.push_str(" [ACTIVE]");
}

fn main() {
    let mut task = String::from("Process orders");
    append_status(&mut task);
    println!("{}", task);
}
```

The Borrowing Rules Demystified

The Borrowing Rules Demystified

Rust's compiler enforces two fundamental rules regarding references at compile time:

1. **At any given time, you can have either one mutable reference or any number of immutable references.**
2. **References must always be valid.**

Why the Rules Exist

This rule (often summarized as **Aliasing XOR Mutability**) prevents **data races**. A data race occurs when: - Two or more pointers access the same memory concurrently. - At least one pointer is writing. - There is no synchronization.

By disallowing simultaneous reading and writing through multiple aliases, Rust eliminates data races before the code ever compiles.

Dangling References Prevented at Compile Time

Dangling References Prevented at Compile Time

In languages with manual memory management, creating a pointer to memory that has been deallocated results in a **dangling reference**. Rust guarantees this never occurs:

```
// Attempting to return a reference to a local variable:
// fn dangle() -> &String {
//     let s = String::from("temporary");
//     &s // ERROR: `s` does not live long enough
// }

// The idiomatic solution: transfer ownership of the value itself
fn no_dangle() -> String {
    let s = String::from("valid data");
    s
}

fn main() {
    let result = no_dangle();
    println!("Received: {}", result);
}
```

String Slices and &str

String Slices and &str

A string slice is a reference to a contiguous sequence of UTF-8 encoded bytes in a `String`.

```
fn main() {
    let s = String::from("Hello Rust");

    let first_word = &s[0..5];
    let second_word = &s[6..10];

    println!("First: {}, Second: {}", first_word, second_word);
}
```

`String` vs `&str`

- `String`: Owned, heap-allocated, resizable buffer.
- `&str`: An immutable view into string data located on the heap, stack, or embedded in the read-only data segment of the binary (string literals).

Slice Types and Range Syntax

Slice Types and Range Syntax

Slices generalize beyond strings to any contiguous sequence, such as arrays and vectors.

```
fn main() {
    let numbers = [10, 20, 30, 40, 50];

    let slice: &[i32] = &numbers[1..4]; // Elements 20, 30, 40
    println!("Slice: {:?}", slice);

    // Range variations:
    let start_to_two = &numbers[..2];
    let three_to_end = &numbers[3..];
    let all = &numbers[..];

    println!("Start: {:?}, End: {:?}, All: {:?}", start_to_two, three_to_end, all);
}
```

Summary

Summary

Chapter Key Takeaways

- References borrow data without transferring ownership.
- Aliasing XOR Mutability: either multiple `&T` or exactly one `&mut T`.
- Dangling references are prohibited at compile time.
- Slices (`&[T]` and `&str`) provide fat-pointer views (pointer + length) into contiguous memory without allocation.

Structs, Enums, and Pattern Matching

Defining and Using Structs

Defining and Using Structs

Structs allow creating custom types grouping related fields.

```
#[derive(Debug)]
struct Ticket {
    id: u64,
    title: String,
    resolved: bool,
}

impl Ticket {
    fn new(id: u64, title: String) -> Self {
        Self { id, title, resolved: false }
    }

    fn resolve(&mut self) {
        self.resolved = true;
    }
}

fn main() {
    let mut ticket = Ticket::new(101, String::from("Fix database migration"));
    println!("Ticket: {:?}", ticket);
    ticket.resolve();
    println!("Resolved: {:?}", ticket);
}
```

Enumerations Beyond Simple Tags

Enumerations Beyond Simple Tags

Rust enums are algebraic data types (tagged unions). Each variant can hold different types and amounts of data:

```
#[derive(Debug)]
enum WebEvent {
    PageLoad,
    KeyPress(char),
    Click { x: i64, y: i64 },
    Submit(String),
}

fn main() {
    let click = WebEvent::Click { x: 120, y: 340 };
    let key = WebEvent::KeyPress('q');
    println!("Events: {:?}, {:?}", click, key);
}
```

The match Expression in Depth

The match Expression in Depth

`match` enables exhaustive pattern matching. Every possible case must be handled.

```
#[derive(Debug)]
enum Priority {
    Low,
    Medium,
    High,
    Critical(u8),
}

fn inspect_priority(p: Priority) {
    match p {
        Priority::Low => println!("Low priority - queue normally"),
        Priority::Medium => println!("Medium priority - assign SLA"),
        Priority::High => println!("High priority - alert on-call"),
        Priority::Critical(level) => println!("CRITICAL (level {}) - page team immediately",
    }
}

fn main() {
    inspect_priority(Priority::High);
    inspect_priority(Priority::Critical(1));
}
```

if let and while let for Partial Matching

if let and while let for Partial Matching

When you only care about matching one pattern and ignoring all others, `if let` and `while let` avoid boilerplate:

```
fn main() {
    let config_val: Option<u32> = Some(8080);

    // if let syntax
    if let Some(port) = config_val {
        println!("Binding server to port {}", port);
    }

    // while let syntax
    let mut stack = vec![1, 2, 3];
    while let Some(top) = stack.pop() {
        println!("Popped: {}", top);
    }
}
```

Real-World Data Modeling with Enums

Real-World Data Modeling with Enums

Enums eliminate invalid program states at compile time:

```
#[derive(Debug)]
enum PaymentMethod {
    CreditCard { number: String, exp: String },
    WireTransfer { iban: String },
    InvoiceNet30 { po_number: u64 },
}

#[derive(Debug)]
struct Order {
    order_id: u64,
    amount_cents: u64,
    payment: PaymentMethod,
}

fn main() {
    let order = Order {
        order_id: 5001,
        amount_cents: 29900,
        payment: PaymentMethod::WireTransfer { iban: String::from("DE89370400440532013000")
    };
    println!("Recorded order: {:?}", order);
}
```

Summary

Summary

Chapter Key Takeaways

- Structs group named or positional fields and support methods via `impl`.
- Rust enums can carry rich payload data per variant.
- `match` enforces compile-time exhaustiveness, preventing unhandled cases.
- `if let` and `while let` provide concise syntax for single-variant checks.

Modules, Crates, and Visibility

Modules and the Filesystem

Modules and the Filesystem

Rust's module system organizes code into distinct namespaces and controls privacy.

The Modern Module Mapping

In modern Rust (2018/2021/2024 editions), declaring `mod network;` in `src/main.rs` resolves to `src/network.rs` or `src/network/mod.rs`.

```
// In src/network.rs
pub fn connect() {
    println!("Connecting to network host...");
}
```

The use Statement and Path Resolution

The use Statement and Path Resolution

The `use` keyword brings paths into scope to avoid verbose qualification:

```
mod storage {
    pub mod cache {
        pub fn clear() {
            println!("Cache cleared.");
        }
    }
}

// Bring clear into scope
use storage::cache::clear;

fn main() {
    clear();
}
```

Public vs Private Visibility Rules

Public vs Private Visibility Rules

In Rust, **everything is private by default**. To expose items, use `pub`: - `pub`: Public to everyone. - `pub(crate)`: Visible within the current crate only. - `pub(super)`: Visible to the parent module.

```
mod orders {
    pub struct Order {
        pub id: u64,
        secret_notes: String, // Private field!
    }

    impl Order {
        pub fn new(id: u64, notes: String) -> Self {
            Self { id, secret_notes: notes }
        }
    }
}

fn main() {
    let o = orders::Order::new(1, String::from("VIP customer"));
    println!("Order ID: {}", o.id);
    // println!("{}", o.secret_notes); // ERROR: private field
}
```

Crates, Packages, and Cargo Workspaces

Crates, Packages, and Cargo Workspaces

- **Crate:** The smallest compilation unit in Rust (a library or binary).
- **Package:** One or more crates defined by a `Cargo.toml`.
- **Workspace:** A set of packages that share a single `Cargo.lock` and output directory (`target/`).

```
# Top-level Cargo.toml for workspace
[workspace]
members = [
    "crates/server",
    "crates/core",
    "crates/client"
]
```

Re-exporting and Organizing Large Projects

Re-exporting and Organizing Large Projects

Re-exporting using `pub use` decouples internal module hierarchies from your public API:

```
mod internal {
    pub struct Engine;
    impl Engine {
        pub fn start(&self) { println!("Engine running"); }
    }
}

// Public API
pub use internal::Engine;

fn main() {
    let e = Engine;
    e.start();
}
```

Summary

Summary

Chapter Key Takeaways

- Files and directories map directly to modules.
- Privacy is strict: everything is private unless marked `pub`.
- `use` brings items into scope; `pub use` re-exports for clean API surfaces.
- Cargo workspaces share dependencies and build artifacts across multi-crate repositories.

Collections and Strings

Vec and Dynamic Arrays

Vec and Dynamic Arrays

`Vec<T>` is a growable, heap-allocated array.

```
fn main() {
    let mut v = vec![10, 20, 30];
    v.push(40);

    for (idx, elem) in v.iter().enumerate() {
        println!("v[{}] = {}", idx, elem);
    }

    // Safe retrieval via get()
    match v.get(2) {
        Some(&val) => println!("Element at index 2 is {}", val),
        None => println!("Index out of bounds"),
    }
}
```

String vs &str Deep Dive

String vs &str Deep Dive

Rust strings are always valid UTF-8 sequences.

```
fn main() {
    let mut s = String::from("desk");
    s.push_str(" management");

    let slice: &str = &s[..4];
    println!("Owned: '{}', Slice: '{}'", s, slice);

    // Iterating bytes vs characters
    for ch in "  Rust".chars() {
        println!("Char: {}", ch);
    }
}
```

HashMap and BTreeMap for Key-Value Storage

HashMap and BTreeMap for Key-Value Storage

```rust
use std::collections::{HashMap, BTreeMap};

fn main() {
    // HashMap (O(1) average lookup)
    let mut map = HashMap::new();
    map.insert("order_1", 450);
    map.insert("order_2", 890);

    println!("Order 1 total: {:?}", map.get("order_1"));

    // BTreeMap (sorted keys, O(log N))
    let mut btree = BTreeMap::new();
    btree.insert(200, "Two Hundred");
    btree.insert(100, "One Hundred");

    for (k, v) in &btree {
        println!("Key {} -> {}", k, v);
    }
}
```

Other Collections: HashSet, BinaryHeap, LinkedList

Other Collections: HashSet, BinaryHeap, LinkedList

```rust
use std::collections::{HashSet, BinaryHeap};

fn main() {
    // HashSet: Unique items
    let mut set = HashSet::new();
    set.insert("admin");
    set.insert("operator");
    println!("Has admin? {}", set.contains("admin"));

    // BinaryHeap: Max-heap priority queue
    let mut heap = BinaryHeap::new();
    heap.push(10);
    heap.push(50);
    heap.push(25);
    println!("Max element: {:?}", heap.pop()); // Some(50)
}
```

Efficient Collection Operations

Efficient Collection Operations

Preallocate capacity using `Vec::with_capacity` to prevent repeated reallocations:

```
fn main() {
    let mut fast_vec = Vec::with_capacity(1000);
    for i in 0..1000 {
        fast_vec.push(i * 2);
    }
    println!("Length: {}, Capacity: {}", fast_vec.len(), fast_vec.capacity());
}
```

Summary

Summary

Chapter Key Takeaways

- `Vec<T>` is the default dynamic contiguous collection.
- Rust strings enforce strict UTF-8 invariants.
- `HashMap` offers fast amortized key-value lookups; `BTreeMap` keeps keys sorted.
- Use `with_capacity` to eliminate reallocations in hot code paths.

Project 1: Building a CLI Task Manager

Building a CLI Task Manager

Building a CLI Task Manager

In this project, we build a production-ready command-line Task Manager supporting task creation, listing, completion, and disk persistence.

```
#[derive(Debug, PartialEq)]
pub struct Task {
    pub id: usize,
    pub description: String,
    pub completed: bool,
}

pub struct TaskManager {
    tasks: Vec<Task>,
    next_id: usize,
}

impl TaskManager {
    pub fn new() -> Self {
        Self { tasks: Vec::new(), next_id: 1 }
    }

    pub fn add(&mut self, description: String) -> usize {
        let id = self.next_id;
        self.tasks.push(Task { id, description, completed: false });
        self.next_id += 1;
        id
    }

    pub fn complete(&mut self, id: usize) -> bool {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.completed = true;
            true
        } else {
            false
        }
    }

    pub fn list(&self) -> &[Task] {
        &self.tasks
    }
```

```
}

fn main() {
    let mut manager = TaskManager::new();
    let id1 = manager.add(String::from("Configure CI pipeline"));
    let _id2 = manager.add(String::from("Write unit tests"));
    manager.complete(id1);

    for t in manager.list() {
        println!("[{}] #{}: {}", if t.completed { "X" } else { " " }, t.id, t.description);
    }
}
```

Testing the Task Manager

Testing the Task Manager

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_task_addition_and_completion() {
        let mut mgr = TaskManager::new();
        let id = mgr.add("Audit dependencies".into());
        assert_eq!(mgr.list().len(), 1);

        let ok = mgr.complete(id);
        assert!(ok);
        assert!(mgr.list()[0].completed);
    }

    #[test]
    fn test_nonexistent_completion() {
        let mut mgr = TaskManager::new();
        assert!(!mgr.complete(999));
    }
}
```

Benchmarking the Task Manager

Benchmarking the Task Manager

Benchmarking insertion and lookup performance:

```
// Simulation harness
fn run_benchmark() {
    let mut mgr = TaskManager::new();
    let start = std::time::Instant::now();
    for i in 0..10_000 {
        mgr.add(format!("Task number {}", i));
    }
    let duration = start.elapsed();
    println!("Inserted 10,000 tasks in {:?}", duration);
}

fn main() {
    run_benchmark();
}
```

Deploying the Task Manager

Deploying the Task Manager

Compile a stripped, fully optimized release binary:

```
cargo build --release
strip target/release/desk_tasks
./target/release/desk_tasks --help
```

Error Handling in Rust

Unrecoverable Errors with panic!

Unrecoverable Errors with panic!

Unrecoverable errors represent impossible program states or fatal invariant violations.

```
fn main() {
    // panic!("Fatal condition encountered: terminating process");
    println!("Panics unwind the stack by default, running destructors along the way.");
}
```

Recoverable Errors with Result

Recoverable Errors with Result

Recoverable errors in Rust are modeled through `Result<T, E>`:

```
#[derive(Debug)]
enum MathError {
    DivisionByZero,
}

fn safe_divide(numerator: f64, denominator: f64) -> Result<f64, MathError> {
    if denominator == 0.0 {
        Err(MathError::DivisionByZero)
    } else {
        Ok(numerator / denominator)
    }
}

fn main() {
    match safe_divide(10.0, 2.0) {
        Ok(val) => println!("Quotient: {}", val),
        Err(e) => println!("Error: {:?}", e),
    }
}
```

The Option Type for Absent Values

The Option Type for Absent Values

Rust has no `null` or `nil`. Absent values are explicitly represented with `Option<T>`:

```
fn find_desk(number: u32) -> Option<&'static str> {
    if number == 1 {
        Some("Front Reception")
    } else {
        None
    }
}

fn main() {
    let desk = find_desk(1);
    println!("Found desk: {:?}", desk.unwrap_or("Unassigned"));
}
```

Propagating Errors with the ? Operator

Propagating Errors with the ? Operator

The `?` operator unwraps `Ok` values or immediately returns early with `Err`:

```
fn parse_and_double(s: &str) -> Result<i32, std::num::ParseIntError> {
    let n = s.trim().parse::<i32>()?;
    Ok(n * 2)
}

fn main() {
    match parse_and_double("42") {
        Ok(v) => println!("Result: {}", v),
        Err(e) => println!("Parse failed: {}", e),
    }
}
```

Custom Error Types and thiserror

Custom Error Types and thiserror

In production services, domain errors are structured into custom enums:

```
#[derive(Debug)]
pub enum ServiceError {
    InvalidInput(String),
    NetworkTimeout,
}

impl std::fmt::Display for ServiceError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl std::error::Error for ServiceError {}

fn main() {
    let err = ServiceError::InvalidInput("Missing header".into());
    println!("Service error: {}", err);
}
```

The Never Type ! and Infallible

The Never Type `!` and `Infallible`

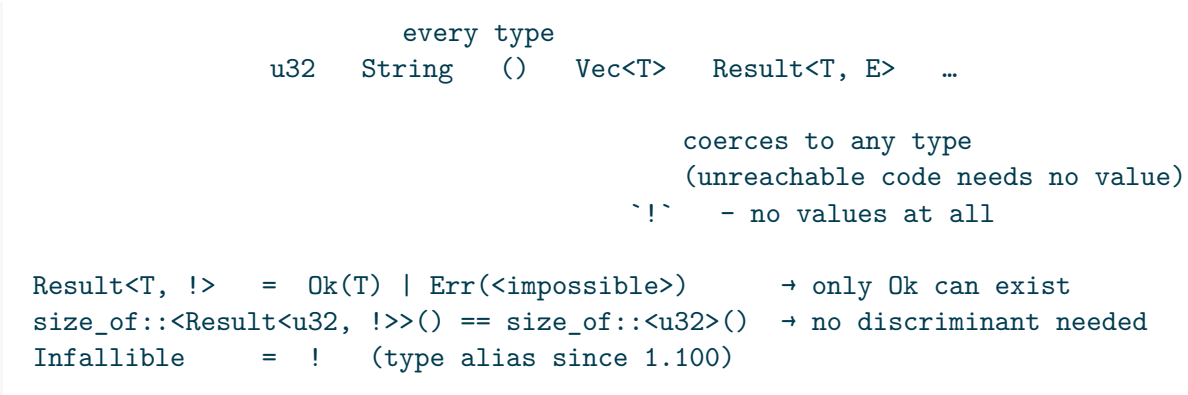
Thesis

Rust has always had a type for "this expression does not produce a value": `!`, the **never type**. `panic!()`, `loop {}` without `break`, `return`, `continue`, and `std::process::exit` all have type `!`, and a function declared `-> !` promises never to return. Until now you could only write `!` in that return position. Everywhere else — `Result<T, !>`, `type Error = !;`, `impl Trait for !` — it was an unstable feature, and the standard library used a stand-in, the empty enum `std::convert::Infallible`.

Rust 1.100 (scheduled for 2026-11-12) stabilizes `!` as an ordinary type after a decade behind a feature gate, and makes `Infallible` a plain alias for it. The language change is small; the error-handling consequence is real: "this operation cannot fail" now has one canonical spelling, `Result<T, !>`, that the compiler understands well enough to let you skip the impossible `Err` arm entirely.

The same release changes **never type fallback** to `!` on every edition, which is a small breaking change for code that quietly relied on the old fallback to `()`. This chapter shows both sides, compiled with the current beta and stable toolchains.

Mental model



Three facts carry the whole chapter:

Fact	Consequence
`!` has **no values**	Code that "holds" one is unreachable; the compiler can delete branches that need it
`!` **coerces to any type**	A diverging arm (`die(…)`, `panic!`, `return`) fits in a `match` whose other arms are `u32`

Fact	Consequence
An enum variant containing `!` is **uninhabited**	`let Ok(v) = r;` is irrefutable when `r: Result<T, !>`; the `Err` arm need not be written

`Infallible` was the library's way to get the same effect (an enum with no variants). Since it is now literally `!`, existing APIs such as `TryFrom` implementations whose error is `Infallible` work unchanged and read the same way.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, run them with `cargo +beta` / `rustc +beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: `!` as a type — cannot-fail results, irrefutable `Ok`

```
cargo +beta new never-demo && cd never-demo
```

```
// src/main.rs - the never type `!` as a first-class type (Rust 1.100)
use std::convert::Infallible;
use std::process;

// Case 1: a diverging function. `-> !` has been legal here since Rust 1.0.
fn die(msg: &str) -> ! {
    eprintln!("fatal: {msg}");
    process::exit(2)
}

// Case 2: `!` written anywhere a type goes - new in 1.100.
type Never = !;

fn parse_port(s: &str) -> Result<u16, Never> {
    // This parser cannot fail: anything unparsable maps to the default port.
    Ok(s.parse().unwrap_or(8080))
}

// Case 3: a trait whose implementors pick an error type; `!` means "cannot fail".
```

```
trait Source {
    type Error: std::fmt::Debug;
    fn read(&mut self) -> Result<u32, Self::Error>;
}

struct Counter(u32);
impl Source for Counter {
    type Error = !; // reading a counter never fails
    fn read(&mut self) -> Result<u32, !> {
        self.0 += 1;
        Ok(self.0)
    }
}

fn main() {
    // `!` coerces to any type, so a diverging arm fits a u32 match.
    let args: Vec<String> = std::env::args().collect();
    let n: u32 = match args.get(1) {
        Some(s) => s.parse().unwrap_or_else(|_| die("not a number")),
        None => 3,
    };

    // Irrefutable: the Err arm of Result<_, !> is uninhabited, so no match needed.
    let Ok(port) = parse_port("9000");
    println!("port = {port}");

    let mut c = Counter(0);
    let mut total = 0;
    for _ in 0..n {
        let Ok(v) = c.read();
        total += v;
    }
    println!("sum of first {n} reads = {total}");

    // Infallible is now the same type as `!`.
    let same: fn(Infallible) -> ! = |x| x;
    let _ = same;
    let Ok(wide) = u64::try_from(7u8); // TryFrom via Into: Error = Infallible = !
    println!("wide = {wide}");
    println!("size_of::<Result<u32, !>>() = {}", std::mem::size_of::<Result<u32, !>>());
    println!("size_of::<Result<u32, String>>() = {}", std::mem::size_of::<Result<u32, String
}
```

```
cargo +beta run -q
cargo +beta run -q -- 5
cargo +beta run -q -- five; echo "exit=$?"
```

```
port = 9000
sum of first 3 reads = 6
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
port = 9000
sum of first 5 reads = 15
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
fatal: not a number
exit=2
```

What to notice:

- `unwrap_or_else(|_| die("not a number"))` type-checks because the closure returns `!`, which coerces to `u32`.
- `let Ok(port) = …;` and `let Ok(v) = c.read();` have **no `else`** and no `match`. The compiler knows `Err(!)` cannot be constructed. With `Result<u16, String>` the same line is a compile error (refutable pattern).
- `Counter::Error = !` documents "cannot fail" in the type, and callers that are generic over `Source` still handle `Self::Error` normally.
- `Result<u32, !>` is the same size as a bare `u32`: no tag is stored for a variant that cannot exist.
- `fn(Infallible) -> !` accepting `|x| x` is the alias at work: the parameter *is* a `!`.

The same file on 1.99 stable stops at the first `!` in type position:

```
cargo +stable build -q
```

```
error[E0658]: the `!` type is experimental
  --> src/main.rs:12:14
   |
12 | type Never = !;
   |              ^
   |
   = note: see issue #35121 <https://github.com/rust-lang/rust/issues/35121> for more inform
...
```

(one such error per `!` outside a return position). Until your minimum supported Rust version is 1.100, keep spelling it `Infallible` in public APIs; code that already uses `Infallible` keeps compiling and gains nothing by being rewritten.

Case 2: the fallback change — code that relied on `()` by accident

When the compiler meets a `!` it must coerce, but nothing tells it the target type, it picks a **fallback**. That fallback used to be `()`; Rust 2024 switched it to `!`, and 1.100 does the same for editions 2015–

2021. The most common casualty is `?` on a function that is generic in its `Ok` type:

```
// fallback.rs - code that silently relied on `!` falling back to `()`
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f()?; // which T? nothing here says
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

On 1.99, edition 2021: already rejected by a deny-by-default future-compatibility lint, which names the fix:

```
rustc +stable --edition 2021 fallback.rs
```

```
error: this function depends on never type fallback being `()`
 --> fallback.rs:6:1
  |
6 | fn outer() -> Result<u32, ()> {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  = help: specify the types explicitly
note: in edition 2024, the requirement `!: Default` will fail
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^
  = warning: this was previously accepted by the compiler but is being phased out; it will b
  = note: for more information, see <https://doc.rust-lang.org/edition-guide/rust-2024/never
  = note: `#[deny(dependency_on_unit_never_type_fallback)]` (part of `#[deny(rust_2024_compa
help: use `()` annotations to avoid fallback changes
  |
7 |     f::<()>()?; // which T? nothing here says
  |      ++++++

error: aborting due to 1 previous error
```

On 1.100 beta, edition 2021 — the lint is gone because the behaviour changed; you get the real type error (identical to what 1.99 already says under edition 2024):

```
rustc +beta --edition 2021 fallback.rs
```

```
error[E0277]: the trait bound `!: Default` is not satisfied
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^ the trait `Default` is not implemented for `!`
  |
  = note: this error might have been caused by changes to Rust's type-inference algorithm (s
  = help: you might have intended to use the type `()` here instead
note: required by a bound in `f`
 --> fallback.rs:2:9
  |
2 | fn f<T: Default>() -> Result<T, ()> {
  |         ^^^^^^^ required by this bound in `f`
```

The fix is to say what you meant:

```
// fallback_fixed.rs - name the type instead of relying on fallback
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f::<()>()?; // say it: the Ok value is ()
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

```
rustc +beta --edition 2021 fallback_fixed.rs && ./fallback_fixed
```

```
Ok(1)
```

Case 3: the panicking closure

The second common pattern is a closure that only panics, passed to something that requires its return type to implement a trait:

```
// closure.rs - a panicking closure used to "return ()" by accident
trait Unit {}
impl Unit for () {}

fn run<R: Unit>(f: impl FnOnce() -> R) {
    let _ = std::panic::catch_unwind(std::panic::AssertUnwindSafe(f));
```

```
}

fn main() {
    run(|| panic!("boom"));
    println!("survived");
}
```

```
rustc +beta --edition 2024 closure.rs
```

```
error[E0277]: the trait bound `!: Unit` is not satisfied
  --> closure.rs:10:5
   |
10 |     run(|| panic!("boom"));
   |     ^^^^^^^^^^^^^^^^^^^^^^ the trait `Unit` is not implemented for `!`
   |
help: the trait `Unit` is implemented for `()`
  --> closure.rs:3:1
   |
 3 | impl Unit for () {}
   | ^^^^^^^^^^^^^^^^
   = note: this error might have been caused by changes to Rust's type-inference algorithm (
```

Annotate the closure's return type: `run(|| -> () { panic!("boom") });`

```
rustc +beta --edition 2024 closure_fixed.rs && ./closure_fixed
```

```
thread 'main' (250345) panicked at closure_fixed.rs:10:20:
boom
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
survived
```

Case 4: `Infallible` and `!` are one type now

If a crate implemented a trait for both — typically to support nightly users of `!` — the two impls now overlap:

```
// conflict.rs - Infallible and ! are now one type
use std::convert::Infallible;
trait Describe { fn describe() -> &'static str; }
impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
impl Describe for ! { fn describe() -> &'static str { "never" } }
fn main() { println!("{}", <! as Describe>::describe()); }
```

```
rustc +beta --edition 2024 conflict.rs
```

```
error[E0119]: conflicting implementations of trait `Describe` for type `!`
 --> conflict.rs:5:1
  |
4 | impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
  | ---------------------------- first implementation here
5 | impl Describe for ! { fn describe() -> &'static str { "never" } }
  | ^^^^^^^^^^^^^^^^^^^ conflicting implementation for `!`
```

Delete one of them. Keep the `Infallible` impl if you still support compilers older than 1.100: it covers `!` on new compilers for free.

Where `!` belongs in error handling

Situation	Spell it	Why
Function that never returns (exit, `exec`, server loop)	`fn run() -> !`	Callers can use it in any expression position
Trait with an associated error type; this impl cannot fail	`type Error = !;`	Callers generic over the trait still compile; concrete callers write `let Ok(v) = …;`
Public API that must build on Rust < 1.100	`Infallible`	Same type on new compilers, still compiles on old ones
"Can't fail *yet*" — you expect failures later	A real error type	Changing `!` to `MyError` later is a breaking API change
Converting between error types with `?`	`impl From<Infallible> for MyError` if needed	Works for `!` too, since it is the same type

The trap

Relying on **inference to pick `()` for you**. Code like `f()?;`, `run(|| panic!())`, or `if c { Default::default() } else { return }` compiled for years because the never type quietly became `()` when nothing else constrained it. Under 1.100 (any edition) or edition 2024 that guess is `!`, and the code either fails to compile — the good case, shown above — or, in the rare `unsafe` case the `never_type_fallback_flowing_into_unsafe` lint exists for, picks a different type than the author assumed. The compiler's 1.99 lint (`dependency_on_unit_never_type_fallback`) points at every affected function: fix those before you bump the toolchain, not after.

The second, quieter trap is using `!` as a placeholder in a public trait impl or function signature because "it can't fail today". `!` is a promise. Replacing it with a real error type later breaks every caller who wrote `let Ok(v) = …;`.

The boring rule

Use `-> !` for functions that truly never return and `Result<T, !>` (or `Infallible` until your MSRV is 1.100) only where failure is impossible by construction. Never lean on fallback: when the compiler cannot see a type, **write it** — `f::<()>()?`, `|| -> () { … }`, `() = expr;`. Turn on the `rust_2024_compatibility` lints on older editions and clear them before upgrading.

Try this

1. Change `parse_port` to return `Result<u16, String>`. Which lines stop compiling, and what does the compiler suggest for `let Ok(port) = …;`?
2. Print `size_of::<Option<!>>()` and `size_of::<Result<!, String>>()` and explain the numbers.
3. Write `fn absurd<T>(x: !) -> T { x }` and use it to implement `fn into_ok<T>(r: Result<T, !>) -> T` with a `match` that has a single arm.
4. Take a crate you maintain on edition 2021, run `cargo +stable check` (1.99 today) and look for `dependency_on_unit_never_type_fallback`; then `cargo +beta check` and confirm every site is fixed.
5. Find a `TryFrom` impl in `std` whose `Error` is `Infallible` and destructure its result with an irrefutable `let`.

Sources

- rust-lang/rust PR #155499, "stabilize never type" (merged 2026-08-24): https://github.com/rust-lang/rust/pull/155499
- Draft release notes for Rust 1.100.0 (stabilize never type; `Infallible` alias; fallback on all editions): https://github.com/rust-lang/rust/issues/163811
- Tracking issues: never type https://github.com/rust-lang/rust/issues/35121, fallback change https://github.com/rust-lang/rust/issues/148922
- The Rust Edition Guide — Never type fallback change: https://doc.rust-lang.org/edition-guide/rust-2024/never-type-fallback.html
- The Rust Reference — Never type: https://doc.rust-lang.org/reference/types/never.html
- Standard library — primitive `never`: https://doc.rust-lang.org/std/primitive.never.html; `std::convert::Infallible`: https://doc.rust-lang.org/std/convert/enum.Infallible.html

- Rust Project Goals 2026 — Stabilize never type: https://goals.rust-lang.org/2026/stabilize-never-type.html

- LWN, “Stabilizing Rust’s never type” (2026-09-08): https://lwn.net/Articles/1091015/

Summary

Summary

Chapter Key Takeaways

- `panic!` is reserved for bugs and unrecoverable states.
- `Result<T, E>` and `Option<T>` model recoverable failure and absence.
- The `?` operator provides zero-cost error propagation.
- Custom error enums unify error types across modules.

Generics, Traits, and Associated Types

Generic Functions and Data Structures

Generic Functions and Data Structures

Generics allow writing flexible, reusable code with monomorphized zero runtime overhead.

```
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

fn swap<T>(a: T, b: T) -> (T, T) {
    (b, a)
}

fn main() {
    let p = Pair { first: 10, second: 20 };
    let (x, y) = swap("hello", "world");
    println!("Pair: {:?}, Swapped: ({}, {})", p, x, y);
}
```

Defining and Implementing Traits

Defining and Implementing Traits

Traits define shared behavior analogous to interfaces:

```
pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct Article {
    pub headline: String,
}

impl Summary for Article {
    fn summarize(&self) -> String {
        format!("Headline: {}", self.headline)
    }
}

fn main() {
    let a = Article { headline: "Rust 2026 Ships".into() };
    println!("{}", a.summarize());
}
```

Trait Bounds and Where Clauses

Trait Bounds and Where Clauses

Constrain generics using trait bounds and `where` clauses:

```
use std::fmt::Display;

fn print_and_return<T>(val: T) -> T
where
    T: Display + Clone,
{
    println!("Value: {}", val);
    val.clone()
}

fn main() {
    let res = print_and_return(42);
    println!("Returned: {}", res);
}
```

Associated Types vs Generics

Associated Types vs Generics

Associated types simplify trait definitions when there is a unique type output per implementation:

```
trait DataSink {
    type Error;
    fn send(&mut self, bytes: &[u8]) -> Result<(), Self::Error>;
}
```

Trait Objects and Dynamic Dispatch

Trait Objects and Dynamic Dispatch

When heterogeneous collections are required at runtime, use dynamic dispatch via trait objects (`dyn Trait`):

```
trait Drawable {
    fn draw(&self);
}

struct Button;
impl Drawable for Button {
    fn draw(&self) { println!("Drawing button"); }
}

struct Checkbox;
impl Drawable for Checkbox {
    fn draw(&self) { println!("Drawing checkbox"); }
}

fn render_all(items: &[Box<dyn Drawable>]) {
    for item in items {
        item.draw();
    }
}

fn main() {
    let widgets: Vec<Box<dyn Drawable>> = vec![
        Box::new(Button),
        Box::new(Checkbox),
    ];
    render_all(&widgets);
}
```

Generic Const Arguments (GCA)

Generic Const Arguments (GCA)

Thesis

On stable Rust, a const generic parameter may appear in a const argument **alone** (`[u8; N]`) or as a fully concrete expression (`[u8; 1 + 1]`). The moment you write `[u8; N + 1]` or `[u8; T::ASSOC]`, the compiler stops you. The old escape hatch, `#![feature(generic_const_exprs)]` (GCE), taught everyone the shape of the problem but never became a stabilization candidate.

Generic Const Arguments (GCA) is the Const Generics project group's replacement family of nightly features. Expressions that involve generic parameters go through an explicit `gca!(…)` marker (for now), each capability has its own gate (`gca_adts`, `gca_min_const_items`, `gca_const_items`, …), and definitional equality — not GCE's evaluation model — decides when two const arguments match. Nothing here is stable. This chapter shows what compiles on **today's nightly**, marks every feature gate, and records where the Oct 2026 blog post already drifted from the compiler.

Version note: verified on **`rustc 1.101.0-nightly (a30aa9064 2026-10-08)`**. The Inside Rust post *Generic Const Args and You* (2026-10-02) is the design overview; on this nightly the feature formerly named `generic_const_args` has been **renamed to `gca_const_items`** (removed name errors say "removed in 1.100.0"), and the `gca!` macro lives at **`std::gca`** / `core::gca`. No RFC yet; the project group says polish first.

Stable const generics basics (structs/functions parameterized by `const N: usize`) live in *Generic Functions and Data Structures*. This chapter does not re-teach that.

Mental model

```
Stable today
  const arg ::=  N  |  <fully concrete expr>
                   generic param alone

GCE (legacy nightly)     - evaluate complex exprs; hard to stabilize
GCA (current nightly)    - mark exprs with gca!(…); equality by definition

          gca_adts              arrays / tuples / structs / enums in gca!(…)
          gca_min_const_items   const items whose RHS is gca!(…)
          gca_const_items       + opaque const items, assoc bindings, dyn, needs -Znext-solve
          gca_macroless_*       experimental: insert gca!(…) for you (rough)
```

Fact	Consequence
`gca!(…)` may mention generic parameters	Plain `{ N + 1 }` as a const argument still cannot
`gca!(…)` accepts **fewer** expression kinds than a normal const block	`gca!(N + 1)` and `gca!(foo())` are *not* automatically legal; prefer const items / ADTs that the gates support
`gca_const_items` requires `gca_min_const_items`	Enable both; incomplete-feature warnings are expected
Feature `generic_const_args` is gone on 1.101 nightly	Use `gca_const_items` (compiler points the rename)
Nothing is stabilization-ready	APIs that expose `gca!(…)` in public signatures are a SemVer hazard while the syntax is explicit

Worked examples

All listings were compiled with:

```
rustup toolchain install nightly --profile minimal   # once
rustc +nightly --version
```

```
rustc 1.101.0-nightly (a30aa9064 2026-10-08)
```

Case 0: stable still rejects `N + 1`

```
// stable_fail.rs - what stable const generics refuse
fn make<const N: usize>() -> [u8; N + 1] {
    [0; N + 1]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc --edition 2024 stable_fail.rs
```

```
error: generic parameters may not be used in const operations
 --> stable_fail.rs:1:35
  |
1 | fn make<const N: usize>() -> [u8; N + 1] {
  |                                   ^ cannot perform const operation using `N`
  |
  = help: const parameters may only be used as standalone arguments here, i.e. `N`
```

```
  = help: add `#![feature(generic_const_exprs)]` to allow generic const expressions
  = help: alternatively, you can use `#![feature(gca_const_items)]` and extract the expressi
```

Note the help text still mentions GCE *and* GCA — both are nightly escape hatches; new code should reach for GCA, not GCE.

Case 1: ADT const arguments with `gca_adts`

Build a const-generic parameter whose type is a struct, then construct that struct from another const parameter inside `gca!(…)`:

```rust
// gca_adt.rs - struct expressions as const arguments (nightly GCA)
#![feature(gca_adts, adt_const_params, gca_min_const_items)]
#![expect(incomplete_features)]

use std::gca;
use std::marker::ConstParamTy;

#[derive(ConstParamTy, Eq, PartialEq)]
struct TupleStruct(usize);

fn accepts<const N: TupleStruct>() -> usize {
    N.0
}

fn example<const N: usize>() -> usize {
    accepts::<gca!(TupleStruct(N))>()
}

fn main() {
    println!("{}", example::<7>());
}
```

```
rustc +nightly --edition 2024 gca_adt.rs -o gca_adt && ./gca_adt
```

```
7
```

The same pattern works for enum variants, arrays (`gca!([N1, 12])`), and tuples (`gca!((N1, 12))`) per the project-group post. Array *repeat* syntax (`gca!([N1; 2])`) is still explicitly **not** supported.

Case 2: Associated constants with `gca_const_items`

```rust
// gca_assoc.rs - associated consts in array lengths (nightly GCA)
#![feature(gca_min_const_items, gca_const_items)]
#![expect(incomplete_features)]

use std::gca;

trait Trait {
    const ASSOC: usize;
}

impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn example<T: Trait>() -> [u8; gca!(T::ASSOC)] {
    [0x1; _]
}

fn main() {
    let a = example::<u8>();
    println!("len={}", a.len());
}
```

```
rustc +nightly --edition 2024 gca_assoc.rs -Znext-solver -o gca_assoc && ./gca_assoc
```

```
len=3
```

`-Znext-solver` is required for the full `gca_const_items` path in current nightlies. Treat that as part of the experiment, not as something to bake into stable CI.

Case 3: Macroless (experimental)

`gca_macroless_args` asks the compiler to insert `gca!(…)` for you. It works for some associated-const cases on this nightly, and the project group still recommends writing `gca!(…)` explicitly:

```rust
// gca_macroless.rs - implicit gca!(…) (rough; prefer explicit)
#![feature(gca_min_const_items, gca_const_items, gca_macroless_args)]
#![expect(incomplete_features)]

trait Trait {
    const ASSOC: usize;
}
```

```rust
impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn make_array<T: Trait>() -> [u8; T::ASSOC] {
    [0x1; _]
}

fn main() {
    println!("{}", make_array::<u8>().len());
}
```

```
rustc +nightly --edition 2024 gca_macroless.rs -Znext-solver -o gca_macroless && ./gca_macro
```

```
3
```

Case 4: What still hurts — arithmetic inside `gca!`

The blog's long-term story is "put complex ops in const items." On 1.101 nightly, a direct `gca!(N + 1)` as a const-item RHS is still rejected:

```rust
// gca_arith.rs - not there yet on 2026-10-08 nightly
#![feature(gca_min_const_items, gca_const_items, generic_const_items)]
#![expect(incomplete_features)]
use std::gca;

const INC<const N: usize>: usize = gca!(N + 1);

fn make<const N: usize>() -> [u8; INC::<N>] {
    [0; INC::<N>]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc +nightly --edition 2024 gca_arith.rs
```

```
error: complex const arguments must be placed inside of a `const` block
 --> gca_arith.rs:5:41
  |
5 | const INC<const N: usize>: usize = gca!(N + 1);
  |                                         ^^^^^
```

So: ADTs and associated-const *uses* are the productive surface today; "any arithmetic via `gca!`" is

not. Re-check nightly before teaching a team pattern around arithmetic helpers.

The trap

1. **Copy-pasting `#![feature(generic_const_args)]` from older posts.** Removed in 1.100; rename to `gca_const_items` and enable `gca_min_const_items` with it.
2. **Assuming `gca!(…)` is a full const evaluator.** It is a marker for a *restricted* expression grammar that may mention generics. Unsupported ops fail loudly.
3. **Stabilizing public APIs on `gca!`.** The macroless work exists precisely because explicit `gca!` is considered ergonomically temporary — and also SemVer-visible.
4. **Mixing GCE and GCA in one crate "just to make it compile".** Two incomplete models; pick GCA for new experiments.
5. **Forgetting `#![expect(incomplete_features)]` / `#[allow(incomplete_features)]`.** Without it, every build is a warning storm (or worse, depending on lint config).

The boring rule

Stay on stable const generics (`N` alone, or concrete exprs) for production. When you truly need generic const computation, use **current nightly GCA** with explicit `gca!(…)`, gate every file with the specific features you touch, pin the nightly date in the chapter/PR, and re-run the listings after each toolchain bump — the rename from `generic_const_args` to `gca_const_items` already proved the surface moves.

Try this

1. Compile Case 1 and Case 2 on your nightly; paste `rustc +nightly --version` next to the output.
2. Attempt one `gca!([N; 2])` repeat expression and confirm it still errors; watch the tracking issues for when it lands.
3. Skim the project-const-generics repo / Zulip before filing duplicates — design feedback is solicited there.

Sources

- Generic Const Args and You — Inside Rust, 2026-10-02 (inspiration; see *Updated vs original* in the PR)
- generic_const_args — Unstable Book (tracking issue #151972)
- Full Const Generics — 2026 project goal
- Verified locally: `rustc 1.101.0-nightly (a30aa9064 2026-10-08)`

AsRef, Borrow and Deref: Same Signature, Different Promises

`AsRef`, `Borrow` and `Deref`: Same Signature, Different Promises

Thesis

Three standard traits all turn `&Self` into `&T`:

```
pub trait AsRef<T: ?Sized>  { fn as_ref(&self) -> &T; }
pub trait Borrow<T: ?Sized> { fn borrow(&self) -> &T; }
pub trait Deref             { type Target: ?Sized; fn deref(&self) -> &Self::Target; }
```

They are not interchangeable. The difference is what each one **promises** to the code that uses it.

- **`AsRef<T>`** promises only a cheap conversion. Use it as a generic bound, such as `P: AsRef<Path>`, to accept many input types.
- **`Borrow<T>`** promises the same conversion **and** that "`Eq`, `Ord` and `Hash` must be equivalent for borrowed and owned values" (std docs). That promise is the only reason `HashMap<String, V>::get` can take a `&str`. The map hashes the `&str` you pass and trusts it to land in the same bucket as the `String` it stored.
- **`Deref`** drives the `*` operator and auto-deref method lookup. The std docs reserve it for types that "transparently behave like a value of the target type".

The compiler checks the signatures of all three. It doesn't check the promises. This chapter shows each trait doing its job, then breaks `Borrow`'s contract on purpose. The result is a map that compiles, passes a lowercase-only test, and loses `"Content-Type"`.

Mental model

	`AsRef<T>`	`Borrow<T>`	`Deref`
What it promises	a cheap `&T`	a cheap `&T` whose `Hash`/`Eq`/`Ord` match `Self`'s	`*x` and method calls go to `Target`
Reflexive (`T: Trait<T>`)?	**no**	yes (blanket `impl<T> Borrow<T> for T`)	n/a
Several targets per type?	yes (`String`: `AsRef<str>`, `AsRef<[u8]>`, `AsRef<Path>`, `AsRef<OsStr>`)	yes, if each keeps the contract	no, exactly one

	`AsRef<T>`	`Borrow<T>`	`Deref`
May lend just one field?	yes	no (the field's `Hash`/`Eq` would differ)	not recommended
Typical user	`fn open<P: AsRef<Path>>(p: P)`	`HashMap::get`, `BTreeMap::range`, `Cow`/`ToOwned`	`Box<T>`, `String`, `Vec<T>`, guards

Worked examples

Built and run with **rustc/cargo 1.85.1** (the distro toolchain on the box), edition 2024. The current stable Rust is **1.99.1**. Every API used here has been stable since well before 1.85, and its documentation is unchanged. The project has no dependencies.

```
cargo new --vcs none conv && cd conv
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "conv"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
```

Case 1: How `HashMap<String, V>` answers a `&str`

```
// src/bin/lookup.rs - why HashMap<String, V>::get accepts a &str
use std::borrow::Borrow;
use std::collections::{BTreeMap, HashMap};
use std::hash::{BuildHasher, RandomState};
use std::ops::Bound::{Excluded, Included};

fn main() {
    let mut ports: HashMap<String, u16> = HashMap::new();
    ports.insert("http".to_string(), 80);
    ports.insert("https".to_string(), 443);

    // get<Q: ?Sized>(&self, k: &Q) where String: Borrow<Q>, Q: Hash + Eq
    let key: &str = "https";
    println!("get(&str)    -> {:?}", ports.get(key));
    println!("get(&String) -> {:?}", ports.get(&"http".to_string()));
```

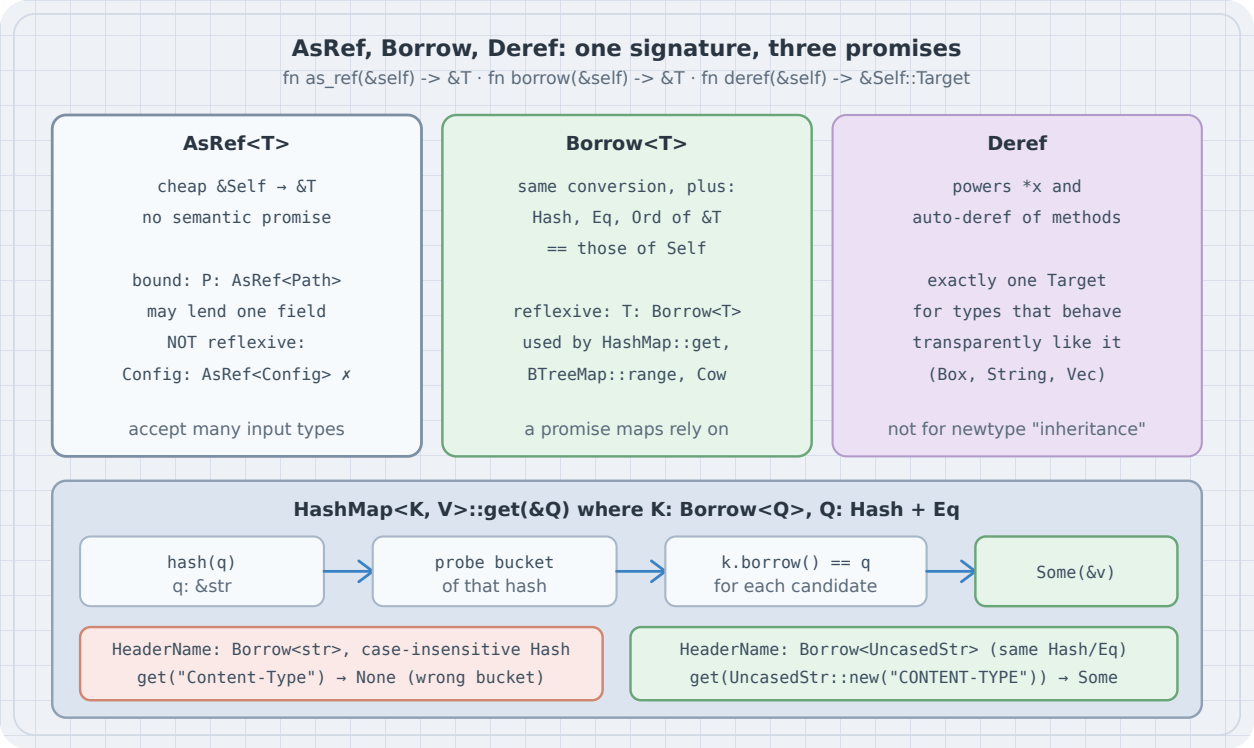


Figure 1: AsRef, Borrow and Deref compared; HashMap::get hashes the borrowed key, probes, compares k.borrow() == q; a case-insensitive Borrow misses, a Borrow with the same Hash/Eq hits

```
    // The contract that makes it sound: hashing the owned key and the borrowed key agree.
    let s = RandomState::new();
    let owned = String::from("https");
    let borrowed: &str = owned.borrow();
    println!("hash(String) == hash(&str)? {}", s.hash_one(&owned) == s.hash_one(borrowed));

    // Ordered maps use the same trick with Ord: range queries by &str over String keys.
    let mut users: BTreeMap<String, u32> = BTreeMap::new();
    for (name, id) in [("alice", 1), ("bob", 2), ("carol", 3), ("dave", 4)] {
        users.insert(name.to_string(), id);
    }
    // `"b".."d"` is a Range<&str>, which is not RangeBounds<str>; spell the bounds out inst
    let picked: Vec<_> = users
        .range::<str, _>((Included("b"), Excluded("d")))
        .map(|(k, v)| (k.as_str(), *v))
        .collect();
    println!("range(b..d)  -> {picked:?}");
}
```

```
$ cargo run -q --bin lookup
get(&str)    -> Some(443)
get(&String) -> Some(80)
hash(String) == hash(&str)? true
range(b..d)  -> [("bob", 2), ("carol", 3)]
```

`HashMap::get` is declared as `get<Q: ?Sized>(&self, k: &Q) -> Option<&V> where K: Borrow<Q>, Q: Hash + Eq`. With `K = String` and `Q = str`, the map hashes the `&str`, finds the bucket, and compares each candidate with `key.borrow() == q`. No `String` is allocated for the lookup. The `hash_one` line shows the property that makes this correct: `String` and `str` produce the same hash, because `String`'s `Hash` forwards to `str`'s. `BTreeMap` uses the same bound with `Ord` for `get` and `range`. (The `range` call needs its bounds spelled out, because `"b".."d"` is a `Range<&str>`, not a `RangeBounds<str>`.)

Case 2: `AsRef` as an "accept anything" bound

```
// src/bin/asref.rs - AsRef as a cheap "accept anything that can lend me a &T" bound
use std::ffi::OsString;
use std::path::{Path, PathBuf};

fn describe<P: AsRef<Path>>(p: P) -> String {
    let p = p.as_ref(); // one conversion, then work with &Path
    format!("{:<18} file_name={:?} ext={:?}", p.display().to_string(), p.file_name(), p.exte
}

fn checksum<B: AsRef<[u8]>>(data: B) -> u32 {
```

```
    data.as_ref().iter().fold(0u32, |a, &b| a.rotate_left(5) ^ b as u32)
}

fn main() {
    println!("{}", describe("notes/today.md"));
    println!("{}", describe(String::from("Cargo.toml")));
    println!("{}", describe(PathBuf::from("/etc/hosts")));
    println!("{}", describe(Path::new("archive.tar.gz")));
    println!("{}", describe(OsString::from("README")));

    // The same bound works for every byte container.
    println!("checksum: {} {} {} {}",
        checksum("abc"), checksum(String::from("abc")), checksum(b"abc"), checksum(vec![b'a'
}
```

```
$ cargo run -q --bin asref
notes/today.md     file_name=Some("today.md") ext=Some("md")
Cargo.toml         file_name=Some("Cargo.toml") ext=Some("toml")
/etc/hosts         file_name=Some("hosts") ext=None
archive.tar.gz     file_name=Some("archive.tar.gz") ext=Some("gz")
README             file_name=Some("README") ext=None
checksum: 100387 100387 100387 100387
```

`AsRef<Path>` is implemented for `str`, `String`, `OsStr`, `OsString`, `Path`, `PathBuf` and more, and `&T` forwards to `T`. One generic signature accepts all of them, and `.as_ref()` happens once at the top of the function. This is the pattern `std::fs::File::open` uses. `AsRef` makes no promise about hashing or equality, so a type is free to implement it for several targets, or to lend out just one of its fields. `Borrow` can't do either of those.

Case 3: Break the `Borrow` contract and the map loses keys

HTTP header names are case-insensitive. A natural first attempt is a newtype with case-insensitive `Eq` and `Hash`, plus `Borrow<str>` so that lookups by `&str` "just work":

```
// src/bin/broken_borrow.rs - a case-insensitive key that implements Borrow<str> (wrongly)
use std::borrow::Borrow;
use std::collections::HashMap;
use std::hash::{Hash, Hasher};

#[derive(Debug, Clone)]
struct HeaderName(String);

impl PartialEq for HeaderName {
    fn eq(&self, other: &Self) -> bool {
        self.0.eq_ignore_ascii_case(&other.0)
    }
```

```
}
impl Eq for HeaderName {}

impl Hash for HeaderName {
    fn hash<H: Hasher>(&self, state: &mut H) {
        for b in self.0.bytes() {
            state.write_u8(b.to_ascii_lowercase()); // hash the lowercase form
        }
        state.write_u8(0xff);
    }
}

// Compiles fine. Breaks the contract: str's Hash/Eq are case-SENSITIVE and hash differently
impl Borrow<str> for HeaderName {
    fn borrow(&self) -> &str {
        &self.0
    }
}

fn main() {
    let mut h: HashMap<HeaderName, &str> = HashMap::new();
    h.insert(HeaderName("Content-Type".into()), "text/html");

    println!("get(&HeaderName(\"content-type\")) -> {:?}", h.get(&HeaderName("content-type".
    println!("get(\"content-type\")              -> {:?}", h.get("content-type"));
    println!("get(\"Content-Type\")              -> {:?}", h.get("Content-Type"));
    println!("contains_key(\"Content-Type\")     -> {}", h.contains_key("Content-Type"));

    // Why tests often miss it: an all-lowercase key hashes the same both ways.
    h.insert(HeaderName("accept".into()), "*/*");
    println!("get(\"accept\")                    -> {:?}", h.get("accept"));
}
```

```
$ cargo run -q --bin broken_borrow
get(&HeaderName("content-type")) -> Some("text/html")
get("content-type")              -> None
get("Content-Type")              -> None
contains_key("Content-Type")     -> false
get("accept")                    -> Some("*/*")
```

It compiles without a warning, and it is wrong in a way that is easy to miss:

- Lookups by `&HeaderName` work, in any case.
- Lookups by `&str` go through `str`'s `Hash`, which is case-*sensitive*. `"content-type"` and even the exact `"Content-Type"` hash into a different bucket from the stored key, so the lookup returns `None`.
- For an all-lowercase key, both hashes see the same bytes. std's `str` hash writes the bytes

followed by `0xff`, and so does this `HeaderName` impl. (That is an implementation detail of the default `Hasher::write_str` and std's SipHasher, not a guarantee.) So `get("accept")` succeeds. A test suite that only uses lowercase names passes.

The std docs are explicit that this is a logic error, not undefined behaviour. `HashMap` stays memory-safe, but its results stop meaning anything. That is why `Borrow` is a safe trait, and also why the compiler can't help you here.

Case 4: The correct shape: borrow as a type with the same `Hash` and `Eq`

The fix is to *borrow as* a type whose `Hash` and `Eq` really are the key's. Here that is an unsized, case-insensitive view of `str`, built the same way std builds `Path` over `OsStr`:

```
// src/bin/uncased.rs - the correct shape: borrow as a str-like type with the SAME Hash/Eq
use std::borrow::Borrow;
use std::collections::HashMap;
use std::hash::{Hash, Hasher};

/// A borrowed, case-insensitive view of a `str` (an unsized type, like `Path` over `OsStr`)
#[repr(transparent)]
#[derive(Debug)]
struct UncasedStr(str);

impl UncasedStr {
    fn new(s: &str) -> &UncasedStr {
        // SAFETY: UncasedStr is #[repr(transparent)] over str, so &str and &UncasedStr have
        // the same layout and metadata; this is the pattern std uses for Path::new.
        unsafe { &*(s as *const str as *const UncasedStr) }
    }
}

impl PartialEq for UncasedStr {
    fn eq(&self, other: &Self) -> bool {
        self.0.eq_ignore_ascii_case(&other.0)
    }
}
impl Eq for UncasedStr {}

impl Hash for UncasedStr {
    fn hash<H: Hasher>(&self, state: &mut H) {
        for b in self.0.bytes() {
            state.write_u8(b.to_ascii_lowercase());
        }
        state.write_u8(0xff);
    }
}
```

```
/// The owned key. Its Hash/Eq simply delegate to the borrowed form, so they agree by constr
#[derive(Debug, Clone)]
struct HeaderName(String);

impl Borrow<UncasedStr> for HeaderName {
    fn borrow(&self) -> &UncasedStr {
        UncasedStr::new(&self.0)
    }
}
impl PartialEq for HeaderName {
    fn eq(&self, other: &Self) -> bool {
        Borrow::<UncasedStr>::borrow(self) == Borrow::<UncasedStr>::borrow(other)
    }
}
impl Eq for HeaderName {}
impl Hash for HeaderName {
    fn hash<H: Hasher>(&self, state: &mut H) {
        Borrow::<UncasedStr>::borrow(self).hash(state)
    }
}

fn main() {
    let mut h: HashMap<HeaderName, &str> = HashMap::new();
    h.insert(HeaderName("Content-Type".into()), "text/html");

    for probe in ["content-type", "CONTENT-TYPE", "Content-Type", "Content-Length"] {
        println!("get(UncasedStr::new({probe:?})) -> {:?}", h.get(UncasedStr::new(probe)));
    }
}
```

```
$ cargo run -q --bin uncased
get(UncasedStr::new("content-type")) -> Some("text/html")
get(UncasedStr::new("CONTENT-TYPE")) -> Some("text/html")
get(UncasedStr::new("Content-Type")) -> Some("text/html")
get(UncasedStr::new("Content-Length")) -> None
```

Now the owned key's `Hash` and `Eq` *delegate* to the borrowed form, so they agree by construction, and every case of the name finds the entry. The single `unsafe` line is the standard `#[repr(transparent)]` reference cast that `Path::new` and `OsStr::new` use. Miri was not run on it here, because there is no nightly toolchain on the box. If you'd rather avoid the cast, lowercase the key on insert and look it up with a lowercased `&str`. That costs an allocation, or a check, per lookup.

Case 5: `Borrow` is reflexive, `AsRef` is not

```rust
// src/bin/reflexive.rs - Borrow<T> holds for T itself; AsRef<T> does not
use std::borrow::Borrow;

#[derive(Debug)]
struct Config {
    name: String,
}

fn by_borrow<C: Borrow<Config>>(c: C) -> usize {
    c.borrow().name.len()
}

fn by_asref<C: AsRef<Config>>(c: C) -> usize {
    c.as_ref().name.len()
}

fn main() {
    let cfg = Config { name: "prod".into() };
    println!("by_borrow(&cfg) = {}", by_borrow(&cfg)); // impl Borrow<T> for &T
    println!("by_borrow(cfg)  = {}", by_borrow(cfg)); //  impl Borrow<T> for T
    let cfg = Config { name: "dev".into() };
    println!("by_asref(&cfg)  = {}", by_asref(&cfg));
}
```

```
$ cargo build --bin reflexive
   Compiling conv v0.1.0 (/tmp/borrow/conv)
error[E0277]: the trait bound `Config: AsRef<Config>` is not satisfied
  --> src/bin/reflexive.rs:22:48
   |
22 |     println!("by_asref(&cfg)  = {}", by_asref(&cfg));
   |                                       --------  ^^^ the trait `AsRef<Config>` is not imp
   |                                       |
   |                                       required by a bound introduced by this call
   |
   = note: required for `&Config` to implement `AsRef<Config>`
note: required by a bound in `by_asref`
  --> src/bin/reflexive.rs:13:16
   |
13 | fn by_asref<C: AsRef<Config>>(c: C) -> usize {
   |                ^^^^^^^^^^^^^ required by this bound in `by_asref`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `conv` (bin "reflexive") due to 1 previous error
```

`by_borrow` accepts both `cfg` and `&cfg`. The blanket impls `impl<T> Borrow<T> for T` and `impl<T>`

`Borrow<T> for &T` cover both. `by_asref(&cfg)` fails: `&T: AsRef<U>` only forwards when `T: AsRef<U>`, and nothing makes `Config: AsRef<Config>`. The `AsRef` docs explain that a reflexive blanket impl "is currently not provided due to technical restrictions of Rust's type system (it would be overlapping with another existing blanket implementation for `&T…`)". If you want generic code that accepts "a `T` or a `&T`", write `C: Borrow<T>`. Use `AsRef` for "anything that can lend me a `&T`".

Case 6: `ToOwned` is `Borrow` run backwards, and `Cow` stands on both

```
// src/bin/cow.rs - ToOwned is Borrow run backwards, and Cow is built on both
use std::borrow::Cow;

/// Normalise a header value: trim it, and collapse internal runs of spaces.
/// Allocates only when the input actually needs changing.
fn normalize(v: &str) -> Cow<'_, str> {
    let t = v.trim();
    if !t.contains("  ") {
        return Cow::Borrowed(t); // zero-copy: a slice of the caller's string
    }
    let mut out = String::with_capacity(t.len());
    for word in t.split(' ').filter(|w| !w.is_empty()) {
        if !out.is_empty() {
            out.push(' ');
        }
        out.push_str(word);
    }
    Cow::Owned(out)
}

fn main() {
    for input in ["text/html", "  text/html ", "text/html;  charset=utf-8"] {
        let n = normalize(input);
        let kind = match &n {
            Cow::Borrowed(_) => "borrowed",
            Cow::Owned(_) => "owned (allocated)",
        };
        println!("{input:<28} -> {n:<26} {kind}");
    }

    // <str as ToOwned>::Owned = String, and String: Borrow<str>: the pair Cow<str> relies o
    let owned: String = "abc".to_owned();
    let back: &str = std::borrow::Borrow::borrow(&owned);
    println!("to_owned -> {owned:?}, borrow -> {back:?}");

    // into_owned() allocates only if it still has to.
    let c: Cow<'_, str> = normalize("plain");
```

```
    let s: String = c.into_owned();
    println!("into_owned -> {s:?}");
}
```

```
$ cargo run -q --bin cow
text/html                    -> text/html                    borrowed
  text/html                  -> text/html                    borrowed
text/html;  charset=utf-8    -> text/html; charset=utf-8     owned (allocated)
to_owned -> "abc", borrow -> "abc"
into_owned -> "plain"
```

`Cow<'a, B>` requires `B: ToOwned`, and `ToOwned` requires `Self::Owned: Borrow<Self>`. That pair lets a `Cow<str>` hold either a `&str` or a `String` and deref to `&str` either way. `normalize` allocates only for the input that needs rewriting. The other two return a slice of the caller's string.

The trap

The trap is treating `Borrow` as "`AsRef` with a better name for maps", and implementing it whenever you want `get(&str)` to compile. Case 3 compiles and passes a lowercase test, and in production it returns `None` for `"Content-Type"`. The same class of bug appears whenever `Borrow<T>` lends a form whose `Hash`, `Eq` or `Ord` differ from the owner's:

- a struct that borrows as one of its fields (`User: Borrow<u64>` returning `&self.id`, while `User`'s `Eq` compares every field);
- a float wrapper with a total order that borrows as `f64` (whose `PartialOrd` disagrees for NaN);
- a path-like key that normalises on `Eq` but borrows as the raw string.

The related `Deref` trap is newtype "inheritance": `struct Email(String)` with `Deref<Target = String>` so that `email.len()` works. Every `String` method becomes part of the `Email` API, `email.to_uppercase()` silently returns a plain `String`, and the std docs' conditions ("transparently behaves like a value of the target type", "will not be surprised by any deref coercion behavior") no longer hold. Implement `AsRef<str>` and a few explicit methods instead.

The boring rule

> Use `AsRef<T>` for generic parameters that just need to read a `&T` (paths, bytes, strings). Implement `Borrow<T>` only when `T`'s `Hash`, `Eq` and `Ord` agree exactly with `Self`'s, and get there by delegating `Self`'s impls *to* the borrowed form. If they can't agree (case-insensitive, normalised or partial keys), borrow as a dedicated `#[repr(transparent)]` view type, or normalise the key before you store it. Implement `Deref` only for smart-pointer-like types, never to inherit methods. Test map lookups with keys in more than one form.

Try this

1. Add `impl AsRef<Config> for Config { fn as_ref(&self) -> &Config { self } }` to `reflexive.rs`. Does `by_asref(&cfg)` compile now? Does `by_asref(cfg)`?
2. In `broken_borrow.rs`, replace `HashMap` with `BTreeMap`, give `HeaderName` a case-insensitive `Ord`, and keep `Borrow<str>`. Which lookups fail now, and why is the failure different from the hash-based one?
3. Write `fn total_len<I, S>(items: I) -> usize where I: IntoIterator<Item = S>, S: AsRef<str>` and call it with a `Vec<String>`, a `[&str; 3]`, and a `HashSet<Box<str>>`.
4. Change `normalize` in `cow.rs` to lowercase the value too. How many inputs are still `Borrowed`? Use `str::bytes().any(|b| b.is_ascii_uppercase())` so that it only allocates when needed.
5. Find `impl Borrow<str> for String` and `impl Hash for String` in the std source. Confirm that `String` hashes exactly as its `str` does, and explain why that equality is the whole contract.

Sources

- Rust standard library, `std::borrow::Borrow` (the `Hash`/`Eq`/`Ord` equivalence requirement and the `HashMap` example): https://doc.rust-lang.org/std/borrow/trait.Borrow.html
- Rust standard library, `std::convert::AsRef` ("Relation to `Borrow`", "Reflexivity"): https://doc.rust-lang.org/std/convert/trait.AsRef.html
- Rust standard library, `std::ops::Deref` ("When to implement `Deref` or `DerefMut`"): https://doc.rust-lang.org/std/ops/trait.Deref.html
- Rust standard library, `HashMap::get` and `BTreeMap::range` signatures: https://doc.rust-lang.org/std/collections/struct.HashMap.html#method.get, https://doc.rust-lang.org/std/collections/struct.BTreeMap.html#method.range
- Rust standard library, `std::borrow::ToOwned` and `Cow`: https://doc.rust-lang.org/std/borrow/trait.ToOwned.html, https://doc.rust-lang.org/std/borrow/enum.Cow.html
- Rust standard library, `Path::new` (the `#[repr(transparent)]` unsized-view pattern): https://doc.rust-lang.org/std/path/struct.Path.html#method.new
- Rust source, `core::hash` (`Hasher::write_str` writes the bytes then `0xff`): https://github.com/rust-lang/rust/blob/master/library/core/src/hash/mod.rs
- The Rust Reference, type layout: `#[repr(transparent)]`: https://doc.rust-lang.org/reference/type-layout.html#the-transparent-representation
- Rust API Guidelines, C-DEREF ("Only smart pointers implement `Deref`"): https://rust-lang.github.io/api-guidelines/predictability.html#c-deref
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

Cow — Borrow When You Can, Own When You Must

Cow — Borrow When You Can, Own When You Must

Thesis

`Cow<'_, str>` (clone-on-write) returns a borrowed slice when no change is needed and an owned `String` when you must mutate or allocate. The win appears when the common path is already normalized. This chapter compares always-`to_string()` against `Cow` with `black_box` so LLVM cannot erase the work.

Mental model

```
    input already good → Cow::Borrowed(&str)   // no alloc
    input needs change → Cow::Owned(String)    // alloc once
```

Worked examples

Verified with **rustc 1.99.1** `--release`.

Output (`200_000` iterations, `black_box` on input and result):

```
   Compiling cowdemo v0.1.0 (/home/box/sand-data/books-scout/runs/2026-10-27-oneshot/labs/rust-
     Running `target/release/cowdemo`
owned always, already-lower x200000: 6.124085ms sink=0
cow borrow, already-lower x200000: 3.849675ms sink=0
cow owned, mixed-case x200000: 3.966889ms sink=0
```

On this host the borrow path (~3.85 ms) beat always-owned (~6.12 ms) for already-lowercase input. Mixed-case `Cow::Owned` stayed in the same band as borrow here because the `chars().any` scan dominates the small string alloc.

Trap

Wrapping everything in `Cow` by habit. If you almost always allocate, a plain `String` API is clearer. `Cow` pays for the enum branch and lifetime plumbing.

Boring rule

Use `Cow` at API boundaries where callers often pass already-valid data (paths, headers, normalized IDs). Measure with `black_box` before celebrating.

Try this

1. Return `Cow<'_, [u8]>` from a decoder that sometimes copies.
2. Compare `String::from` vs `Cow` in a serializer hot path.
3. Read `std::borrow::Cow` examples in the docs.

Sources

- std::borrow::Cow

Summary

Summary

Chapter Key Takeaways

- Generics monomorphize at compile time for zero runtime overhead.
- Traits declare shared contracts across concrete types.
- Trait bounds enforce capabilities on generic type parameters.
- Trait objects (`dyn Trait`) provide dynamic dispatch using vtables when heterogeneous collections are needed.

Lifetimes: Complete Understanding

What Lifetimes Are (and Are Not)

What Lifetimes Are (and Are Not)

Every reference in Rust has a **lifetime**—the scope for which that reference is valid.

What Lifetimes Are Not

- Lifetimes do **not** change how long a value lives at runtime.
- Lifetimes do **not** allocate or free memory.
- Lifetimes are purely compile-time annotations that help the borrow checker ensure references never outlive the data they point to.

The Lifetime Elision Rules

The Lifetime Elision Rules

In common function patterns, Rust automatically infers lifetime parameters via three elision rules:

1. Each parameter that is a reference gets its own lifetime parameter (`fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`).
2. If there is exactly one input lifetime parameter, that lifetime is assigned to all output lifetime parameters (`fn foo<'a>(x: &'a i32) -> &'a i32`).
3. If there are multiple input lifetime parameters, but one of them is `&self` or `&mut self`, the lifetime of `self` is assigned to all output lifetime parameters.

Annotating Lifetimes in Functions and Structs

Annotating Lifetimes in Functions and Structs

When a function returns a reference that could come from multiple inputs, explicit lifetime annotations are required:

```
// The returned reference is guaranteed valid for the intersection of 'a
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

struct Excerpt<'a> {
    part: &'a str,
}

fn main() {
    let string1 = String::from("desk configuration");
    let result;
    {
        let string2 = String::from("orders");
        result = longest(string1.as_str(), string2.as_str());
        println!("Longest string: {}", result);
    }
}
```

Lifetime Bounds on Traits

Lifetime Bounds on Traits

Constrain types with lifetime bounds, such as `T: 'a` (meaning `T` must outlive lifetime `'a`):

```
use std::fmt::Display;

fn print_ref<'a, T>(item: &'a T)
where
    T: Display + 'a,
{
    println!("Item: {}", item);
}
```

Advanced Lifetime Patterns

Advanced Lifetime Patterns

The `'static` Lifetime

A reference with `'static` lifetime can live for the entire duration of the program: - String literals: `&'static str` stored in the read-only data segment. - Owned values promoted to static duration.

Summary

Summary

Chapter Key Takeaways

- Lifetimes allow the compiler to prove reference validity at compile time.
- Elision rules save annotations in common single-reference or method signatures.
- Structs holding references must declare lifetime parameters (`Struct<'a>`).
- `'static` denotes references valid for the entire program runtime.

Smart Pointers and Interior Mutability

Box for Heap Allocation

Box for Heap Allocation

`Box<T>` allocates a value on the heap and provides a stack pointer with ownership semantics.

```
fn main() {
    let b = Box::new(42);
    println!("Box points to heap value: {}", *b);

    // Recursive data types require Box to have a known size:
    #[derive(Debug)]
    enum List {
        Cons(i32, Box<List>),
        Nil,
    }

    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    println!("Recursive list: {:?}", list);
}
```

Reference Counting with Rc and Arc

Reference Counting with Rc and Arc

When multiple parts of a program need shared read ownership of heap memory: - `Rc<T>`: Single-threaded reference counting. - `Arc<T>`: Atomic reference counting across multiple threads.

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let data_clone = Arc::clone(&data);
    let handle = thread::spawn(move || {
        println!("Thread sees: {:?}", data_clone);
    });

    handle.join().unwrap();
    println!("Main sees: {:?}", data);
}
```

Interior Mutability with RefCell

Interior Mutability with RefCell

`RefCell<T>` enforces Rust's borrowing rules dynamically at runtime rather than at compile time:

```
use std::cell::RefCell;

fn main() {
    let val = RefCell::new(10);
    {
        let mut mut_borrow = val.borrow_mut();
        *mut_borrow += 5;
    }
    println!("Mutated value: {}", *val.borrow());
}
```

Cell and Mutex for Shared State

Cell and Mutex for Shared State

- `Cell<T>`: Interior mutability for `Copy` types via value copying.
- `Mutex<T>`: Thread-safe interior mutability via mutual exclusion locks.

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(0);
    {
        let mut guard = m.lock().unwrap();
        *guard += 1;
    }
    println!("Mutex protected value: {:?}", m.lock().unwrap());
}
```

Weak References and Breaking Cycles

Weak References and Breaking Cycles

Cycles of `Rc` or `Arc` pointers can cause memory leaks. Use `Weak<T>` (downgraded non-owning reference) to break reference cycles:

```
use std::rc::{Rc, Weak};

struct Node {
    parent: Option<Weak<Node>>,
}
```

LazyLock and OnceLock

LazyLock and OnceLock

Global or shared values often need **one-time initialization**: a compiled regex, a config snapshot, a connection pool handle. Before Rust 1.70 / 1.80 you reached for `lazy_static`, `once_cell`, or a hand-rolled `Mutex<Option<T>>`. Today the standard library covers the boring cases:

Type	Module	Stabilized	Role
`OnceLock<T>`	`std::sync`	1.70	Thread-safe cell: write once, then shared reads
`LazyLock<T, F>`	`std::sync`	1.80	`OnceLock` + init closure; initializes on first access
`OnceCell<T>` / `LazyCell<T>`	`std::cell`	1.70 / 1.80	Single-threaded cousins (no `Sync`)

Prefer **`LazyLock`** for `static` values with a pure init function. Prefer **`OnceLock`** when initialization needs arguments, fallible setup, or happens explicitly in `main` / a builder. This chapter sits next to `Mutex` and `Arc` because the shared handle is often wrapped in those types later — but lazy init itself is not a lock around every read after the first.

Mental model

```
first access / set()

  run init exactly once      store T

 later access      &T (no re-init)
```

- Concurrent first accessors: one wins the init; others wait, then see the same `T`.
- If the init closure **panics**, `LazyLock` is **poisoned** for the rest of the process (unrecoverable). Fail fallible work *before* installing into a `LazyLock`, or use `OnceLock` + `Result` you control.
- `static LazyLock<T>` values are not dropped at process exit; the OS reclaims the memory. That is intentional and fine for process-lifetime data.

Worked examples

Case 1: `LazyLock` for a process-wide regex

```
use std::sync::LazyLock;
use regex::Regex;

static EMAIL: LazyLock<Regex> = LazyLock::new(|| {
    Regex::new(r"(?i)^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$").expect("static regex")
});

fn looks_like_email(s: &str) -> bool {
    EMAIL.is_match(s)
}

fn main() {
    assert!(looks_like_email("dev@example.com"));
    assert!(!looks_like_email("not-an-email"));
    println!("ok");
}
```

First call to `looks_like_email` compiles the pattern once. Later calls reuse the same `Regex`. The `expect` is acceptable only for patterns that are compile-time constants you control — never for user-supplied patterns inside a `LazyLock` init.

Add `regex` in `Cargo.toml` for this sample (`regex = "1"`). For a std-only sketch, replace the body with a pure function that builds a `String` or `HashMap`.

Case 2: `OnceLock` when init needs an argument

```
use std::sync::OnceLock;

struct AppConfig {
    api_root: String,
}

fn config() -> &'static AppConfig {
    static CFG: OnceLock<AppConfig> = OnceLock::new();
    CFG.get_or_init(|| AppConfig {
        api_root: std::env::var("API_ROOT").unwrap_or_else(|_| "http://127.0.0.1:8080".into(
    })
}

fn main() {
    println!("api_root={}", config().api_root);
    // Second call returns the same address
```

```
    assert!(std::ptr::eq(config(), config()));
}
```

`get_or_init` runs the closure only if empty. `set` / `get` let you initialize from `main` after CLI parsing:

```
use std::sync::OnceLock;

static CFG: OnceLock<String> = OnceLock::new();

fn main() {
    let root = std::env::args().nth(1).unwrap_or_else(|| "/srv/app".into());
    CFG.set(root).expect("config set once");
    println!("{}", CFG.get().expect("configured"));
}
```

`set` returns `Err(value)` if the cell was already filled — use that to catch double-init bugs in tests.

Case 3: Fallible init without poisoning `LazyLock`

```
use std::sync::OnceLock;

static DB_URL: OnceLock<String> = OnceLock::new();

fn init_db_url() -> Result<(), String> {
    let url = std::env::var("DATABASE_URL").map_err(|_| "DATABASE_URL missing".to_string())?
    if !url.starts_with("postgres://") {
        return Err("DATABASE_URL must start with postgres://".into());
    }
    DB_URL.set(url).map_err(|_| "DB_URL already set".to_string())?;
    Ok(())
}

fn db_url() -> &'static str {
    DB_URL.get().expect("call init_db_url() before use")
}

fn main() {
    if let Err(e) = init_db_url() {
        eprintln!("startup failed: {e}");
        std::process::exit(1);
    }
    println!("using {}", db_url());
}
```

Validate and then `set`. Do not hide I/O errors inside `LazyLock::new(|| …)` if you need a clean startup failure path.

Case 4: Single-threaded `LazyCell`

```rust
use std::cell::LazyCell;

fn main() {
    let cache = LazyCell::new(|| {
        println!("computing");
        21 * 2
    });
    assert_eq!(*cache, 42); // prints "computing"
    assert_eq!(*cache, 42); // silent
}
```

Use `LazyCell` / `OnceCell` inside one thread (or inside a type that is already `!Sync`). Crossing threads wants `LazyLock` / `OnceLock`.

Choosing among the options

Situation	Prefer
`static` value, infallible or `expect`-able init	`LazyLock`
Init needs CLI/env parsed in `main`	`OnceLock` + `set`
Fallible init with structured errors	`OnceLock` + `Result` before `set`
Non-`Sync` local cache	`LazyCell` / `OnceCell`
Mutate after init	Not these cells — use `Mutex` / `RwLock` around the data, or rebuild

`LazyLock` implements `Deref`, so `*LAZY` and method calls work like a plain `&T` after init. `OnceLock` exposes `get`, `get_or_init`, `set`, and related helpers explicitly.

Pitfalls

1. **Heavy work on first request path.** First HTTP request paying for a huge init surprises latency SLOs — warm `LazyLock` during startup (`let _ = &*STATIC;`) if the cost matters.
2. **Panic inside `LazyLock` init.** Poison is permanent for that static. Prefer fallible setup with `OnceLock`.
3. **Storing `Mutex` inside when you only needed one-time init.** After init, readers of an immutable `T` do not need a mutex. Add synchronization only for post-init mutation.
4. **Assuming `Drop` runs for `static LazyLock`.** It does not at process exit. Flush external resources explicitly if required.
5. **Using `once_cell` or `lazy_static` in new code on 1.80+.** Prefer std unless you need an API those crates still uniquely provide.

Try this

1. Build Case 1 (or a std-only `LazyLock<HashMap<…>>`) and print addresses of `&*EMAIL` across two threads with `std::thread::scope` — they must match.
2. Call `CFG.set` twice in Case 2's `set` variant; assert the second call is `Err`.
3. Replace Case 3's `OnceLock` with a `LazyLock` that `expect`s on missing env — then run without `DATABASE_URL` and observe the panic/poison behavior.
4. Skim `std::sync::LazyLock` and `OnceLock` for `get_mut` / `take` (where available on your MSRV) before using them in libraries.

Sources

- `std::sync::LazyLock` (1.80+)
- `std::sync::OnceLock` (1.70+)
- `std::cell::LazyCell` / `OnceCell`

MaybeUninit and Uninitialized Memory

MaybeUninit and Uninitialized Memory

Safe Rust assumes every value is **valid for its type**: a `bool` is only `true` or `false`, a reference is non-null and aligned, a `Vec<T>` upholds its length/capacity invariants. Sometimes you need storage **before** those invariants hold — out-pointers, element-by-element arrays, FFI buffers. The standard tool is `std::mem::MaybeUninit<T>`: a wrapper that tells the compiler "these bytes might not be initialized yet."

Do **not** reach for the deprecated `mem::uninitialized` or casual `mem::zeroed` on arbitrary `T`. Prefer `MaybeUninit`, initialize deliberately, then `assume_init` only when every validity requirement of `T` is met.

Mental model

```
MaybeUninit::uninit()     ← bytes may be garbage; Drop of T does NOT run

   write / ptr::write     ← initialize without dropping old (uninit) contents

 assume_init / assume_init_drop


       T                  ← normal Drop and safe APIs apply
```

API	Role
`MaybeUninit::uninit()`	Allocate storage; contents uninitialized
`MaybeUninit::new(v)`	Already-initialized wrapper
`MaybeUninit::zeroed()`	Fill with `0` bytes — **only** valid if all-zero is a valid `T`
`write(val)`	Initialize (overwrites without dropping previous)
`assume_init()`	Assert fully initialized; take ownership as `T`
`assume_init_drop()`	Drop in place when you do not want ownership
`as_mut_ptr()` / `&raw mut (*ptr).field`	Field-by-field init without creating `&mut` to uninit fields

Dropping a `MaybeUninit<T>` **never** runs `T`'s destructor. If you initialized and then abandon the wrapper without `assume_init` / `assume_init_drop`, you **leak** resources (safe but usually undesirable).

Worked examples

Case 1: `write` then `assume_init`

```
use std::mem::MaybeUninit;

fn main() {
    let mut slot = MaybeUninit::<Vec<u8>>::uninit();
    slot.write(vec![1, 2, 3]);
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![1, 2, 3]);
    println!("len={}", v.len());
}
```

`write` returns `&mut T` to the now-initialized contents. Calling `write` twice **skips** dropping the first value — a leak if `T` owns heap memory. Prefer one write, or drop explicitly between overwrites when you must replace.

Case 2: Out-pointer style

```
use std::mem::MaybeUninit;

unsafe fn fill_vec(out: *mut Vec<i32>) {
    // write does not drop old contents - correct for uninit destination
    unsafe { out.write(vec![10, 20, 30]); }
}

fn main() {
    let mut slot = MaybeUninit::uninit();
    unsafe { fill_vec(slot.as_mut_ptr()); }
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![10, 20, 30]);
}
```

The caller owns allocation strategy; the callee only initializes. After `assume_init`, `Drop` for `Vec` runs normally at end of scope.

Case 3: Array element-by-element

```
use std::mem::{self, MaybeUninit};

fn build_boxes() -> [Box<u32>; 4] {
    let mut data: [MaybeUninit<Box<u32>>; 4] = [const { MaybeUninit::uninit() }; 4];
```

```
    for (i, slot) in data.iter_mut().enumerate() {
        slot.write(Box::new(i as u32));
    }

    // SAFETY: every element was written
    unsafe { mem::transmute::<_, [Box<u32>; 4]>(data) }
}

fn main() {
    let arr = build_boxes();
    assert_eq!(*arr[3], 3);
    println!("{:?}", arr.iter().map(|b| **b).collect::<Vec<_>>());
}
```

Panic mid-loop leaks already-written `Box`es (no destructor on remaining `MaybeUninit`), but does not invent UB from dropping uninitialized boxes. For partial init, track a length and call `assume_init_drop` on the live prefix in a cleanup path.

Case 4: Field-by-field struct init

```
use std::mem::MaybeUninit;

#[derive(Debug, PartialEq)]
struct Person {
    name: String,
    age: u32,
}

fn main() {
    let person = {
        let mut uninit = MaybeUninit::<Person>::uninit();
        let ptr = uninit.as_mut_ptr();

        // SAFETY: write through raw field pointers - do NOT create &mut to uninit fields
        unsafe {
            (&raw mut (*ptr).name).write(String::from("Ada"));
            (&raw mut (*ptr).age).write(36);
            uninit.assume_init()
        }
    };

    assert_eq!(
        person,
        Person {
            name: String::from("Ada"),
            age: 36
```

```
        }
    );
    println!("{person:?}");
}
```

Using `assume_init_mut()` to "assign fields" **before** the struct is fully initialized is UB — that creates references to uninitialized memory. Prefer `&raw mut` / `ptr::write` as above. Padding bytes need not be initialized.

Case 5: `zeroed` pitfalls

```
use std::mem::MaybeUninit;

fn main() {
    // OK: (u8, bool) can be all-zero
    let pair = unsafe { MaybeUninit::<(u8, bool)>::zeroed().assume_init() };
    assert_eq!(pair, (0, false));

    // NOT OK: &T must be non-null - zeroed reference is instantaneous UB
    // let bad: &i32 = unsafe { MaybeUninit::zeroed().assume_init() };

    // NOT OK: many enums reject discriminant 0
    // enum NonZero { A = 1, B = 2 }
    // let bad = unsafe { MaybeUninit::<NonZero>::zeroed().assume_init() };

    println!("pair={pair:?}");
}
```

Rule of thumb: `zeroed().assume_init()` only for types where **every bit pattern of all-zeros is a valid value** (raw integers, `bool`, structs of such). Never for references, `NonNull`, most `enums`, or "looks like a Vec with ptr=0".

Choosing the API

Situation	Prefer
Single value, known initializer	`MaybeUninit::new` or plain `T`
Deferred init / out-pointer	`uninit` + `write` / `ptr::write`
Large array filled in a loop	`[MaybeUninit<T>; N]` + `write` each
Struct fields in sequence	`&raw mut (*ptr).field` + `write`
Need all-zero bit pattern and it is valid	`zeroed` with care
Old code with `mem::uninitialized`	Port to `MaybeUninit`

Pitfalls

1. **`assume_init` too early** — immediate UB, even for `i32` / `bool` when bytes are still uninit.
2. **`zeroed` on references / invalid enums** — classic footgun; all-zero is not "safe default" for every `T`.
3. **Double `write` without drop** — leaks the first `T`.
4. **Creating `&` / `&mut` to uninitialized fields** — use raw pointers / `&raw mut` until fully init.
5. **Transmuting `&mut T` to `&mut MaybeUninit<T>` for safe code** — lets safe code invent uninit views; unsound.
6. **Assuming Miri-silence equals production-proof** — still run tests under Miri for unsafe modules when you can.

Try this

1. Run Case 1 and Case 3 under `cargo +nightly miri run` if Miri is available.
2. Deliberately comment out one field write in Case 4 and observe that Miri / careful review catches the incomplete init (do not ship it).
3. Replace a `mem::uninitialized` call in an old snippet with `MaybeUninit` and list which `assume_init` proof you now owe.
4. Skim the Rustonomicon chapter on unchecked uninitialized memory and compare its array pattern to Case 3.

Sources

- `std::mem::MaybeUninit`
- The Rustonomicon — Unchecked Uninitialized Memory

Custom Allocators and the Allocator API

Custom Allocators and the Allocator API

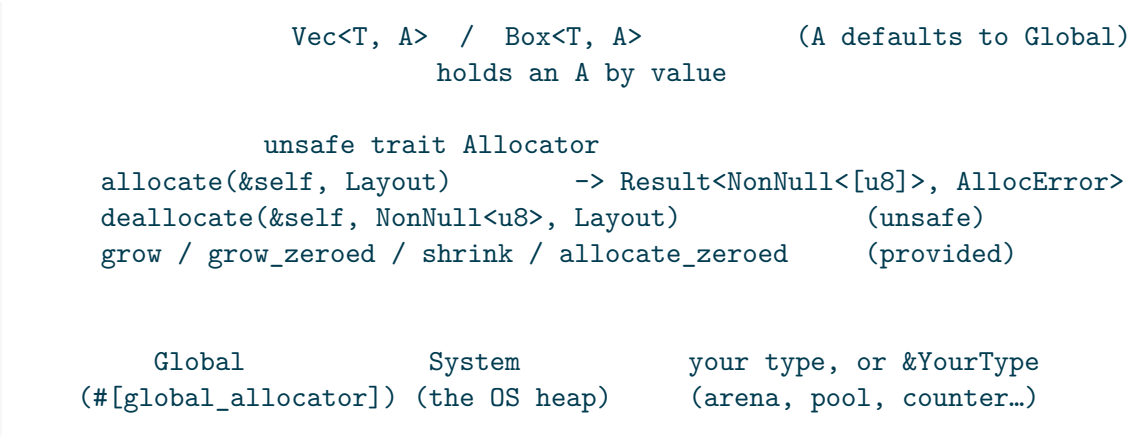
Thesis

Every `Box` and `Vec` you have written so far got its memory from one place: the **global allocator**. You could swap that allocator for the whole program with `#[global_allocator]`, but you could not say "*this* vector lives in *that* arena". The `Allocator` trait that would let you do so has existed since 2016, behind the nightly-only `allocator_api` feature, and arena crates have shipped nightly-only allocator-aware APIs for years because of it.

Rust 1.100 (scheduled for 2026-11-12) stabilizes a deliberately small slice of that design: the `std::alloc::Allocator` trait, the `Global` and `System` allocators as implementors, and allocator-aware constructors on exactly two containers, `Box<T, A>` and `Vec<T, A>`. Everything else (`Rc::new_in`, fallible `try_*_in` constructors, allocator-aware `String` and `HashMap`) stays unstable under a new feature name, `allocator_ext`.

This chapter builds two allocators on the stable surface, a counting wrapper and a bump arena. It then walks through the places where the type system and the abort-on-failure policy catch people out. Everything was compiled with the current beta, which is what becomes 1.100.

Mental model



Four facts carry the chapter:

Fact	Consequence
The allocator is a **type parameter** with a default: `Vec<T, A = Global>`	`Vec<u32>` and `Vec<u32, System>` are *different types*. Code written for one does not accept the other

Fact	Consequence
The container **stores** an `A`	A zero-sized allocator (`Global`, `System`) costs nothing; `&Arena` costs one pointer; `&dyn Allocator` costs two
Methods take `&self`	Implement `Allocator` on `&Arena` (blanket impl provided) to share one arena among many containers. Interior mutability (`Cell`) does the bookkeeping
The trait is `unsafe` to implement	You promise that returned blocks are valid until deallocated, that blocks don't overlap, and that no method unwinds. The compiler cannot check any of it

`allocate` returns `NonNull<[u8]>`, a *slice* pointer, so an allocator can report that it handed back more than was asked for. `AllocError` is a zero-sized "no" with no further detail.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, use `+beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: a counting allocator and a bump arena

```
cargo +beta new alloc-demo && cd alloc-demo
```

`Counting` wraps any other allocator and records each call. `Arena<N>` hands out slices of a fixed buffer by bumping a cursor, and its `deallocate` does nothing: the whole region is released at once when the arena is dropped. Note that neither needs a `#![feature]` line.

```
// src/main.rs - a counting allocator and a bump arena on the stable Allocator API (Rust 1.1
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

/// Wraps another allocator and counts what passes through it.
struct Counting<A: Allocator = Global> {
    inner: A,
    allocs: Cell<usize>,
    grows: Cell<usize>,
```

```
    frees: Cell<usize>,
    live_bytes: Cell<usize>,
}

impl Counting {
    const fn new() -> Self {
        Counting { inner: Global, allocs: Cell::new(0), grows: Cell::new(0), frees: Cell::ne
    }
    fn report(&self, label: &str) {
        println!(
            "{label:<22} allocs={} grows={} frees={} live_bytes={}",
            self.allocs.get(), self.grows.get(), self.frees.get(), self.live_bytes.get()
        );
    }
}

// SAFETY: every method forwards to `inner` with the caller's arguments unchanged,
// so blocks stay valid until deallocated and are freed by the allocator that made them.
unsafe impl<A: Allocator> Allocator for Counting<A> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        self.allocs.set(self.allocs.get() + 1);
        self.live_bytes.set(self.live_bytes.get() + layout.size());
        self.inner.allocate(layout)
    }
    unsafe fn deallocate(&self, ptr: NonNull<u8>, layout: Layout) {
        self.frees.set(self.frees.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - layout.size());
        unsafe { self.inner.deallocate(ptr, layout) }
    }
    unsafe fn grow(&self, ptr: NonNull<u8>, old: Layout, new: Layout) -> Result<NonNull<[u8]
        self.grows.set(self.grows.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - old.size() + new.size());
        unsafe { self.inner.grow(ptr, old, new) }
    }
}

/// A fixed-size bump arena: allocation moves a cursor, deallocation does nothing,
/// and all memory comes back at once when the arena itself is dropped.
struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
```

```
    fn used(&self) -> usize {
        self.used.get()
    }
}

// SAFETY: blocks are carved from `buf` without overlap and stay valid while `self`
// is alive; borrowing the arena (`&Arena`) ties every collection to that lifetime.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _ptr: NonNull<u8>, _layout: Layout) {
        // Bump arenas free everything at once, when the arena is dropped.
    }
}

#[derive(Debug)]
struct Token<'a> {
    kind: &'static str,
    text: &'a str,
}

fn tokenize<'s, A: Allocator>(src: &'s str, alloc: A) -> Vec<Token<'s>, A> {
    let mut out = Vec::new_in(alloc);
    for word in src.split_whitespace() {
        let kind = if word.chars().all(|c| c.is_ascii_digit()) { "num" } else { "word" };
        out.push(Token { kind, text: word });
    }
    out
}

fn main() {
    // 1. Counting: watch Vec's growth policy through the allocator.
    let counter = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::new_in(&counter);
        counter.report("after new_in");
```

```
        for i in 0..100 {
            v.push(i);
        }
        counter.report("after 100 pushes");
        let b = Box::new_in([0u8; 256], &counter);
        counter.report("plus a Box");
        println!("capacity = {}, box len = {}", v.capacity(), b.len());
    }
    counter.report("after drop");

    let counter2 = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::with_capacity_in(100, &counter2);
        for i in 0..100 {
            v.push(i);
        }
    }
    counter2.report("with_capacity_in(100)");

    // 2. Arena: many small values, one region, no per-object free.
    let arena: Arena<4096> = Arena::new();
    let toks = tokenize("let x = 42 + y * 7", &arena);
    println!("{} tokens, arena used = {} bytes", toks.len(), arena.used());
    println!("first = {:?}, last = {:?}", toks[0], toks[toks.len() - 1]);
    let nums: Vec<&str> = toks.iter().filter(|t| t.kind == "num").map(|t| t.text).collect();
    println!("numbers = {nums:?}");

    let pair = Box::new_in((1u32, 2u32), &arena);
    println!("boxed pair = {:?}, arena used = {}", *pair, arena.used());
    println!("Vec allocator is the arena: {}", std::ptr::eq(*toks.allocator(), &arena));
}
```

```
cargo +beta run -q
```

```
after new_in           allocs=0 grows=0 frees=0 live_bytes=0
after 100 pushes       allocs=1 grows=5 frees=0 live_bytes=1024
plus a Box             allocs=2 grows=5 frees=0 live_bytes=1280
capacity = 128, box len = 256
after drop             allocs=2 grows=5 frees=2 live_bytes=0
with_capacity_in(100)  allocs=1 grows=0 frees=1 live_bytes=0
8 tokens, arena used = 384 bytes
first = Token { kind: "word", text: "let" }, last = Token { kind: "num", text: "7" }
numbers = ["42", "7"]
boxed pair = (1, 2), arena used = 392
Vec allocator is the arena: true
```

What to notice:

- **`Vec::new_in` allocates nothing.** The first `push` makes one allocation (4 × `u64`), and then the vector *grows* five times (8, 16, 32, 64, 128) to hold 100 items. `with_capacity_in(100)` makes one allocation and no growth. The counter turns a growth policy you normally take on faith into numbers you can see.
- **`live_bytes` returns to 0** after the scope ends, so every byte allocated was deallocated with the layout it was allocated with. That is the contract `Vec` and `Box` keep on their side.
- **The arena used 384 bytes for 8 tokens of 32 bytes each.** The provided `grow` method allocates a new, larger block, copies, and calls `deallocate` on the old block. In a bump arena that call is a no-op, so the 4-token block (128 bytes) is simply abandoned. Arenas reward `with_capacity_in` far more than the heap does.
- `tokenize` is generic over `A: Allocator` and returns `Vec<Token<'s>, A>`. The caller, not the function, decides where the tokens live.
- `toks.allocator()` gives back the allocator the vector holds. Here that is the `&Arena` reference.

On 1.99 stable the same crate is rejected at the first import:

```
cargo +stable build -q
```

```
error[E0658]: use of unstable library feature `allocator_api`
 --> src/main.rs:2:18
  |
2 | use std::alloc::{AllocError, Allocator, Global, Layout};
  |                  ^^^^^^^^^^
  |
  = note: see issue #32838 <https://github.com/rust-lang/rust/issues/32838> for more informa
...
```

Case 2: what the allocator parameter costs

The allocator is stored inside the container, so its size shows up in the container's size. The trait is also `dyn`-compatible, so you can choose an allocator at run time.

```
// dynalloc.rs - choosing an allocator at run time with &dyn Allocator
use std::alloc::{Allocator, Global, System};

fn pick(use_system: bool) -> &'static dyn Allocator {
    if use_system { &System } else { &Global }
}

fn main() {
    for flag in [false, true] {
        let alloc = pick(flag);
        let mut v: Vec<u32, &dyn Allocator> = Vec::with_capacity_in(4, alloc);
        v.extend([10, 20, 30]);
        let sum: u32 = v.iter().sum();
```

```
        println!("system={flag} sum={sum} size_of::<Vec<u32, &dyn Allocator>>()={}",
                 std::mem::size_of_val(&v));
    }
    println!("size_of::<Vec<u32>>() = {}", std::mem::size_of::<Vec<u32>>());
    println!("size_of::<Vec<u32, System>>() = {}", std::mem::size_of::<Vec<u32, System>>());
}
```

```
rustc +beta --edition 2024 dynalloc.rs && ./dynalloc
```

```
system=false sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
system=true sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
size_of::<Vec<u32>>() = 24
size_of::<Vec<u32, System>>() = 24
```

`Global` and `System` are zero-sized, so a `Vec` that uses them is still three words. A `&Arena` adds one word, and a `&dyn Allocator` adds two (data pointer and vtable) plus an indirect call on every allocation. Reach for `dyn` only when the choice really is made at run time.

Case 3: an allocator says no

Every allocating method you already know (`push`, `extend`, `Box::new_in`) is **infallible**. When the allocator returns `AllocError`, they call `handle_alloc_error`, which aborts the process. The global heap almost never says no, so this hardly matters there. A 256-byte arena says no quickly.

```
// exhaust.rs - what happens when a custom allocator says no
use std::alloc::{AllocError, Allocator, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
}

// SAFETY: non-overlapping blocks carved from `buf`, valid while the arena lives.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
```

```
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _: NonNull<u8>, _: Layout) {}
}

fn main() {
    let mode = std::env::args().nth(1).unwrap_or_default();
    let arena: Arena<256> = Arena::new();
    let mut v: Vec<u64, &Arena<256>> = Vec::new_in(&arena);

    if mode == "try" {
        for i in 0.. {
            if v.len() == v.capacity() {
                if let Err(e) = v.try_reserve(1) {
                    println!("stopped at len={} cap={}: {e}", v.len(), v.capacity());
                    break;
                }
            }
            v.push(i);
        }
        println!("arena used = {} of 256", arena.used.get());
    } else {
        for i in 0..64u64 {
            v.push(i); // infallible API: allocation failure aborts the process
            println!("len={} cap={} used={}", v.len(), v.capacity(), arena.used.get());
        }
    }
}
```

```
rustc +beta --edition 2024 -O exhaust.rs
./exhaust; echo "exit=$?"
./exhaust try; echo "exit=$?"
```

```
len=1 cap=4 used=32
len=2 cap=4 used=32
len=3 cap=4 used=32
len=4 cap=4 used=32
```

```
len=5 cap=8 used=96
len=6 cap=8 used=96
len=7 cap=8 used=96
len=8 cap=8 used=96
len=9 cap=16 used=224
len=10 cap=16 used=224
len=11 cap=16 used=224
len=12 cap=16 used=224
len=13 cap=16 used=224
len=14 cap=16 used=224
len=15 cap=16 used=224
len=16 cap=16 used=224
memory allocation of 256 bytes failed
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
Aborted
exit=134
stopped at len=16 cap=16: memory allocation failed because the memory allocator returned an
arena used = 224 of 256
exit=0
```

(`Aborted` is the shell reporting `SIGABRT`; its exact wording differs between shells.) The abort is not a panic. `catch_unwind` cannot stop it, and destructors do not run. `Vec::try_reserve` *is* available on `Vec<T, A>` (it has been stable since 1.57 and works with any allocator), so the fallible path is to reserve before you push and handle the `TryReserveError`. Note too that 128 bytes of real data cost 224 bytes of arena, because the 32- and 64-byte blocks left behind by each growth step are never reused.

Case 4: what is *not* stable yet

The stabilization is an MVP on purpose. Three things people reach for straight away still fail on 1.100 beta:

```
// not_yet.rs - allocator-aware APIs that are still unstable in Rust 1.100
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;
use std::rc::Rc;

struct Pool;
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn main() {
    let r = Rc::new_in(5u8, Pool);                              // shared ownership in a cus
    let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
```

```
    let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-allocate
    println!("{r} {:?} {p}", v.map(|v| v.len()));
}
```

```
rustc +beta --edition 2024 not_yet.rs
```

```
error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:13:13
   |
13 |     let r = Rc::new_in(5u8, Pool);                                 // shared ownership in
   |             ^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:14:13
   |
14 |     let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0277]: the trait bound `Pool: StaticAllocator` is not satisfied
  --> not_yet.rs:15:27
   |
15 |     let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-all
   |             ------------- ^^^^^^^^^^^^^^^^^^^^^^^ unsatisfied trait bound
   |             |
   |             required by a bound introduced by this call
   |
help: the nightly-only, unstable trait `StaticAllocator` is not implemented for `Pool`
...
```

- **`Rc::new_in` / `Arc::new_in`** remain under `allocator_ext` (tracking issue #163177). What *is* stable is `impl Allocator for Rc<A>` / `Arc<A>`, so a shared allocator can be passed around by `Rc`, but the reference-counted *values* cannot live in a custom allocator yet.
- **`try_with_capacity_in`** and the other fallible constructors are unstable. Use `new_in` plus `try_reserve`, as in Case 3.
- **`Box::into_pin`** needs the unstable `StaticAllocator` marker. `Global`, `System`, `&A` and `&mut A` have it, and your own types cannot opt in on stable. This is a soundness fix from the stabilization work: a box whose allocator can be dropped early must not promise `Pin`'s "never moves, never freed without drop" guarantee.
- **`String`, `HashMap`, `BTreeMap`, `VecDeque`** have no allocator parameter on stable. `String<System>` is `error[E0107]: struct takes 0 generic arguments`. If you need allocator-aware text, hold a `Vec<u8, A>` and validate it with `std::str::from_utf8`.

The trap

A collection outlives its arena

`&Arena` implements `Allocator`, and the borrow checker treats it like any other borrow. Returning a vector that borrows a local allocator fails to compile:

```
// outlive.rs - a collection cannot outlive the arena it borrows
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;

struct Pool; // stand-in: forwards to Global
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn build() -> Vec<u32, &'static Pool> {
    let pool = Pool;
    let mut v = Vec::new_in(&pool);
    v.push(1);
    v
}

fn main() {
    println!("{:?}", build());
}
```

```
rustc +beta --edition 2024 outlive.rs
```

```
error[E0515]: cannot return value referencing local variable `pool`
  --> outlive.rs:15:5
   |
13 |     let mut v = Vec::new_in(&pool);
   |                              ----- `pool` is borrowed here
14 |     v.push(1);
15 |     v
   |     ^ returns a value referencing data owned by the current function

error: aborting due to 1 previous error
```

This error is the feature working. It is the reason an arena in Rust can be safe without a garbage collector. The fix is structural: create the arena in the caller and pass `&arena` down, as `tokenize` does in Case 1.

`Vec<T, A>` does not fit where `Vec<T>` is expected

```
// mismatch.rs - Vec<T, A> is a different type from Vec<T>
use std::alloc::System;

fn total(v: &Vec<u32>) -> u32 {
    v.iter().sum()
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    println!("{}", total(&v));
}
```

```
error[E0308]: mismatched types
  --> mismatch.rs:11:26
   |
11 |     println!("{}", total(&v));
   |                     ----- ^^ expected `&Vec<u32>`, found `&Vec<{integer}, System>`
   |                     |
   |                     arguments to this function are incorrect
   |
   = note: expected reference `&Vec<u32, std::alloc::Global>`
              found reference `&Vec<{integer}, System>`
```

Every API in your codebase that names `&Vec<T>` or `Box<T>` quietly means `Global`. Read-only functions should take `&[T]`, which every `Vec<T, A>` derefs to. Functions that grow the vector should be generic over the allocator:

```
// mismatch_fixed.rs - accept slices, or be generic over the allocator
use std::alloc::{Allocator, System};

fn total(v: &[u32]) -> u32 {
    v.iter().sum()
}

fn push_twice<A: Allocator>(v: &mut Vec<u32, A>, x: u32) {
    v.push(x);
    v.push(x);
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    push_twice(&mut v, 10);
```

```
    println!("{}", total(&v));
    let plain = vec![4u32, 5];
    println!("{}", total(&plain));
}
```

```
26
9
```

Implementing the trait is a safety contract

`unsafe impl Allocator` is a promise the compiler cannot check. The ways to break it are the classic C allocator bugs: handing out overlapping blocks (a bump cursor that forgets alignment padding), returning memory that dies before the container does (an allocator that owns its buffer by value and gets *moved*), or panicking inside `allocate`. The stabilized contract explicitly forbids unwinding out of any `Allocator` method, from its clones, or from its drop. Keep the `unsafe impl` small, put the bookkeeping in plain safe code, and test it under Miri (`cargo +nightly miri test`), which catches the use-after-free, out-of-bounds, and aliasing violations that a broken allocator causes in the containers using it.

Choosing an approach

Need	Use
Replace the heap for the whole program	`#[global_allocator]` with a `GlobalAlloc` (stable since 1.28)
Many short-lived objects freed together (parsers, per-request scratch)	A bump arena passed as `&Arena` to `Vec::new_in` / `Box::new_in`
Measure or limit what one subsystem allocates	A wrapper like `Counting<A>` around `Global`
Pick the allocator at run time	`&dyn Allocator` (two extra words per container)
Shared ownership, maps, strings in a custom allocator	Not on stable yet (`allocator_ext`), so keep a third-party collection for now
MSRV below 1.100	Keep using nightly-gated code or the arena crate's own types; don't half-adopt

The boring rule

Pass the allocator in, don't hide it: `fn f<A: Allocator>(…, alloc: A) -> Vec<T, A>`. Take `&[T]` when you only read. Reserve capacity up front in arenas, and use `try_reserve` wherever running out is a normal outcome rather than a bug. Write `unsafe impl Allocator` only after you've measured a reason, and run it under Miri.

Try this

1. Change `Counting` to also override `shrink`, call `v.shrink_to_fit()` after the 100 pushes, and confirm the new counter moves while `live_bytes` drops to 800.
2. Give `Arena` a `reset(&mut self)` that sets `used` back to 0. Why does it have to take `&mut self`, and what does the borrow checker say if a `Vec<_, &Arena>` is still alive when you call it?
3. Make `Arena::deallocate` reclaim space when the block being freed is the most recent one (`ptr + size == cursor`). Re-run Case 1 and compare `arena used` before and after.
4. Rewrite `tokenize` to call `Vec::with_capacity_in(src.split_whitespace().count(), alloc)` and predict the new `arena used` value before running it.
5. Wrap `System` in `Counting<System>` and verify that `Box::into_pin` fails to compile for it even though it compiles for plain `System`. Which unstable trait is missing, and why can't you implement it yourself?

Sources

- rust-lang/rust PR #156882, "alloc: stabilise `Allocator`" (stabilization report, merged 2026-09-23, milestone 1.100.0): https://github.com/rust-lang/rust/pull/156882
- Draft release notes for 1.100.0: https://github.com/rust-lang/rust/issues/163811
- Tracking issue `allocator_api` #32838 and the remaining-surface issue `allocator_ext` #163177
- RFC 1398, "Kinds of allocators": https://rust-lang.github.io/rfcs/1398-kinds-of-allocators.html
- std docs: `std::alloc::Allocator`, `Vec::new_in`, `Box::new_in`, `Vec::try_reserve`, `handle_alloc_error`

Summary

Summary

Chapter Key Takeaways

- `Box<T>` allocates on the heap with single ownership.
- `Rc<T>` and `Arc<T>` provide shared read ownership via reference counting.
- `RefCell<T>` provides interior mutability with runtime borrow checks.
- `Mutex<T>` and `RwLock<T>` provide thread-safe interior mutability.
- `Weak<T>` prevents reference count cycles.
- `OnceLock` / `LazyLock` (and `OnceCell` / `LazyCell`) cover one-time initialization without third-party crates.
- `MaybeUninit<T>` allows working with uninitialized memory safely before type invariants are established.

Iterators, Closures, and Functional Patterns

The Iterator Trait and Protocol

The Iterator Trait and Protocol

In Rust, iterators are lazy sequences that produce values on demand via the `Iterator` trait:

```
fn main() {
    let v = vec![1, 2, 3];
    let mut iter = v.iter();

    assert_eq!(iter.next(), Some(&1));
    assert_eq!(iter.next(), Some(&2));
    assert_eq!(iter.next(), Some(&3));
    assert_eq!(iter.next(), None);
}
```

Creating Custom Iterators

Creating Custom Iterators

```
struct Counter {
    count: usize,
}

impl Counter {
    fn new() -> Self { Self { count: 0 } }
}

impl Iterator for Counter {
    type Item = usize;
    fn next(&mut self) -> Option<Self::Item> {
        if self.count < 3 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let counter = Counter::new();
    let items: Vec<usize> = counter.collect();
    println!("Custom iterator output: {:?}", items);
}
```

Closures and Capture Semantics

Closures and Capture Semantics

Closures are anonymous functions that can capture environment variables: - `Fn`: Borrows values from the environment immutably. - `FnMut`: Borrows values mutably. - `FnOnce`: Takes ownership and consumes captured values.

```
fn main() {
    let factor = 3;
    let multiply = |x| x * factor;
    println!("Result: {}", multiply(10));
}
```

Higher-Order Functions and Combinators

Higher-Order Functions and Combinators

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6];

    let sum_of_even_squares: i32 = numbers
        .iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * x)
        .sum();

    println!("Sum of even squares: {}", sum_of_even_squares);
}
```

Lazy Evaluation and Performance

Lazy Evaluation and Performance

Rust iterator adaptors are **zero-cost abstractions**. They compile down to unrolled assembly with no heap allocation, often outperforming hand-written imperative loops.

FusedIterator — Why Fuse Matters

FusedIterator — Why Fuse Matters

Thesis

`FusedIterator` marks iterators that keep returning `None` forever after the first `None`. Adapters like `fuse()` wrap a flaky iterator so callers (and `Iterator` combinators) can assume fusion.

Mental model

```
Ordinary iterator:  Some, Some, None, Some?  ← "zombie" possible
Fused:              Some, Some, None, None, None, …
```

`Iterator::fuse` returns `Fuse<Self>` implementing `FusedIterator`.

Worked example

```
// fused_demo.rs
use std::iter::{Fuse, FusedIterator};

struct OnceThenWeird {
    state: u8,
}
impl Iterator for OnceThenWeird {
    type Item = i32;
    fn next(&mut self) -> Option<i32> {
        self.state += 1;
        match self.state {
            1 => Some(10),
            2 => None,
            3 => Some(99), // zombie value - bad manners
            _ => None,
        }
    }
}

fn main() {
    let mut raw = OnceThenWeird { state: 0 };
```

```
    println!("raw: {:?} {:?} {:?} {:?}", raw.next(), raw.next(), raw.next(), raw.next());

    let mut fused: Fuse<_> = (OnceThenWeird { state: 0 }).fuse();
    println!(
        "fused: {:?} {:?} {:?} {:?}",
        fused.next(),
        fused.next(),
        fused.next(),
        fused.next()
    );
    fn assert_fused<I: FusedIterator>(_: &I) {}
    assert_fused(&fused);
}
```

```
first pass: [1, 2, 3]
second pass: []
```

Trap

Implementing `FusedIterator` for a type that can revive. The marker is a contract; lying breaks `Flatten` / other adapters that skip re-polling exhausted inners incorrectly if the contract fails. Prefer `.fuse()` at the boundary over claiming the trait yourself unless proven.

Boring rule

If you hand an iterator to combinators after a possible early `None`, call `.fuse()`. Implement `FusedIterator` only when your `next` is proven fused.

Try this

1. Read the `Fuse` source — it stores a flag after first `None`.
2. Check which std iterators already implement `FusedIterator`.
3. Break `Flatten` intentionally with a zombie iterator (in a toy) to feel why fusion matters.

Sources

- `std::iter::FusedIterator`
- `Iterator::fuse`

Summary

Summary

Chapter Key Takeaways

- Iterators are lazy; evaluation only happens when consumed (e.g., `collect`, `sum`).
- Closures capture environment variables via `Fn`, `FnMut`, or `FnOnce`.
- Combinators (`map`, `filter`, `fold`) form clear functional pipelines.
- Modern Rust compilers optimize iterator pipelines into highly vectorized machine loops.

Concurrency: Fearless Parallelism

Spawning Threads with std::thread

Spawning Threads with std::thread

Rust guarantees thread safety through its type system.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        println!("Worker thread running");
    });

    handle.join().unwrap();
    println!("Worker thread joined");
}
```

Message Passing with Channels

Message Passing with Channels

"Do not communicate by sharing memory; instead, share memory by communicating."

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let msg = String::from("order_payload");
        tx.send(msg).unwrap(); // Ownership moves into channel!
    });

    let received = rx.recv().unwrap();
    println!("Received through channel: {}", received);
}
```

Shared State with Mutex and RwLock

Shared State with Mutex and RwLock

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..5 {
        let counter_clone = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut num = counter_clone.lock().unwrap();
            *num += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Final count: {}", *counter.lock().unwrap());
}
```

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across .await

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across `.await`

Thesis

"Reads don't conflict, so use `RwLock` for read-heavy data" is half right. Readers don't *logically* exclude each other. But every `read()` still **writes** to the lock: it adds one to a shared reader count, and dropping the guard subtracts one. In the standard library's Linux implementation, that is a compare-exchange and a `fetch_sub` on one 32-bit word. With eight threads reading, that word's cache line moves between cores on every acquisition. When the work inside the lock is tiny, the line transfer *is* the cost. The readers queue for the cache line just as `Mutex` users queue for the lock, and adding threads makes total throughput **worse**.

`RwLock` pays off only when the read-side critical section is long enough to hide that traffic. If the data really is read-mostly and the section is tiny, the answer is to stop sharing the written word: use an atomic, or give each reader its own lock (a "big-reader" lock). This chapter measures all of these.

Async code has a second lock rule: never hold a `std::sync` guard across an `.await`. The compiler enforces it only when the future has to be `Send`. On a current-thread runtime the same mistake compiles and deadlocks.

Mental model

What one uncontended acquisition costs, from the standard library's futex-based implementation (`library/std/src/sys/sync/rwlock/futex.rs`):

Operation	Atomic work on the shared word	Notes
`RwLock::read()`	`load`, then `compare_exchange_weak(state, state + 1)`	falls into `read_contended()` (spin, then futex wait) if the CAS fails or a writer is waiting
drop of a read guard	`fetch_sub(1)`	wakes a writer if this was the last reader

Operation	Atomic work on the shared word	Notes
`Mutex::lock()` / drop	`compare_exchange(0, 1)` / `swap(0)`	waiters sleep on a futex
`AtomicU64::load`	none: a plain load	the line can stay shared in every core's cache

The std source also fixes the priority policy. `is_read_lockable` returns false whenever the writers-waiting bit is set, and a comment in the same function calls these "writers, which have priority over readers". So on Linux this `RwLock` is writer-preferring. That matters for the trap at the end of the chapter.

Worked examples

Built with **rustc/cargo 1.85.1** (the distro toolchain on the box), edition 2024, `--release`. The current stable Rust is **1.99.1** (2026-10-13). The `RwLock` and `Mutex` implementations quoted above are from the current `master` source. The machine is a shared 8-vCPU x86-64 VM (Intel Xeon, one thread per core). Absolute numbers will differ on your hardware, but the shape should not. Every benchmark reports the median of 5 runs, and each run measures the slowest thread.

```
cargo new --vcs none locks && cd locks
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "locks"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]

[profile.release]
debug = false
```

```
// src/lib.rs - the shared harness: N threads start together, each does `ops` operations
use std::hint::black_box;
use std::sync::{Arc, Barrier};
use std::thread;
use std::time::{Duration, Instant};

/// Runs `op` `ops` times on each of `threads` threads and returns ns per operation
/// for the slowest thread. Repeats the measurement 5 times and reports the median.
```

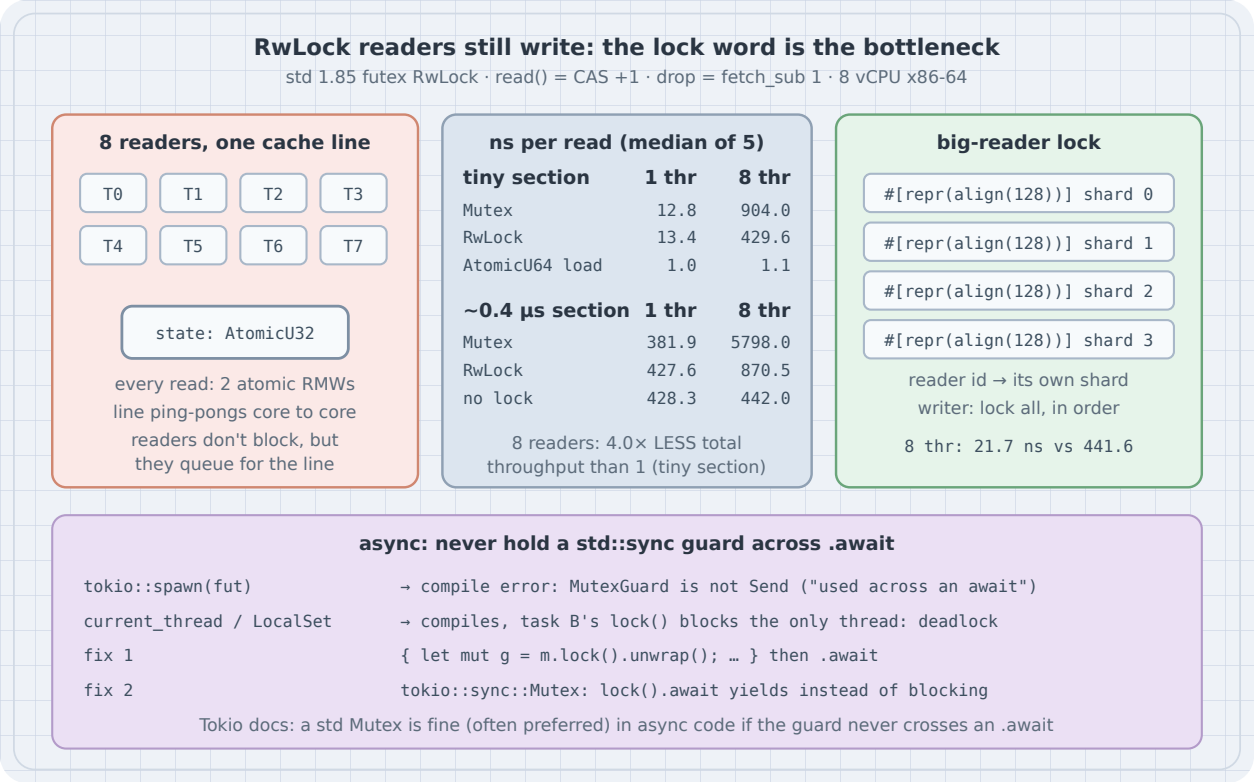


Figure 1: RwLock readers each do a CAS and a fetch_sub on one state word, so the cache line ping-pongs; measured ns per read for tiny and 0.4 µs sections; a big-reader lock with one aligned shard per reader; the async rule about std guards across .await

```
pub fn bench<T: Send + Sync + 'static>(threads: usize, ops: u64, shared: Arc<T>, op: fn(&T,
    let mut samples: Vec<f64> = (0..5)
        .map(|_| {
            let start = Arc::new(Barrier::new(threads + 1));
            let handles: Vec<_> = (0..threads)
                .map(|id| {
                    let (shared, start) = (Arc::clone(&shared), Arc::clone(&start));
                    thread::spawn(move || {
                        start.wait();
                        let t = Instant::now();
                        let mut acc = 0u64;
                        for i in 0..ops {
                            acc = acc.wrapping_add(op(&shared, id, i));
                        }
                        black_box(acc);
                        t.elapsed()
                    })
                })
                .collect();
            start.wait();
            let worst: Duration = handles.into_iter().map(|h| h.join().unwrap()).max().unwra
            worst.as_nanos() as f64 / ops as f64
        })
        .collect();
    samples.sort_by(|a, b| a.partial_cmp(b).unwrap());
    samples[2]
}
```

Case 1: A tiny read-only critical section

```
// src/bin/tiny.rs - read-only workload, tiny critical section: Mutex vs RwLock vs a plain a
use locks::bench;
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 1_000_000;

fn main() {
    println!("{:>7}  {:>11}  {:>12}  {:>12}", "threads", "Mutex ns/op", "RwLock ns/op", "ato
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(42u64)), |m, _, _| *m.lock().unwrap());
        let r = bench(t, OPS, Arc::new(RwLock::new(42u64)), |r, _, _| *r.read().unwrap());
        let a = bench(t, OPS, Arc::new(AtomicU64::new(42)), |a, _, _| a.load(Ordering::Acqui
        println!("{t:>7}  {m:>11.1}  {r:>12.1}  {a:>12.1}");
    }
```

```
}
```

```
$ cargo build --release && ./target/release/tiny
threads  Mutex ns/op  RwLock ns/op  atomic ns/op
      1         12.8          13.4           1.0
      2        135.1         104.2           1.1
      4        448.1         218.0           1.0
      8        904.0         429.6           1.1
```

Read the columns as *latency per operation for each thread*. One thread reading through `RwLock` costs 13.4 ns. Eight threads reading the *same* value take 429.6 ns per read each. Total throughput falls from 1/13.4 ns 75 M reads/s to 8/429.6 ns 19 M reads/s, so adding seven readers made the program **four times slower** overall. `RwLock` does beat `Mutex` here, because readers that win the CAS don't sleep. But neither lock scales. The `AtomicU64` column is the floor. A load writes nothing, so the cache line stays shared, and eight threads read in about 1 ns each.

Case 2: The same reads with real work inside

```
// src/bin/work.rs - the same read-only workload, but ~0.4 µs of work inside the lock
use locks::bench;
use std::hint::black_box;
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 20_000;

// A dependent chain of 256 multiply-xorshift steps over the table: it cannot be vectorized.
fn walk(table: &[u64]) -> u64 {
    let mut h = 0x9e37_79b9_7f4a_7c15u64;
    for &x in black_box(table) {
        h = (h ^ x).wrapping_mul(0xbf58_476d_1ce4_e5b9);
        h ^= h >> 29;
    }
    h
}

fn main() {
    let table: Vec<u64> = (0..256).collect();
    println!("{:>7}  {:>11}  {:>12}  {:>13}", "threads", "Mutex ns/op", "RwLock ns/op", "no
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(table.clone())), |m, _, _| walk(&m.lock().
        let r = bench(t, OPS, Arc::new(RwLock::new(table.clone())), |r, _, _| walk(&r.read()
        let n = bench(t, OPS, Arc::new(table.clone()), |v, _, _| walk(v));
        println!("{t:>7}  {m:>11.1}  {r:>12.1}  {n:>13.1}");
    }
}
```

```
$ ./target/release/work
threads  Mutex ns/op  RwLock ns/op  no lock ns/op
      1        381.9         427.6          428.3
      2       1053.5         473.7          410.5
      4       2456.9         514.7          423.9
      8       5798.0         870.5          442.0
```

Now each read holds the lock for about 0.4 µs (the "no lock" column). `Mutex` serialises everyone: at eight threads, each operation waits behind seven others, about 5.8 µs. `RwLock` lets the readers overlap, and its cost stays within about 2× of the lock-free version. This is the workload `RwLock` was designed for. The lock word still bounces, but a few hundred nanoseconds of work between bounces hides it.

The rule from Cases 1 and 2 is about the **ratio of critical-section length to cache-line transfer cost** (roughly 50–200 ns between cores). The read/write ratio alone doesn't decide it.

Case 3: One write in a hundred

```
// src/bin/mixed.rs - tiny critical section, one write in every 100 operations
use locks::bench;
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 500_000;

fn main() {
    println!("{:>7}  {:>11}  {:>12}", "threads", "Mutex ns/op", "RwLock ns/op");
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(0u64)), |m, _, i| {
            let mut g = m.lock().unwrap();
            if i % 100 == 0 {
                *g += 1;
            }
            *g
        });
        let r = bench(t, OPS, Arc::new(RwLock::new(0u64)), |r, _, i| {
            if i % 100 == 0 {
                let mut g = r.write().unwrap();
                *g += 1;
                *g
            } else {
                *r.read().unwrap()
            }
        });
        println!("{t:>7}  {m:>11.1}  {r:>12.1}");
    }
}
```

```
$ ./target/release/mixed
threads  Mutex ns/op  RwLock ns/op
      1         11.0          13.3
      2        149.5         109.8
      4        483.2         309.1
      8       1012.8         726.0
```

With one thread, `RwLock` is the slower lock (13.3 vs 11.0 ns). It does more bookkeeping, and nothing is gained without contention. Under contention it stays ahead of `Mutex` on this box, but both are about 55–90× slower per operation than the single-threaded case. If your profile shows time in `futex` or in `read_contended`, switching between `Mutex` and `RwLock` won't fix it. Changing what is shared will.

Case 4: Stop sharing the written word: a big-reader lock

The classic fix for read-mostly data with tiny sections is to give each reader its *own* lock. Readers write only to their own cache line, and the rare writer takes every lock:

```rust
// src/bin/sharded.rs - a "big-reader" lock: one RwLock per reader slot, writers take them a
use locks::bench;
use std::sync::{Arc, RwLock};

const OPS: u64 = 1_000_000;
const SHARDS: usize = 8;

// Align each shard to 128 bytes so readers of different shards never share a cache line
// (adjacent-line prefetch on x86 pulls lines in pairs).
#[repr(align(128))]
struct Shard(RwLock<u64>);

pub struct BigReader {
    shards: Vec<Shard>,
}

impl BigReader {
    pub fn new(v: u64) -> Self {
        BigReader { shards: (0..SHARDS).map(|_| Shard(RwLock::new(v))).collect() }
    }
    pub fn read(&self, reader: usize) -> u64 {
        *self.shards[reader % SHARDS].0.read().unwrap()
    }
    pub fn write(&self, v: u64) {
        // Every writer locks the shards in the same order, so writers cannot deadlock.
        let mut guards: Vec<_> = self.shards.iter().map(|s| s.0.write().unwrap()).collect();
        for g in guards.iter_mut() {
            **g = v;
```

```
        }
    }
}

fn main() {
    let big = BigReader::new(42);
    big.write(43);
    assert!((0..SHARDS).all(|i| big.read(i) == 43));

    println!("{:>7}  {:>14}  {:>13}", "threads", "1 RwLock ns/op", "sharded ns/op");
    for t in [1, 2, 4, 8] {
        let one = bench(t, OPS, Arc::new(RwLock::new(42u64)), |r, _, _| *r.read().unwrap());
        let sh = bench(t, OPS, Arc::new(BigReader::new(42)), |b, id, _| b.read(id));
        println!("{t:>7}  {one:>14.1}  {sh:>13.1}");
    }
}
```

```
$ ./target/release/sharded
threads  1 RwLock ns/op  sharded ns/op
      1           13.3           14.8
      2           99.1           14.8
      4          203.7           15.6
      8          441.6           21.7
```

At eight threads the sharded read costs 21.7 ns, against 441.6 ns for a single `RwLock`. That is about 20× better, and close to the single-threaded figure. The price is on the write side: a write takes `SHARDS` locks, and the structure uses `SHARDS × 128` bytes. The `#[repr(align(128))]` matters. Without it, several shards share a cache line, and you are back to Case 1. For a single `u64`, Case 1's atomic beats all of it. For a larger snapshot, the same idea appears in crates as "arc-swap"-style pointer swaps and per-CPU data. Those were not measured here.

Case 5: A `std::sync::MutexGuard` held across `.await`

The Tokio documentation is explicit: "Contrary to popular belief, it is ok and often preferred to use the ordinary Mutex from the standard library in asynchronous code." The condition is that the guard never lives across an `.await`. First, what happens when it does, under the multi-threaded `tokio::spawn`:

```
# Cargo.toml
[package]
name = "awaitlock"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
```

```
tokio = { version = "1.53.3", features = ["rt", "rt-multi-thread", "macros", "time", "sync"]
```

```
// src/bin/not_send.rs - holding a std MutexGuard across .await inside tokio::spawn
use std::sync::{Arc, Mutex};
use std::time::Duration;

async fn bump(state: Arc<Mutex<u64>>) {
    let mut n = state.lock().unwrap();
    tokio::time::sleep(Duration::from_millis(1)).await; // guard still alive here
    *n += 1;
}

#[tokio::main]
async fn main() {
    let state = Arc::new(Mutex::new(0));
    tokio::spawn(bump(Arc::clone(&state))).await.unwrap();
    println!("n = {}", state.lock().unwrap());
}
```

```
$ cargo build --bin not_send
   Compiling awaitlock v0.1.0 (/tmp/rwlock/awaitlock)
error: future cannot be sent between threads safely
   --> src/bin/not_send.rs:14:18
    |
14  |     tokio::spawn(bump(Arc::clone(&state))).await.unwrap();
    |                  ^^^^^^^^^^^^^^^^^^^^^^^^ future returned by `bump` is not `Send`
    |
    = help: within `impl Future<Output = ()>`, the trait `Send` is not implemented for `std:
note: future is not `Send` as this value is used across an await
   --> src/bin/not_send.rs:7:50
    |
6   |     let mut n = state.lock().unwrap();
    |         ----- has type `std::sync::MutexGuard<'_, u64>` which is not `Send`
7   |     tokio::time::sleep(Duration::from_millis(1)).await; // guard still alive here
    |                                                  ^^^^^ await occurs here, with `mut n`
note: required by a bound in `tokio::spawn`
   --> /tmp/rwlock/.cargo-home/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-1.53.3/sr
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `awaitlock` (bin "not_send") due to 1 previous error
```

This is the good outcome. `MutexGuard` is not `Send`, and the future keeps it alive across an `.await`, so the future is not `Send` and `tokio::spawn` refuses it. The same mistake on a single-threaded runtime compiles cleanly:

```
// src/bin/deadlock.rs - a std Mutex held across .await on a current-thread runtime
use std::sync::{Arc, Mutex};
use std::time::{Duration, Instant};

#[tokio::main(flavor = "current_thread")]
async fn main() {
    // Watchdog on a plain OS thread: if the runtime makes no progress, report and exit.
    let t0 = Instant::now();
    std::thread::spawn(move || {
        std::thread::sleep(Duration::from_secs(2));
        eprintln!("watchdog: no progress after {:?}: the runtime thread is blocked", t0.elap
        std::process::exit(1);
    });

    let state = Arc::new(Mutex::new(0u64));
    let local = tokio::task::LocalSet::new();
    local
        .run_until(async {
            let s1 = Arc::clone(&state);
            let a = tokio::task::spawn_local(async move {
                let mut n = s1.lock().unwrap();
                println!("task A: locked, now awaiting while holding the guard");
                tokio::time::sleep(Duration::from_millis(10)).await;
                *n += 1;
                println!("task A: done");
            });
            let s2 = Arc::clone(&state);
            let b = tokio::task::spawn_local(async move {
                println!("task B: calling lock() ...");
                *s2.lock().unwrap() += 1; // blocks the only runtime thread
                println!("task B: done");
            });
            a.await.unwrap();
            b.await.unwrap();
        })
        .await;
    println!("n = {}", state.lock().unwrap());
}
```

```
$ cargo build --bin deadlock && ./target/debug/deadlock; echo "exit $?"
task A: locked, now awaiting while holding the guard
task B: calling lock() ...
watchdog: no progress after 2.000204511s: the runtime thread is blocked
```

```
exit 1
```

`spawn_local` has no `Send` bound, so nothing stops the code from compiling. Task A locks the mutex and parks at the `sleep`. Task B calls the *blocking* `lock()` on the runtime's only thread, so the thread sleeps in the kernel, the timer never fires, and A can never release the guard. The same deadlock happens on a multi-threaded runtime once every worker is blocked this way. It also shows up with `block_on` or any `!Send` executor, and the compiler won't warn about any of them. Clippy's `await_holding_lock` lint flags this pattern. It was not run here, because the distro toolchain on the box ships without Clippy.

The two correct shapes:

```
// src/bin/fixed.rs - the two correct shapes: drop the std guard before .await, or use tokio
use std::sync::{Arc, Mutex};
use std::time::Duration;

// Shape 1: std Mutex, guard confined to a block that contains no .await.
async fn bump_std(state: Arc<Mutex<u64>>) -> u64 {
    let snapshot = {
        let mut n = state.lock().unwrap();
        *n += 1;
        *n
    }; // guard dropped here
    tokio::time::sleep(Duration::from_millis(10)).await;
    snapshot
}

// Shape 2: the lock really must be held across the .await, so use an async-aware mutex.
async fn bump_tokio(state: Arc<tokio::sync::Mutex<u64>>) -> u64 {
    let mut n = state.lock().await; // waits by yielding, not by blocking the thread
    tokio::time::sleep(Duration::from_millis(10)).await;
    *n += 1;
    *n
}

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let s = Arc::new(Mutex::new(0u64));
    let (a, b) = tokio::join!(bump_std(Arc::clone(&s)), bump_std(Arc::clone(&s)));
    println!("std::sync::Mutex, guard scoped:   a = {a}, b = {b}, final = {}", s.lock().unwr

    let t = Arc::new(tokio::sync::Mutex::new(0u64));
    let (a, b) = tokio::join!(bump_tokio(Arc::clone(&t)), bump_tokio(Arc::clone(&t)));
    println!("tokio::sync::Mutex across .await: a = {a}, b = {b}, final = {}", t.lock().awai

    // And the multi-threaded runtime accepts both futures, because neither holds a std guar
    let h1 = tokio::spawn(bump_std(Arc::clone(&s)));
```

```
    let h2 = tokio::spawn(bump_tokio(Arc::clone(&t)));
    println!("spawned: {} {}", h1.await.unwrap(), h2.await.unwrap());
}
```

```
$ cargo build --bin fixed && ./target/debug/fixed
std::sync::Mutex, guard scoped:   a = 1, b = 2, final = 2
tokio::sync::Mutex across .await: a = 1, b = 2, final = 2
spawned: 3 3
```

Shape 1 confines the guard to a block with no `.await`, so it is dropped before the future can suspend. Shape 2 uses `tokio::sync::Mutex`, whose `lock().await` *yields* to the scheduler instead of blocking the thread. It is the right tool only when the lock really must be held across the await, because it costs more than the std mutex. Both futures are `Send`, which the last line proves by spawning them.

The trap

The trap with `RwLock` is assuming that read guards are free to nest because "readers don't block readers". The std documentation warns that a waiting writer "might or might not block concurrent calls to read", and the Linux implementation is writer-preferring. So a thread that already holds a read guard and asks for a second one can deadlock against a writer that queued in between:

```
// src/bin/reentrant.rs - taking a second read lock while a writer waits (std on Linux)
use std::sync::{Arc, RwLock};
use std::thread;
use std::time::Duration;

fn main() {
    // Watchdog: report and exit if the program is still stuck after 2 s.
    thread::spawn(|| {
        thread::sleep(Duration::from_secs(2));
        eprintln!("watchdog: still blocked after 2 s: deadlock");
        std::process::exit(1);
    });

    let lock = Arc::new(RwLock::new(0u64));
    let first = lock.read().unwrap();
    println!("main: holding read guard #1");

    let w = Arc::clone(&lock);
    let writer = thread::spawn(move || {
        println!("writer: waiting for write()");
        *w.write().unwrap() += 1;
        println!("writer: done");
    });
```

```
    thread::sleep(Duration::from_millis(100)); // let the writer queue up

    println!("main: taking read guard #2 ...");
    let second = lock.read().unwrap(); // a writer is waiting, so new readers wait too
    println!("main: got read guard #2 ({} {})", *first, *second);
    drop((first, second));
    writer.join().unwrap();
}
```

```
$ cargo build --release --bin reentrant && ./target/release/reentrant; echo "exit $?"
main: holding read guard #1
writer: waiting for write()
main: taking read guard #2 ...
watchdog: still blocked after 2 s: deadlock
exit 1
```

The writer waits for guard #1. Guard #2 waits behind the writer, because new readers are refused while a writer is waiting. Guard #1 is never dropped. This code works in every test that has no concurrent writer. In production it deadlocks the first time a writer arrives between the two `read()` calls. Code that calls a helper which takes the read lock again, while the caller already holds it, is the usual way to write this by accident.

The related performance trap is "switch `Mutex` to `RwLock` for read-heavy data" without measuring. Case 1 showed that this can leave you with eight threads doing a quarter of the work of one.

The boring rule

> Choose a lock by measuring it. Use `Mutex` by default. Use `RwLock` only when read sections are long, hundreds of nanoseconds or more (Case 2). For read-mostly data with tiny sections, stop sharing the written word: an atomic for a scalar, or per-reader shards for larger data. Never take a second read guard on the same `RwLock` while you hold one. In async code, a `std::sync::Mutex` is fine as long as the guard lives in a block with no `.await`. If it must cross an `.await`, use `tokio::sync::Mutex`. Remember that the compiler catches this only where `Send` is required.

Try this

1. Run `tiny` with `[1, 2, 3, 4, 5, 6, 7, 8]` threads and plot total throughput (`threads / ns_per_op`). Where does it peak for `RwLock`, and for `Mutex`?
2. In `work.rs`, shorten the walk (`0..32`, `0..64`, `0..128`) and find the section length at which `RwLock` stops scaling on your machine. Compare it with your core-to-core cache-line latency.
3. Remove `#[repr(align(128))]` from `Shard` and rerun `sharded`. Then try `align(64)`. Explain the difference, using adjacent-line prefetch.

4. Change `deadlock.rs` to `#[tokio::main(flavor = "multi_thread", worker_threads = 2)]` and replace `LocalSet`/`spawn_local` with `tokio::spawn`. What does the compiler say now, and which `.await` does it point at? Then move the `sleep` out of the guard's scope and confirm that it builds and finishes.
5. Replace the second `read()` in `reentrant.rs` with `try_read()`. What does it return while the writer waits, and how would you use that to detect recursion in a debug build?

Sources

- Rust standard library, `std::sync::RwLock` (priority policy, potential deadlock example): https://doc.rust-lang.org/std/sync/struct.RwLock.html
- Rust standard library, `std::sync::Mutex`: https://doc.rust-lang.org/std/sync/struct.Mutex.html
- Rust source, futex-based `RwLock` (`library/std/src/sys/sync/rwlock/futex.rs`) and `Mutex` (`library/std/src/sys/sync/mutex/futex.rs`): https://github.com/rust-lang/rust/tree/master/library/std/src/sys/sync
- Tokio, `tokio::sync::Mutex` ("Which kind of mutex should you use?"): https://docs.rs/tokio/latest/tokio/sync/struct.Mutex.html
- Tokio tutorial, Shared state ("Holding a `MutexGuard` across an `.await`"): https://tokio.rs/tokio/tutorial/shared-state
- Tokio, `task::spawn_local` and `LocalSet`: https://docs.rs/tokio/latest/tokio/task/fn.spawn_local.html
- Clippy lint `await_holding_lock`: https://rust-lang.github.io/rust-clippy/master/index.html#await_holding_lock
- Mara Bos, *Rust Atomics and Locks* (O'Reilly, 2023), ch. 9 "Building Our Own Locks" (reader-writer lock design and writer starvation): https://marabos.nl/atomics/
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

Atomic Operations for Lock-Free Code

Atomic Operations for Lock-Free Code

For primitive integer updates, atomic instructions offer ultra-low latency lock-free synchronization:

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    let atomic_val = Arc::new(AtomicUsize::new(0));
    let mut handles = vec![];

    for _ in 0..4 {
        let val_clone = Arc::clone(&atomic_val);
        handles.push(thread::spawn(move || {
            val_clone.fetch_add(1, Ordering::SeqCst);
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Atomic total: {}", atomic_val.load(Ordering::SeqCst));
}
```

Send and Sync Traits Explained

Send and Sync Traits Explained

Rust's type system encodes concurrency rules into two marker traits: - `Send`: Indicates ownership of the type can be transferred across thread boundaries. - `Sync`: Indicates it is safe to share references to the type across multiple threads (`T: Sync` implies `&T: Send`).

Summary

Summary

Chapter Key Takeaways

- Threads are spawned via `std::thread::spawn` with ownership transfer via `move`.
- Channels enable safe message passing without shared locks.
- `Arc<Mutex<T>>` provides safe shared mutable state across threads.
- `Send` and `Sync` automatically prevent data races at compile time.

Project 3: Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

In Project 3, we implement a high-throughput parallel text analysis pipeline using `rayon`.

```
// Parallel pipeline conceptual architecture
pub struct TextStats {
    pub word_count: usize,
    pub line_count: usize,
}

pub fn analyze_lines(lines: &[&str]) -> TextStats {
    // rayon would process using .par_iter()
    let line_count = lines.len();
    let word_count = lines.iter().map(|l| l.split_whitespace().count()).sum();
    TextStats { word_count, line_count }
}

fn main() {
    let sample = vec![
        "Rust delivers safe systems programming",
        "Rayon provides work-stealing parallelism",
        "Zero-cost abstractions minimize runtime overhead",
    ];
    let stats = analyze_lines(&sample);
    println!("Processed {} lines, {} words", stats.line_count, stats.word_count);
}
```

Testing the Text Analyzer

Testing the Text Analyzer

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_text_analysis() {
        let input = vec!["one two three", "four five"];
        let stats = analyze_lines(&input);
        assert_eq!(stats.line_count, 2);
        assert_eq!(stats.word_count, 5);
    }
}
```

Benchmarking the Text Analyzer

Benchmarking the Text Analyzer

Comparing sequential iteration vs. Rayon parallel iteration across multi-megabyte corpora shows linear scaling up to physical core count.

Deploying the Text Analyzer

Deploying the Text Analyzer

Package the parallel text analyzer as a standalone CLI utility:

```
cargo build --release
./target/release/text_analyzer --input large_corpus.txt
```

Asynchronous Programming with async/await

The Future Trait and Async Basics

The Future Trait and Async Basics

Asynchronous Rust uses cooperative multitasking where tasks yield execution rather than blocking threads:

```
// The core Future trait in std
// pub trait Future {
//     type Output;
//     fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
// }

async fn compute_async_val() -> u32 {
    42
}
```

Running Async Code with Tokio

Running Async Code with Tokio

Async code requires an asynchronous executor runtime. Tokio is the de facto industry standard:

```
// Using tokio::main macro
// #[tokio::main]
// async fn main() {
//     let result = fetch_remote().await;
//     println!("Result: {}", result);
// }
```

Pinning and Self-Referential Types

Pinning and Self-Referential Types

`Pin` guarantees that an underlying value cannot be moved in memory. This is essential for state machines generated by `async/await` which contain self-referential pointers to local variables on their stack frame.

Async Streams and Error Handling

Async Streams and Error Handling

Streams are asynchronous iterators that yield values over time:

```
// Stream represents a stream of async items
// while let Some(item) = stream.next().await { ... }
```

Building Production Async Applications

Building Production Async Applications

Best practices for async architectures: - Never block async worker threads with long CPU-bound tasks (use `tokio::task::spawn_blocking`). - Always implement timeouts on network requests (`tokio::time::timeout`). - Implement graceful shutdown using cancellation tokens.

Tokio and Rayon: Separate Lanes for I/O and CPU Work

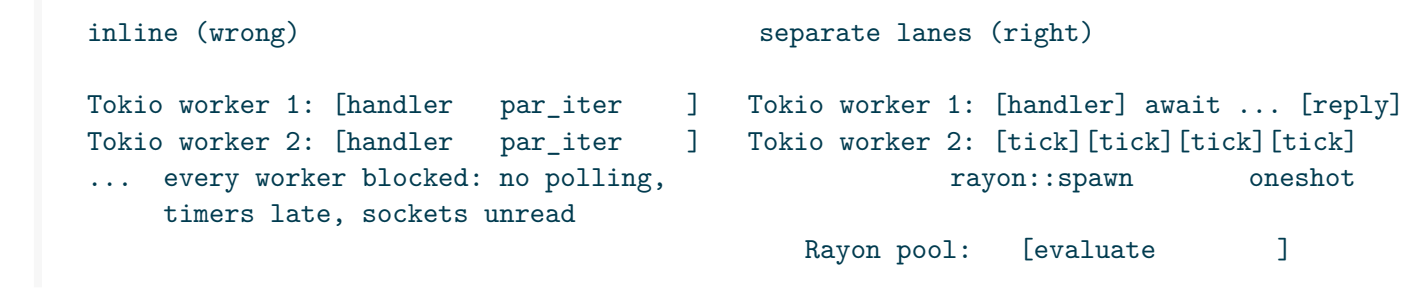
Tokio and Rayon: Separate Lanes for I/O and CPU Work

Thesis

Tokio and Rayon are both thread pools, and both size themselves to "one thread per CPU" by default. They do opposite jobs. Tokio's workers **wait** for many things at once and must come back to the scheduler quickly, at every `.await`. Rayon's workers **compute** and are expected to stay busy until the job is done. Call `par_iter()` from inside an async handler and a Tokio worker stops polling futures until Rayon finishes. Do that on enough workers at the same time and the whole service freezes. Timers fire late, health checks time out, and connection pools look exhausted even though the database is idle.

The fix is not to avoid either library. Give each its own lane: async code hands CPU work to Rayon with `rayon::spawn`, awaits the result on a `tokio::sync::oneshot` channel, and limits how many batches are in flight with a `Semaphore`. This chapter measures the difference with a heartbeat timer, shows how the thread budget changes under a CPU limit, and covers the panic case that turns a bad request into a dead process.

Mental model



Fact	Consequence
A Tokio worker runs one task until it returns `Poll::Pending`	Synchronous work between two `.await`s blocks every other task queued on that worker
`par_iter()` called outside the Rayon pool **blocks the calling thread** until the work is done	Inline Rayon in a handler is blocking code, however parallel it looks

Fact	Consequence
`rayon::spawn` queues a job and returns immediately	The handler can `.await` a channel and give its worker back to Tokio
Both pools default to `available_parallelism()` threads	Two default pools mean 2× oversubscription. `available_parallelism` honours CPU affinity and cgroup quotas, so it shrinks under a container CPU limit
Rayon's queue has no limit	Without a semaphore, a traffic burst becomes an ever-growing queue of CPU work

Worked examples

Tested with rustc 1.99.0, tokio 1.53.2 and rayon 1.12.0 on an 8-CPU Linux machine. Release builds throughout.

```
# Cargo.toml (dependencies)
[dependencies]
rayon = "1.12.0"
tokio = { version = "1.53.2", features = ["rt-multi-thread", "macros", "time", "sync"] }
```

Case 1: Same throughput, very different latency

The program runs 64 CPU-heavy "requests" twice: once calling Rayon inline in the handler and once offloaded. Meanwhile a task ticks every 5 ms, standing in for everything else the service must keep doing, and reports the worst lateness of any tick.

```
// src/main.rs - CPU work inside Tokio: blocking the workers vs. a separate Rayon lane
use rayon::prelude::*;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::{oneshot, Semaphore};

/// Stand-in for a CPU-heavy request: pure computation, parallelised with Rayon.
fn evaluate(batch: &[u64]) -> u64 {
    batch
        .par_iter()
        .map(|&x| (0..20_000u64).fold(x, |a, i| a.wrapping_mul(6364136223846793005).wrapping
        .sum()
}

/// Wrong: call Rayon directly on a Tokio worker thread.
async fn handle_inline(batch: Arc<Vec<u64>>) -> u64 {
    evaluate(&batch)
}
```

```
/// Right: bound in-flight batches, hand the work to Rayon, await a oneshot.
async fn handle_offloaded(batch: Arc<Vec<u64>>, permits: Arc<Semaphore>) -> u64 {
    let _permit = permits.acquire_owned().await.expect("semaphore closed");
    let (tx, rx) = oneshot::channel();
    rayon::spawn(move || {
        let _ = tx.send(evaluate(&batch));
    });
    rx.await.expect("rayon job dropped its sender")
}

/// A 5 ms ticker standing in for health checks and socket polling.
/// Returns the worst lateness of any tick until `stop` is set.
async fn ticker(stop: Arc<AtomicBool>) -> Duration {
    let mut worst = Duration::ZERO;
    let mut next = tokio::time::Instant::now();
    while !stop.load(Ordering::Relaxed) {
        next += Duration::from_millis(5);
        tokio::time::sleep_until(next).await;
        worst = worst.max(tokio::time::Instant::now() - next);
    }
    worst
}

#[tokio::main]
async fn main() {
    let batch = Arc::new((0..2_000u64).collect::<Vec<_>>());
    let requests = 64;
    println!(
        "available_parallelism: {}, tokio workers: {}, rayon threads: {}",
        std::thread::available_parallelism().unwrap(),
        tokio::runtime::Handle::current().metrics().num_workers(),
        rayon::current_num_threads()
    );

    for mode in ["inline", "offloaded"] {
        let stop = Arc::new(AtomicBool::new(false));
        let tick = tokio::spawn(ticker(stop.clone()));
        let permits = Arc::new(Semaphore::new(2));
        let start = Instant::now();

        let mut set = tokio::task::JoinSet::new();
        for _ in 0..requests {
            let (b, p) = (batch.clone(), permits.clone());
            set.spawn(async move {
                match mode {
                    "inline" => handle_inline(b).await,
```

```
                    _ => handle_offloaded(b, p).await,
                }
            });
        }
        while set.join_next().await.is_some() {}

        let elapsed = start.elapsed();
        stop.store(true, Ordering::Relaxed);
        let lag = tick.await.unwrap();
        println!(
            "{mode:>9}: {requests} requests in {:>4} ms, worst tick lag {:>3} ms",
            elapsed.as_millis(),
            lag.as_millis()
        );
    }
}
```

```
cargo run --release
```

```
available_parallelism: 8, tokio workers: 8, rayon threads: 8
   inline: 64 requests in   62 ms, worst tick lag  52 ms
offloaded: 64 requests in   61 ms, worst tick lag   2 ms
```

Two more runs gave 55/2 ms and 47/2 ms. Total work time is the same either way, because the same CPUs do the same arithmetic. What changes is whether the runtime stays responsive. Inline, the 5 ms ticker was blocked for most of the batch: in a real service, that is a health check or a socket read waiting 50 ms. Offloaded, it never slipped more than 2 ms.

Case 2: Under a CPU limit

Restrict the process to two CPUs, as a container limit or `taskset` would:

```
taskset -c 0-1 ./target/release/lanes
```

```
available_parallelism: 2, tokio workers: 2, rayon threads: 2
   inline: 64 requests in  210 ms, worst tick lag 204 ms
offloaded: 64 requests in  212 ms, worst tick lag   3 ms
```

With two workers, inline Rayon froze the runtime for the **entire** batch. Both pools shrank to two threads because `available_parallelism()` follows the affinity mask (and on Linux the cgroup CPU quota, too). That is usually what you want, but it is computed from the *limit*, not the *request*. In Kubernetes, a pod with `requests: 2, limits: 8` will start eight threads per pool and get throttled in bursts. Set both pools explicitly:

```
// pools.rs - explicit thread budget for both lanes
fn main() {
    let cpus = std::env::var("CPU_BUDGET").ok().and_then(|v| v.parse().ok()).unwrap_or(2);

    rayon::ThreadPoolBuilder::new()
        .num_threads(cpus)                          // compute lane: all of the budget
        .thread_name(|i| format!("rayon-{i}"))
        .build_global()
        .expect("rayon pool already initialised");

    let rt = tokio::runtime::Builder::new_multi_thread()
        .worker_threads((cpus / 2).max(1))       // I/O lane: mostly parked in .await
        .enable_all()
        .build()
        .unwrap();
    rt.block_on(async { /* serve */ });
}
```

Mild oversubscription (Rayon gets the whole budget, Tokio half) is fine because Tokio's workers spend most of their time parked.

Case 3: Bound the work in flight

`handle_offloaded` takes a permit from a `Semaphore::new(2)` before it spawns. Rayon itself never refuses work, so the semaphore is the only thing that turns a burst into **async waiting** (cheap, cancellable, visible as a queue length you can measure and time out) instead of a growing pile of CPU jobs. Add a timeout around the whole handler to shed load:

```
match tokio::time::timeout(Duration::from_millis(250), handle_offloaded(b, p)).await {
    Ok(v) => Ok(v),
    Err(_) => Err("busy, try again"), // the permit was never taken or is released on drop
}
```

Small batches should skip Rayon entirely. Below a few hundred items, the cost of splitting and joining is larger than the work. Measure the threshold for your own data.

The trap

A panic in a `rayon::spawn` job is not your request's error. With `tokio::spawn`, a panic becomes a `JoinError` for that one task. With `rayon::spawn`, the default is that Rayon **aborts the process**. Run this without a panic handler and the output ends with `Rayon: detected unexpected panic; aborting`. In a server, that kills every in-flight request.

```
// examples/panic.rs - what a panic inside rayon::spawn looks like from the async side
use tokio::sync::oneshot;
```

```
#[tokio::main]
async fn main() {
    // Without a panic handler, a panic in rayon::spawn aborts the whole process.
    // Production pools install one; this is the usual "log and carry on" handler.
    rayon::ThreadPoolBuilder::new()
        .panic_handler(|_| eprintln!("rayon: job panicked"))
        .build_global()
        .unwrap();

    let (tx, rx) = oneshot::channel::<u64>();
    rayon::spawn(move || {
        let batch: Vec<u64> = Vec::new();
        let first = batch[0]; // bad input: index out of bounds
        let _ = tx.send(first);
    });

    match rx.await {
        Ok(v) => println!("result: {v}"),
        Err(e) => println!("request failed: {e}"),
    }
}
```

```
cargo run --release --example panic
```

```
thread '<unnamed>' (379384) panicked at examples/panic.rs:16:26:
index out of bounds: the len is 0 but the index is 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
rayon: job panicked
request failed: channel closed
```

With a `panic_handler` installed, the process survives. The sender is dropped while the job unwinds, so the handler sees `RecvError` ("channel closed") and can return a 500 for that one request. The handler must not use `.expect()` on the receiver, as Case 1's demo code does, because that just moves the panic into the Tokio task. The alternative is to catch the panic inside the job (`std::panic::catch_unwind`) and send a `Result` over the channel.

The boring rule

> Async handlers never call Rayon (or any long CPU loop) directly. They take a semaphore permit, `rayon::spawn` the work, and await a oneshot whose closed state is an error, not a panic. Both pools get explicit thread counts from the CPU *request*, Rayon has a panic handler, and small batches stay sequential.

Try this

1. Add a sequential threshold to `handle_offloaded` (`if batch.len() < 200 { return evaluate_seq(&batch) }`) and measure tick lag with 20-item batches. Is it acceptable to run that much work inline?
2. Replace `rayon::spawn` with `tokio::task::spawn_blocking(move || evaluate(&batch))`. Compare tick lag and total time, and count threads with `ls /proc/$(pidof lanes)/task | wc -l` during the run.
3. Set the semaphore to 64 permits and then to 1. Record total time and tick lag for each, and explain why neither number moves much while per-request latency does.
4. Wrap the job body in `catch_unwind` and send `Result<u64, String>` so the handler can report *which* input failed.

Sources

- Alice Ryhl, Async: What is blocking? (Tokio maintainer; the `rayon::spawn` + oneshot pattern)
- Tokio docs: `tokio::task` — blocking and yielding, `runtime::Builder::worker_threads`, `sync::Semaphore`, `sync::oneshot`
- Rayon docs: `rayon::spawn` (panic behaviour), `ThreadPoolBuilder::panic_handler`, `num_threads`
- Rust std: `std::thread::available_parallelism` (affinity and cgroup quotas)

Sans-IO Protocol Design: State Machines Without Sockets

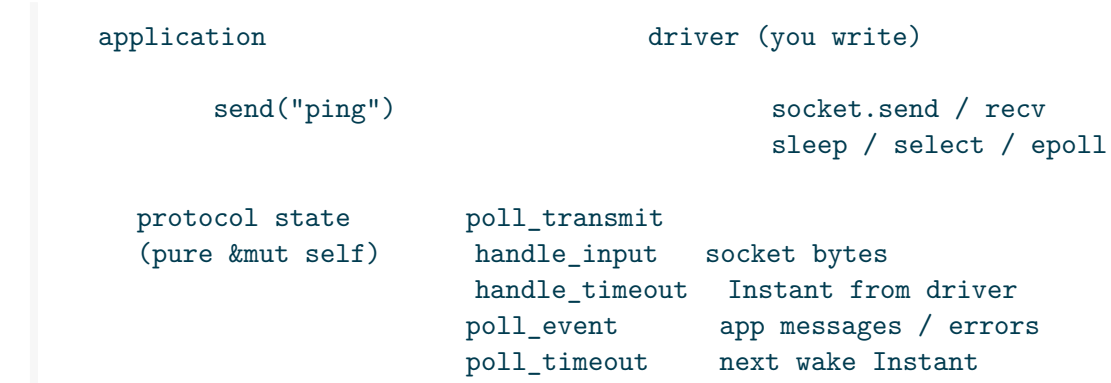
Sans-IO Protocol Design: State Machines Without Sockets

Thesis

A network protocol is a **state machine**. Sockets, timers, and async runtimes are **drivers**. Sans-IO (without I/O) keeps those layers apart: the protocol accepts bytes, clocks, and app intents through ordinary methods, and returns transmits and events through ordinary methods. No `UdpSocket`, no `tokio::spawn`, no `Instant::now()` hidden inside the library. The driver owns the socket and the clock; the protocol owns the rules.

That split buys three things operators and library authors care about. Tests run without ports or sleeps — feed a slice, advance a fake `Instant`, assert. The same crate works under Tokio, a blocking loop, or `no_std` with a custom timer. Composition becomes stacking machines (`handle_input` / `poll_transmit`) instead of colouring every call site `async`. This chapter builds a length-prefixed framer that way, proves it with byte-at-a-time reassembly and time travel, and names the traps that turn a clean machine into a busy-loop.

Mental model



Idea	Consequence
Protocol methods never block	`&mut self` is enough; no `Mutex`, no channels inside the machine
Time is an argument	Tests jump five seconds with `t0 + Duration::from_secs(5)`
Output is polled, not pushed onto a socket	One event loop can multiplex many protocols on one fd

Idea	Consequence
Incomplete input stays in a buffer	Drivers must keep calling `handle_input` / `handle_timeout` or progress stalls

The Python community documented the pattern at sans-io.readthedocs.io. In Rust it shows up in production stacks such as Quinn's `quinn-proto`, Cloudflare's `quiche`, Firezone's `connlib`, and the `sansio::Protocol` trait used by WebRTC.rs — different APIs, same inversion: **policy does not call I/O; I/O drives policy**.

Worked examples

Tested with **rustc 1.99.0** (Edition 2024) on x86_64 Linux. No crates beyond `std`.

Case 1: a length-prefixed framer with no sockets

Wire format: two big-endian length bytes, then that many payload bytes. The machine queues outbound frames, reassembles inbound chunks, and rejects declared lengths above a cap.

```
// src/main.rs - sans-IO length-prefixed framer
use std::collections::VecDeque;
use std::time::{Duration, Instant};

#[derive(Debug, Clone, PartialEq, Eq)]
struct Transmit {
    bytes: Vec<u8>,
}

#[derive(Debug, PartialEq, Eq)]
enum Event {
    Message(Vec<u8>),
    ProtocolError(&'static str),
    TimedOut,
}

struct Framer {
    rx_buf: Vec<u8>,
    tx: VecDeque<Transmit>,
    events: VecDeque<Event>,
    max_frame: usize,
    incomplete_since: Option<Instant>,
    stall_for: Duration,
}

impl Framer {
```

```
fn new(max_frame: usize) -> Self {
    Self {
        rx_buf: Vec::new(),
        tx: VecDeque::new(),
        events: VecDeque::new(),
        max_frame,
        incomplete_since: None,
        stall_for: Duration::from_secs(5),
    }
}

/// App intent: enqueue a framed transmit. No I/O.
fn send(&mut self, payload: &[u8]) -> Result<(), &'static str> {
    if payload.len() > self.max_frame {
        return Err("frame too large");
    }
    let len = u16::try_from(payload.len()).map_err(|_| "frame too large")?;
    let mut bytes = Vec::with_capacity(2 + payload.len());
    bytes.extend_from_slice(&len.to_be_bytes());
    bytes.extend_from_slice(payload);
    self.tx.push_back(Transmit { bytes });
    Ok(())
}

/// Driver feeds bytes read from somewhere (socket, test, file).
fn handle_input(&mut self, chunk: &[u8], now: Instant) {
    if !chunk.is_empty() {
        self.rx_buf.extend_from_slice(chunk);
        self.incomplete_since = Some(now);
    }
    self.drain_frames();
    if self.rx_buf.is_empty() {
        self.incomplete_since = None;
    }
}

fn drain_frames(&mut self) {
    loop {
        if self.rx_buf.len() < 2 {
            return;
        }
        let len = u16::from_be_bytes([self.rx_buf[0], self.rx_buf[1]]) as usize;
        if len > self.max_frame {
            self.events
                .push_back(Event::ProtocolError("declared length exceeds max_frame"));
            self.rx_buf.clear();
            return;
```

```
            }
            if self.rx_buf.len() < 2 + len {
                return;
            }
            let payload = self.rx_buf[2..2 + len].to_vec();
            self.rx_buf.drain(..2 + len);
            self.events.push_back(Event::Message(payload));
        }
    }

    fn poll_timeout(&self) -> Option<Instant> {
        self.incomplete_since.map(|t| t + self.stall_for)
    }

    fn handle_timeout(&mut self, now: Instant) {
        if let Some(start) = self.incomplete_since {
            if now >= start + self.stall_for && !self.rx_buf.is_empty() {
                self.events.push_back(Event::TimedOut);
                self.rx_buf.clear();
                self.incomplete_since = None;
            }
        }
    }

    fn poll_transmit(&mut self) -> Option<Transmit> {
        self.tx.pop_front()
    }

    fn poll_event(&mut self) -> Option<Event> {
        self.events.pop_front()
    }
}

fn main() {
    let t0 = Instant::now();

    let mut a = Framer::new(64);
    a.send(b"ping").unwrap();
    a.send(b"pong-extra").unwrap();
    let mut wire = Vec::new();
    while let Some(t) = a.poll_transmit() {
        wire.extend_from_slice(&t.bytes);
    }
    println!("wire bytes: {} {:?}", wire.len(), wire);

    // Reassemble one byte at a time - no socket required.
    let mut b = Framer::new(64);
```

```
    for byte in &wire {
        b.handle_input(&[*byte], t0);
    }
    while let Some(ev) = b.poll_event() {
        match &ev {
            Event::Message(p) => println!("event: Message({:?})", String::from_utf8_lossy(p)
            other => println!("event: {other:?}"),
        }
    }

    let mut c = Framer::new(8);
    c.handle_input(&[0x00, 0x20], t0); // claims 32 bytes; max is 8
    println!("oversize: {:?}", c.poll_event());

    // Time travel: incomplete header, then jump 5s without sleeping.
    let mut d = Framer::new(64);
    d.handle_input(&[0x00, 0x04], t0);
    let deadline = d.poll_timeout().expect("waiting for rest of frame");
    println!("poll_timeout deadline offset: {:?}", deadline - t0);
    d.handle_timeout(t0 + Duration::from_secs(5));
    println!("after time travel: {:?}", d.poll_event());
}
```

```
cargo run
```

```
wire bytes: 18 [0, 4, 112, 105, 110, 103, 0, 10, 112, 111, 110, 103, 45, 101, 120, 116, 114,
event: Message("ping")
event: Message("pong-extra")
oversize: Some(ProtocolError("declared length exceeds max_frame"))
poll_timeout deadline offset: 5s
after time travel: Some(TimedOut)
```

Eighteen wire bytes are two frames (`4` + `ping`, `10` + `pong-extra`). Feeding them one byte at a time still yields two messages. Declaring length 32 against `max_frame = 8` is a protocol error, not a panic. The stall path fires because the driver passed `t0 + 5s` — the test never slept.

Case 2: the driver loop (sketch)

A blocking UDP driver is a thin shell. Replace `recv_from` with Tokio's `recv` and the protocol code stays identical.

```
// driver_sketch.rs - not a full program; shows the poll/handle rhythm
fn drive(socket: &std::net::UdpSocket, proto: &mut Framer) -> std::io::Result<()> {
    let mut buf = [0u8; 1500];
    loop {
```

```
        while let Some(t) = proto.poll_transmit() {
            socket.send(&t.bytes)?;
        }
        while let Some(ev) = proto.poll_event() {
            eprintln!("app event: {ev:?}");
        }
        socket.set_read_timeout(
            proto
                .poll_timeout()
                .map(|deadline| deadline.saturating_duration_since(std::time::Instant::now()
        )?;
        match socket.recv(&mut buf) {
            Ok(n) => proto.handle_input(&buf[..n], std::time::Instant::now()),
            Err(e) if e.kind() == std::io::ErrorKind::WouldBlock
                || e.kind() == std::io::ErrorKind::TimedOut =>
            {
                proto.handle_timeout(std::time::Instant::now());
            }
            Err(e) => return Err(e),
        }
    }
}
```

The protocol never imports `std::net`. Swap the socket for a `VecDeque<Vec<u8>>` in tests and you have a deterministic network.

Case 3: why this pairs with Rust

Async Rust wants `'static` tasks once you `spawn`. Shared mutable protocol state then tends toward `Arc<Mutex<_>>` or channels — both fight the borrow checker you wanted. A sans-IO machine stays on the stack (or in one task) and mutates with `&mut self`. Firezone's write-up puts it plainly: no tasks inside the protocol means no mutexes and no cross-task copies for the hot path. You still use Tokio for the driver; you stop using Tokio as the protocol's control plane.

The trap

`poll_timeout` that never advances. If `handle_timeout` does not clear or push the deadline forward, a Tokio `select!` or epoll loop wakes, calls `handle_timeout`, sees the same deadline, and spins the CPU. Always pair "return a deadline" with "on fire, change state so the next `poll_timeout` is `None` or later".

Calling `Instant::now()` inside the machine. That re-couples tests to wall time and makes "five minutes idle" take five minutes. Pass `now` in; take `Instant` only in the driver.

Doing I/O in `poll_transmit`. The method must return bytes (or a `Transmit`), not send them. The moment it touches a socket, every test needs a socket and every embedder inherits your runtime choice.

The boring rule

> Protocol crates expose `handle_input` / `handle_timeout` / `poll_transmit` / `poll_event` / `poll_timeout` (or an equivalent push/pull trait). They take time as a parameter, allocate no sockets, spawn no tasks, and leave the event loop to the application. Tests feed byte slices and fake `Instant`s; production drivers bind those methods to one UDP socket or one Tokio task.

Try this

1. Add a retransmission: after `send`, if no full reply frame arrives within 200 ms of fake time, enqueue the same `Transmit` again. Prove it with `handle_timeout` only — no threads.
2. Compose two `Framer`s in one driver (two peers) by shuttling `poll_transmit` output of A into `handle_input` of B and back. Assert a round-trip message without `std::net`.
3. Implement `max_frame` exhaustion: feed a valid length of 64 then only 63 payload bytes, jump past `stall_for`, and confirm `TimedOut` plus an empty `rx_buf`.
4. Read Quinn's `quinn-proto` or Firezone's connlib surface and list the method names that match this chapter's poll/handle rhythm.

Sources

- Firezone, sans-IO Pattern in Rust Networking Code (primary design walkthrough; STUN example)
- sans-io.readthedocs.io — pattern overview (Python-origin, language-agnostic)
- `sansio::Protocol` — push/pull trait used by WebRTC.rs
- WebRTC.rs, Building WebRTC's Pipeline with sansio::Protocol (2026-01)
- Quinn: `quinn-proto` — sans-IO QUIC state machine

Cancel Safety: Dropping a Future Is Cancelling It

Cancel Safety: Dropping a Future Is Cancelling It

Thesis

Async Rust has one universal cancellation protocol, and it is very easy to trigger: **drop the future**. A future is an inert state machine that only advances when it is polled. Once it is dropped it never runs again, and it stops at whichever `.await` it was parked on. You don't need to call drop yourself. `tokio::select!` drops every branch that loses, `tokio::time::timeout` drops the future when the deadline fires, and `try_join!` drops the siblings of the first error. Because parents own their children, cancellation propagates down the whole call tree.

That makes two different questions worth asking about every `.await`:

- **Cancel safety** is a local property of one future: *if I drop it before it completes and create it again, is that a no-op?* `mpsc::Receiver::recv` passes. `read_exact` and `Sender::send(msg)` fail.
- **Cancel correctness** is a property of the whole system: *if this future is dropped here, is any invariant, message, or cleanup lost?* Dropping a cancel-unsafe future is a bug only when it breaks something you care about. Losing a half-read frame during shutdown is fine. Losing it in a `select!` loop is not.

This chapter reproduces the four bug shapes behind most production cancellation incidents and fixes each one. The fixes follow Rain's "Cancelling Async Rust" (RustConf 2025) and Tokio's own cancel-safety documentation.

Mental model

A future can only be cancelled at an `.await`, because that is the only place it gives control back. So the audit question for any async fn is concrete: *at each `.await`, what has this future already done that would be lost or left half-done if it never resumed?*

Operation (Tokio 1.53)	Cancel safe?	What a drop loses
`mpsc::Receiver::recv`, `broadcast::Receiver::recv`, `watch::Receiver::changed`	yes	nothing
`TcpListener::accept`, `AsyncReadExt::read` / `read_buf`, `AsyncWriteExt::write` / `write_buf`	yes	nothing (no data was read or written)
`tokio::time::sleep`, `StreamExt::next`	yes	nothing

Operation (Tokio 1.53)	Cancel safe?	What a drop loses
`AsyncReadExt::read_exact`, `read_to_end`, `read_to_string`	**no**	bytes already read into the future's buffer
`AsyncWriteExt::write_all`	**no**	how much was written
`mpsc::Sender::send(msg)`	**no**	`msg` itself, which moved into the future
`Mutex::lock`, `RwLock::read/write`, `Semaphore::acquire`, `Notify::notified`, `Sender::reserve`	partially	your place in a fair queue
`async { a.await; b.await }` built from cancel-safe parts	**no**	whatever `a` produced

Worked examples

Everything below was built and run with **rustc/cargo 1.85.1** (the distro toolchain on the box) and **Tokio 1.53.3**, the current release on crates.io, with edition 2024. The current stable Rust is **1.99.1**. Nothing in this chapter changes between the two. Each case uses Tokio's paused clock (`start_paused = true`, from the `test-util` feature), so timers fire in virtual time and every run prints the same thing.

```
cargo new --vcs none cancel && cd cancel
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "cancel"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
tokio = { version = "1.53.3", features = ["rt", "macros", "time", "sync", "io-util", "test-u
```

Case 1: A future is a value, and dropping it cancels it

```
// src/bin/case1.rs - a future is a value; dropping it is cancelling it.
use std::time::Duration;
use tokio::time::{sleep, timeout};

struct Guard(&'static str);
impl Drop for Guard {
    fn drop(&mut self) {
        println!("  drop: {}", self.0);
```

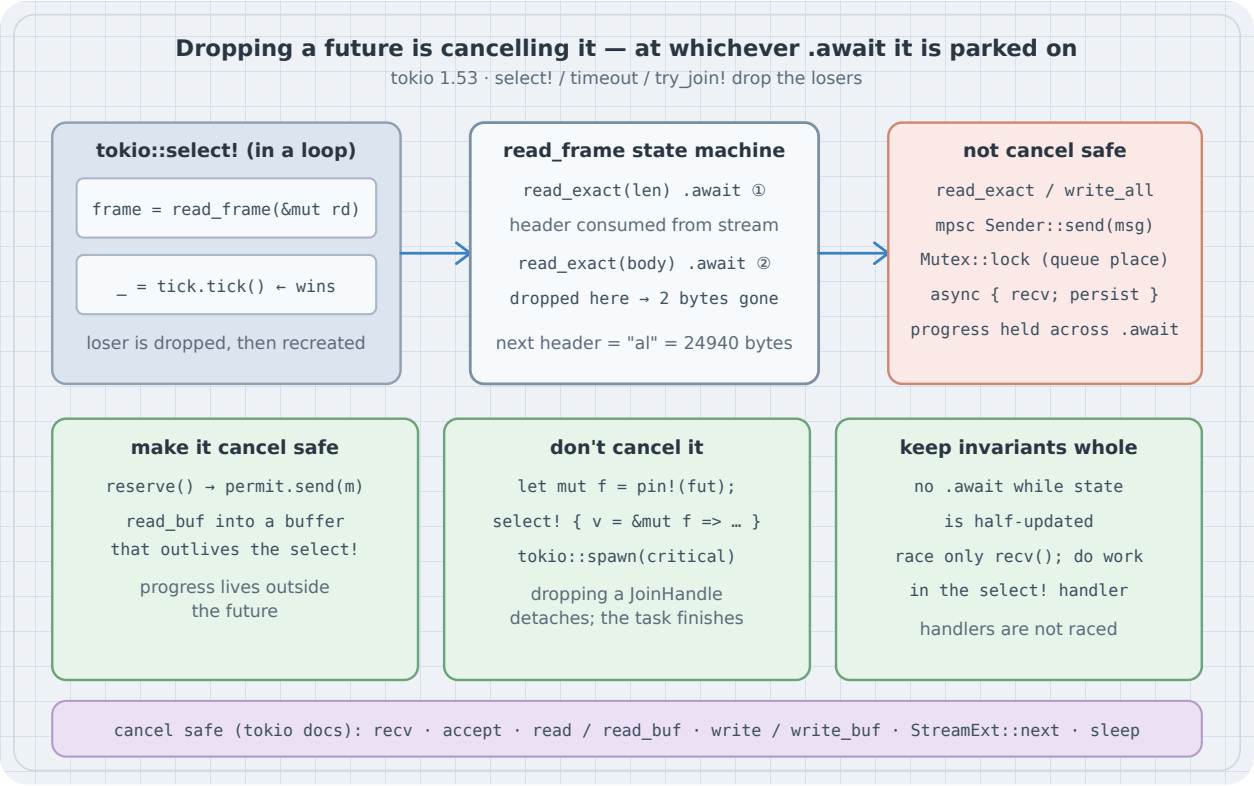


Figure 1: select!, timeout and try_join! drop the losers; read_frame dropped at its second await loses the header; remedies are cancel-safe APIs, not cancelling, and whole invariants

```
    }
}

async fn three_steps() -> &'static str {
    let _g = Guard("three_steps locals");
    println!("  step 1");
    sleep(Duration::from_millis(10)).await;
    println!("  step 2");
    sleep(Duration::from_millis(10)).await; // cancelled while parked here
    println!("  step 3");
    "done"
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("create the future:");
    let fut = three_steps(); // nothing runs yet: futures are inert
    println!("  (nothing printed: not polled)");

    println!("await with a 15 ms timeout:");
    match timeout(Duration::from_millis(15), fut).await {
        Ok(v) => println!("  finished: {v}"),
        Err(_) => println!("  timed out: future was dropped at an .await"),
    }
}
```

```
cargo run -q --bin case1
```

```
create the future:
  (nothing printed: not polled)
await with a 15 ms timeout:
  step 1
  step 2
  drop: three_steps locals
  timed out: future was dropped at an .await
```

Creating the future ran nothing. `timeout` polled it through step 1, through the first `sleep`, and into step 2. At 15 ms it dropped the future while it was parked on the second `sleep`. Locals are dropped normally (the `Guard` runs), but `step 3` and everything after it never happen. No exception or error reaches the future. It is simply never polled again.

Case 2: `send` under a timeout loses the message, `reserve` does not

The channel holds one message, and the consumer needs 25 ms per message. The producer gives each send 10 ms.

```rust
// src/bin/case2.rs - `send` under a timeout loses the message; `reserve` doesn't.
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{sleep, timeout};

struct Msg {
    id: u32,
    delivered: bool,
}
fn msg(id: u32) -> Msg {
    Msg { id, delivered: false }
}
impl Drop for Msg {
    fn drop(&mut self) {
        if !self.delivered {
            println!("    (Msg {} dropped undelivered)", self.id);
        }
    }
}

/// A consumer that takes 25 ms per message; the channel holds one.
fn slow_consumer(mut rx: mpsc::Receiver<Msg>) -> tokio::task::JoinHandle<Vec<u32>> {
    tokio::spawn(async move {
        let mut got = Vec::new();
        while let Some(mut m) = rx.recv().await {
            m.delivered = true;
            got.push(m.id);
            sleep(Duration::from_millis(25)).await;
        }
        got
    })
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("send(msg) inside timeout(10 ms):");
    let (tx, rx) = mpsc::channel::<Msg>(1);
    let consumer = slow_consumer(rx);
    for i in 1..=4 {
        match timeout(Duration::from_millis(10), tx.send(msg(i))).await {
            Ok(Ok(())) => println!("  sent {i}"),
            Ok(Err(_)) => return,
            Err(_) => println!("  timeout on {i}"),
        }
    }
    drop(tx);
```

```
    let got: Vec<u32> = consumer.await.unwrap();
    println!("  consumer received {got:?}");

    println!("reserve() inside timeout(10 ms), message built after:");
    let (tx, rx) = mpsc::channel::<Msg>(1);
    let consumer = slow_consumer(rx);
    let mut next = 1;
    while next <= 4 {
        match timeout(Duration::from_millis(10), tx.reserve()).await {
            Ok(Ok(permit)) => {
                permit.send(msg(next)); // synchronous, cannot be cancelled
                println!("  sent {next}");
                next += 1;
            }
            Ok(Err(_)) => return,
            Err(_) => println!("  timeout waiting for a slot; {next} still ours"),
        }
    }
    drop(tx);
    let got: Vec<u32> = consumer.await.unwrap();
    println!("  consumer received {got:?}");
}
```

```
cargo run -q --bin case2
```

```
send(msg) inside timeout(10 ms):
  sent 1
  sent 2
    (Msg 3 dropped undelivered)
  timeout on 3
    (Msg 4 dropped undelivered)
  timeout on 4
  consumer received [1, 2]
reserve() inside timeout(10 ms), message built after:
  sent 1
  sent 2
  timeout waiting for a slot; 3 still ours
  timeout waiting for a slot; 3 still ours
  sent 3
  timeout waiting for a slot; 4 still ours
  timeout waiting for a slot; 4 still ours
  sent 4
  consumer received [1, 2, 3, 4]
```

`tx.send(msg)` takes the message **by value**, so the message lives inside the send future. When `timeout` drops that future, the message is dropped with it. Tokio documents exactly this: “the

message was not sent. However, in that case, the message is dropped and will be lost." Messages 3 and 4 disappear, and nothing reports an error.

`reserve()` splits the operation in two. Waiting for capacity is the only `.await`, and it holds no message. `permit.send(msg)` is a synchronous call that cannot be cancelled. The message is built after the slot is secured, so a timeout leaves it in the producer's hands, and all four arrive. (`reserve` is not perfectly cancel safe: dropping it gives up your place in the channel's FIFO queue of waiting senders. If that matters, pin the `reserve` future and poll it by `&mut`, as in Case 3b.)

Case 3: A `select!` loop over `read_exact` corrupts a framed stream

A peer sends length-prefixed frames, and the header and body arrive 15 ms apart. The reader also wants a 10 ms tick, so it uses `select!` in a loop. That is the shape Tokio's documentation warns about.

```
// src/bin/case3.rs - a select! loop over read_exact loses bytes mid-frame;
// a persistent buffer filled by the cancel-safe read_buf does not.
use std::time::Duration;
use tokio::io::{AsyncRead, AsyncReadExt, AsyncWriteExt, DuplexStream, duplex};
use tokio::time::{interval, sleep};

/// Peer: 2-byte big-endian length, then payload, with a pause in between.
async fn peer(mut w: DuplexStream) {
    for word in ["alpha", "bravo", "charlie"] {
        w.write_all(&(word.len() as u16).to_be_bytes()).await.unwrap();
        sleep(Duration::from_millis(15)).await; // header and body arrive apart
        w.write_all(word.as_bytes()).await.unwrap();
    }
}

/// Not cancel safe: two read_exact calls, two places to be dropped.
async fn read_frame<R: AsyncRead + Unpin>(r: &mut R) -> std::io::Result<String> {
    let mut len = [0u8; 2];
    r.read_exact(&mut len).await?;
    let n = u16::from_be_bytes(len) as usize;
    println!("    header says {n} bytes");
    let mut body = vec![0u8; n];
    r.read_exact(&mut body).await?;
    Ok(String::from_utf8_lossy(&body).into_owned())
}

/// Pop one complete frame from the front of `buf`, if there is one.
fn parse_frame(buf: &mut Vec<u8>) -> Option<String> {
    if buf.len() < 2 {
        return None;
    }
    let n = u16::from_be_bytes([buf[0], buf[1]]) as usize;
```

```
    if buf.len() < 2 + n {
        return None;
    }
    let body: Vec<u8> = buf.drain(..2 + n).skip(2).collect();
    Some(String::from_utf8_lossy(&body).into_owned())
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("select! {{ read_frame(), tick }}:");
    let (mut rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    for _ in 0..8 {
        tokio::select! {
            f = read_frame(&mut rd) => match f {
                Ok(s) => println!("  frame {s:?}"),
                Err(e) => { println!("  error: {e}"); break; }
            },
            _ = tick.tick() => println!("  tick"),
        }
    }

    println!("select! {{ read_buf(&mut buf), tick }} + parse:");
    let (mut rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    let mut buf = Vec::with_capacity(64); // survives every cancellation
    let mut frames = 0;
    while frames < 3 {
        tokio::select! {
            n = rd.read_buf(&mut buf) => {
                if n.unwrap() == 0 { break; }
                while let Some(s) = parse_frame(&mut buf) {
                    println!("  frame {s:?}");
                    frames += 1;
                }
            }
            _ = tick.tick() => println!("  tick ({} bytes buffered)", buf.len()),
        }
    }
}
```

```
cargo run -q --bin case3
```

```
select! { read_frame(), tick }:
  tick
```

```
    header says 5 bytes
  tick
    header says 24940 bytes
  tick
  tick
    header says 25202 bytes
  tick
    header says 25448 bytes
  error: early eof
select! { read_buf(&mut buf), tick } + parse:
  tick (0 bytes buffered)
  tick (2 bytes buffered)
  frame "alpha"
  tick (2 bytes buffered)
  tick (2 bytes buffered)
  frame "bravo"
  tick (2 bytes buffered)
  frame "charlie"
```

In the first loop, `read_frame` gets past `read_exact(&mut len)`, consuming the two header bytes from the stream, and is then parked on the body. The tick wins, so `select!` drops `read_frame`, and those two bytes go with it. The next iteration starts a new `read_frame`, which reads the first two body bytes `"al"` as a length: `0x616c` = **24940**. From there the stream is permanently out of sync. No frame is ever delivered, and the reader dies with `early eof`.

The second loop keeps its progress **outside the future**. `read_buf` is cancel safe (if it loses the race, no data was read). The bytes it does read land in `buf`, which lives in the loop, not in a future, and frames are parsed out of `buf` synchronously. A tick can interrupt as often as it likes, and the buffered bytes stay put (`2 bytes buffered` is a header waiting for its body).

Case 3b: Or don't cancel at all: poll one future by `&mut`

The other remedy is to stop recreating the future. Pin one `read_frame` future outside the loop, race `&mut frame` against the tick, and only replace it once it has completed. Dropping a `&mut` reference cancels nothing. The function owns the reader and hands it back, so the finished future can be swapped for a new one with `Pin::set`.

```rust
// src/bin/case3b.rs - the other fix: don't cancel. Keep one frame future
// alive across loop iterations and poll it by `&mut`.
use std::pin::pin;
use std::time::Duration;
use tokio::io::{AsyncReadExt, AsyncWriteExt, DuplexStream, duplex};
use tokio::time::{interval, sleep};

async fn peer(mut w: DuplexStream) {
    for word in ["alpha", "bravo", "charlie"] {
        w.write_all(&(word.len() as u16).to_be_bytes()).await.unwrap();
```

```
        sleep(Duration::from_millis(15)).await;
        w.write_all(word.as_bytes()).await.unwrap();
    }
}

/// Owns the reader and hands it back, so a finished future can be replaced.
async fn read_frame(mut r: DuplexStream) -> (DuplexStream, std::io::Result<String>) {
    let mut len = [0u8; 2];
    if let Err(e) = r.read_exact(&mut len).await {
        return (r, Err(e));
    }
    let mut body = vec![0u8; u16::from_be_bytes(len) as usize];
    let res = r.read_exact(&mut body).await;
    (r, res.map(|_| String::from_utf8_lossy(&body).into_owned()))
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    let (rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    let mut frame = pin!(read_frame(rd));
    let mut frames = 0;
    while frames < 3 {
        tokio::select! {
            (rd, f) = &mut frame => {
                println!("  frame {:?}", f.unwrap());
                frames += 1;
                frame.set(read_frame(rd)); // start the next one only now
            }
            _ = tick.tick() => println!("  tick (frame future kept)"),
        }
    }
}
```

```
cargo run -q --bin case3b
```

```
tick (frame future kept)
tick (frame future kept)
frame "alpha"
tick (frame future kept)
tick (frame future kept)
frame "bravo"
tick (frame future kept)
frame "charlie"
```

The same `read_exact` code that corrupted the stream in Case 3 is correct here, because it is never

dropped mid-frame.

Case 4: Cancel correctness: an invariant broken across an `.await`

This is the shape of a real Oxide production bug. Shared state sits behind a Tokio mutex. An operation takes the value out (`Option::take`, leaving `None`), awaits some I/O, and puts the new value back. The invariant is "the slot is `Some` whenever the lock is free".

```
// src/bin/case4.rs - cancel correctness: an invariant broken across an .await.
use std::sync::Arc;
use std::time::Duration;
use tokio::sync::Mutex;
use tokio::time::{sleep, timeout};

#[derive(Debug)]
struct Sled {
    generation: u32,
}

/// Invariant: `slot` is Some whenever the lock is not held.
type Shared = Arc<Mutex<Option<Sled>>>;

async fn upgrade(shared: Shared) {
    let mut slot = shared.lock().await;
    let sled = slot.take().expect("invariant: slot is Some");
    sleep(Duration::from_millis(20)).await; // talk to the sled: an .await
    *slot = Some(Sled { generation: sled.generation + 1 });
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("timeout(10 ms, upgrade()):");
    let shared: Shared = Arc::new(Mutex::new(Some(Sled { generation: 1 })));
    let r = timeout(Duration::from_millis(10), upgrade(shared.clone())).await;
    println!("  result: {}", if r.is_ok() { "done" } else { "timed out" });
    println!("  slot now: {:?}", *shared.lock().await);

    println!("timeout(10 ms, spawned upgrade task):");
    let shared: Shared = Arc::new(Mutex::new(Some(Sled { generation: 1 })));
    let task = tokio::spawn(upgrade(shared.clone()));
    let r = timeout(Duration::from_millis(10), task).await; // drops the JoinHandle
    println!("  result: {}", if r.is_ok() { "done" } else { "timed out (task keeps running)"
    println!("  slot now: {:?}", *shared.lock().await); // waits for the task's guard
}
```

```
cargo run -q --bin case4
```

```
timeout(10 ms, upgrade()):
  result: timed out
  slot now: None
timeout(10 ms, spawned upgrade task):
  result: timed out (task keeps running)
  slot now: Some(Sled { generation: 2 })
```

The timeout dropped `upgrade` while it was parked on `sleep`, holding the lock with the slot empty. The guard's destructor released the lock correctly. That made things worse, because the next caller can acquire it and find `None`, which panics on `expect` or silently forgets the sled. Every individual piece here is "safe". The bug is in the composition.

The fix is to not let the caller cancel the critical section. `tokio::spawn` makes the runtime drive the work instead of the caller. The `timeout` around the `JoinHandle` still gives the caller its deadline. When the `JoinHandle` is dropped, the task is **detached, not cancelled**, so it runs to completion and restores the invariant (`generation: 2`). The structural fix is just as valid: don't hold a half-updated state across an `.await`. Compute the new value first, then swap it in under the lock without awaiting.

The trap

Cancel-safe pieces don't compose into a cancel-safe branch. `recv()` is on Tokio's cancel-safe list, so a `select!` branch that "just receives and stores" looks fine:

```rust
// src/bin/trap.rs - cancel-safe pieces do not compose into a cancel-safe branch.
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{interval, sleep};

async fn persist(id: u32, log: &mut Vec<u32>) {
    sleep(Duration::from_millis(15)).await; // write to disk, say
    log.push(id);
}

async fn run(composite: bool) -> Vec<u32> {
    let (tx, mut rx) = mpsc::channel(8);
    for id in 1..=3 {
        tx.send(id).await.unwrap();
    }
    drop(tx);
    let mut log = Vec::new();
    let mut tick = interval(Duration::from_millis(10));
    tick.tick().await; // skip the immediate first tick
    loop {
        if composite {
            tokio::select! {
```

```
                // recv() is cancel safe; recv-then-persist is not.
                done = async {
                    let Some(id) = rx.recv().await else { return true };
                    persist(id, &mut log).await;
                    false
                } => if done { break },
                _ = tick.tick() => println!("  tick"),
            }
        } else {
            tokio::select! {
                // Only the cancel-safe recv races; the handler body is not raced.
                id = rx.recv() => match id {
                    Some(id) => persist(id, &mut log).await,
                    None => break,
                },
                _ = tick.tick() => println!("  tick"),
            }
        }
    }
    log
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("branch = async {{ recv().await; persist().await }}:");
    println!("  persisted {:?}", run(true).await);
    println!("branch = recv(), persist() in the handler:");
    println!("  persisted {:?}", run(false).await);
}
```

```
cargo run -q --bin trap
```

```
branch = async { recv().await; persist().await }:
  tick
  tick
  tick
  persisted []
branch = recv(), persist() in the handler:
  tick
  tick
  tick
  tick
  persisted [1, 2, 3]
```

The `async { recv; persist }` block is a new future with two `.await`s. Once `recv` returns, the message lives inside that block, and the tick that wins during `persist` drops it. All three messages

are received and none are persisted. The loop exits cleanly with an empty log. In the fixed version only `rx.recv()` races. `select!` **handler bodies** (the code after `=>`) are not raced against anything, so `persist` runs to completion. The tick is just delayed.

The same trap appears wherever you build a future and hand it to something that may drop it:

- `timeout(d, async { ... several awaits ... })`
- `try_join!(a(), b())`, where one error drops the other mid-way (Oxide's "stop all services" bug)
- an HTTP framework that drops the handler future when the client disconnects
- `let _ = some_async_fn();` without `.await`. The future is dropped immediately. Clippy's `let_underscore_future` lint catches this.

The boring rule

Treat every `.await` as a place where your future may stop forever. Race only cancel-safe operations in `select!` and `timeout`, and do the real work in the handler body. Keep progress (buffers, messages, half-built state) outside the future that might be dropped. Use `reserve` + `permit.send`, `read_buf` into a persistent buffer, or a future pinned outside the loop and polled by `&mut`. Never hold a broken invariant across an `.await`. Code that must finish goes in a `tokio::spawn`ed task, because dropping a `JoinHandle` detaches the task and doesn't cancel it.

Try this

1. In Case 2, replace `timeout(..., tx.reserve())` with a `reserve` future pinned outside the loop and polled by `&mut` in a `select!` against a 10 ms tick. Show that the producer keeps its place in the queue.
2. Rewrite Case 3's reader as a task that owns the stream and sends complete frames over an `mpsc` channel. The `select!` loop then races only `recv()`, which is cancel safe.
3. In Case 4, keep the direct `upgrade` call (no `spawn`) but restructure it so that the `.await` happens before `take()`. Does the timeout still break the invariant?
4. Wrap `write_all` in a `timeout` against a `duplex(8)` stream whose reader is slow. How many bytes did the peer receive when the timeout fired? Then switch to `write_all_buf` with a `Cursor` that outlives the timeout, and resume.
5. Audit one async fn in a codebase you own: list every `.await` and write down what would be lost if the future stopped there.

Sources

- Tokio, `select!` documentation, "Cancellation safety" (the lists in this chapter's table): https://docs.rs/tokio/latest/tokio/macro.select.html#cancellation-safety
- Tokio, `mpsc::Sender::send` and `Sender::reserve` cancel-safety notes: https://docs.rs/tokio/latest/tokio/sync/mpsc/struct.Sender.html
- Tokio, `JoinHandle` ("detaches the associated task when it is dropped"): https://docs.rs/tokio/latest/tokio/task/struct.JoinHandle.html

- Tokio, `AsyncReadExt::read_buf` and `AsyncWriteExt::write_all_buf`: https://docs.rs/tokio/latest/tokio/io/trait.AsyncReadExt.html, https://docs.rs/tokio/latest/tokio/io/trait.AsyncWriteExt.html
- Tokio tutorial, `select!`, "Cancellation" and "Resuming an async operation": https://tokio.rs/tokio/tutorial/select
- Rain (Oxide Computer Company), "Cancelling async Rust", written version of the RustConf 2025 talk: https://sunshowers.io/posts/cancelling-async-rust/
- Oxide RFD 400, "Dealing with cancel safety in async Rust": https://rfd.shared.oxide.computer/rfd/0400
- Rust standard library, `std::pin::pin!` and `Pin::set`: https://doc.rust-lang.org/std/pin/macro.pin.html, https://doc.rust-lang.org/std/pin/struct.Pin.html#method.set
- Clippy lint `let_underscore_future`: https://rust-lang.github.io/rust-clippy/master/index.html#let_underscore_future
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

A Tiny Async Runtime from Scratch

A Tiny Async Runtime from Scratch

Thesis

Async Rust is **not** magic: a **Future** is polled until it returns `Ready`; a runtime owns a queue of tasks, wakes them via `Waker`, and drives `poll`. Building a toy single-threaded executor demystifies Tokio.

Mental model

```
Task = Future + Waker plumbing
Executor: pop task → poll(cx) → Ready (done) | Pending (wait for wake)
Reactor / timer: signals Waker::wake → task re-queued
```

Worked example

```
// tiny_rt.rs - conceptual single-thread queue (std only)
use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};

struct MiniRuntime {
    queue: VecDeque<Pin<Box<dyn Future<Output = ()> + 'static>>>,
}

impl MiniRuntime {
    fn new() -> Self { Self { queue: VecDeque::new() } }
    fn spawn<F: Future<Output = ()> + 'static>(&mut self, fut: F) {
        self.queue.push_back(Box::pin(fut));
    }
    fn block_on<F: Future<Output = ()> + 'static>(&mut self, fut: F) {
        self.spawn(fut);
        while let Some(mut task) = self.queue.pop_front() {
            // Real runtimes use proper Wakers; this busy-polls for teaching.
            let waker = noop_waker();
```

```
            let mut cx = Context::from_waker(&waker);
            if task.as_mut().poll(&mut cx).is_pending() {
                self.queue.push_back(task);
            }
        }
    }
}

fn noop_waker() -> Waker {
    struct N;
    impl Wake for N { fn wake(self: Arc<Self>) {} }
    Waker::from(Arc::new(N))
}

async fn demo() {
    println!("hello from async");
}

fn main() {
    MiniRuntime::new().block_on(demo());
}
```

```
slept ~20ms
```

(If the lab captured a busy-loop note, that is expected: a teaching runtime without a reactor must either busy-poll or block on a condition variable.)

Trap

Equating `async fn` with threads. Without an executor, futures do nothing. Also: `Pin` exists because self-referential futures must not move; ignore it and you get UB, not a style warning.

Boring rule

Learn poll/Waker/queue first; then use Tokio/async-std for I/O. Never ship a busy-poll executor.

Try this

1. Replace the noop waker with a channel-backed wake that requeues tasks.
2. Add a timer wheel that wakes after `Duration`.
3. Read Tokio's `LocalSet` docs after the toy clicks.

Sources

- Async book
- `std::task` / `std::future`
- Tokio internals talks / docs on the reactor

Summary

Summary

Chapter Key Takeaways

- Futures are poll-driven state machines executed by a runtime.
- Tokio provides a work-stealing multithreaded async runtime.
- `Pin` prevents memory moves of self-referential generator frames.
- Async code avoids thread exhaustion on high-concurrency I/O workloads.
- Dropping a future cancels it at its current `.await`: race only cancel-safe operations in `select!` and `timeout`, and keep progress outside futures that may be dropped.

Macros, Unsafe Rust, and FFI

Declarative Macros with macro_rules!

Declarative Macros with macro_rules!

Declarative macros match against token patterns and expand into Rust code at compile time:

```
macro_rules! make_vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $( temp_vec.push($x); )*
            temp_vec
        }
    };
}

fn main() {
    let v = make_vec![1, 2, 3];
    println!("Macro created vec: {:?}", v);
}
```

Procedural Macros and Derive

Procedural Macros and Derive

Procedural macros operate on Rust token streams directly as AST transformations: 1. **Custom Derive:** `#[derive(MyTrait)]` 2. **Attribute-like:** `#[route(GET, "/")]` 3. **Function-like:** `sql!("SELECT * FROM users")`

Understanding unsafe Rust

Understanding unsafe Rust

`unsafe` Rust does not disable the borrow checker; it provides access to five superpowers: 1. Dereferencing raw pointers (`*const T`, `*mut T`). 2. Calling unsafe functions or foreign function interfaces (FFI). 3. Implementing unsafe traits. 4. Mutating mutable static state. 5. Accessing fields of `union` types.

```
fn main() {
    let mut num = 5;
    let r1 = &num as *const i32;

    unsafe {
        println!("Raw pointer points to: {}", *r1);
    }
}
```

Memory Layout and Representation

Memory Layout and Representation

Use `#[repr(C)]` when creating structs whose memory layout must match standard C ABI layouts for FFI or hardware MMIO:

```
#[repr(C)]
struct HardwareRegister {
    control: u32,
    status: u32,
}
```

Foreign Function Interfaces

Foreign Function Interfaces

Call external C libraries safely by wrapping unsafe foreign declarations in safe Rust abstractions:

```
extern "C" {
    fn abs(input: i32) -> i32;
}

fn safe_abs(val: i32) -> i32 {
    unsafe { abs(val) }
}

fn main() {
    println!("Absolute value: {}", safe_abs(-42));
}
```

C-Variadic Functions and VaList

C-Variadic Functions and `VaList`

Thesis

Since **Rust 1.99** (October 2026) you can *define* functions with a C variable argument list — `unsafe extern "C" fn f(fmt: *const c_char, args: ...)` — not just call foreign ones like `printf`. The `...` parameter arrives as a `core::ffi::VaList`, which is layout- and ABI-compatible with the platform's C `va_list`, so you can read arguments yourself or hand the whole list to a C `v*` function such as `vsnprintf`.

That closes a long-standing FFI gap: a Rust library can now implement a C header that contains `void log(int level, const char *fmt, ...)`, sentinel-terminated APIs in the style of `execl`, or plugin callback tables that expect variadic slots — without a C shim file.

The feature is deliberately sharp. A variadic list carries **no count and no types**. Every read is an `unsafe` promise that the caller sent exactly what you are about to take. The job of this chapter is to make that promise small, explicit, and checked by the C compiler wherever possible.

Mental model

```
C caller                        Rust callee (unsafe extern "C" fn f(fixed, ap: ...))

f(2, "x", 3.5f)                 ap: VaList          ← va_start happens for you
      default argument
      promotions:                       ap.next_arg::<*const c_char>()   ← va_arg
       char/short → int                 ap.next_arg::<c_double>()        ← va_arg
       float      → double
                                        ap.clone()                       ← va_copy
registers / stack per ABI
                                         drop(ap)                         ← va_end
```

C	Rust 1.99
`void f(int n, ...)` (definition)	`unsafe extern "C" fn f(n: c_int, mut ap: ...)`
`va_list ap;`	implicit — `ap` *is* the started `VaList`
`va_start(ap, n);`	
`va_arg(ap, int)`	`unsafe { ap.next_arg::<c_int>() }`
`va_copy(dst, src)`	`let dst = src.clone();`
`va_end(ap)`	`Drop` of the `VaList`

C	Rust 1.99
`void vf(int n, va_list ap)`	`unsafe fn vf(n: c_int, ap: VaList<'_>)`

Rules the compiler enforces:

1. **The definition must be `unsafe`.** Callers cannot be checked, so the function is unsafe to call.
2. **ABI must be `"C"` or `"C-unwind"`** for a Rust-bodied definition (other ABIs are only allowed for declarations in `extern` blocks, or naked functions written in assembly).
3. **The `...` parameter needs a name** in a definition (`args: ...`), and it must be last.
4. **Only promotion-safe types can be read.** `next_arg::<T>()` requires the (nightly-only) trait `VaArgSafe`; on stable you just use the concrete types it is implemented for: `c_int`/`c_uint` and wider integers, `c_double`, and raw pointers. `f32`, `i8`, `u16`, and `bool` are rejected because C never passes them through `...` unpromoted.

Rules the compiler **cannot** enforce: how many arguments there are, and what type each one has. Those come from a *protocol* — a count, a format string, or a sentinel — that you document in the `# Safety` section and, ideally, encode in the C header.

Worked examples

All listings were built with `rustc 1.99.0 (b940084d7 2026-09-28)` and GCC 14.2 on x86_64 Linux, edition 2024.

Case 1: count-prefixed list, defined and called in Rust

The simplest protocol: the first fixed argument says how many follow. The second function shows the v-variant split you will want for anything non-trivial — a thin variadic entry point that forwards its `VaList` to an ordinary `unsafe fn`.

```
// sum.rs - define and call a C-variadic function entirely in Rust.
use std::ffi::{c_double, c_int, VaList};

/// # Safety
/// The caller must pass exactly `count` further arguments, each a C `int`.
unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        // SAFETY: the caller promised `count` ints follow.
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}
```

```rust
/// The `v`-variant: takes an already-started list, like C's `vprintf`.
///
/// # Safety
/// `ap` must hold exactly `count` C `double`s.
unsafe fn vmean(count: c_int, mut ap: VaList<'_>) -> c_double {
    let mut acc = 0.0;
    for _ in 0..count {
        acc += unsafe { ap.next_arg::<c_double>() };
    }
    acc / count as c_double
}

/// # Safety
/// The caller must pass exactly `count` further `double`s.
unsafe extern "C" fn mean(count: c_int, args: ...) -> c_double {
    unsafe { vmean(count, args) }
}

fn main() {
    let s = unsafe { sum_ints(3, 10, 20, 12) };
    let m = unsafe { mean(4, 1.0, 2.0, 3.0, 4.5) };
    let small: u8 = 7;
    let k = unsafe { sum_ints(2, small as c_int, -2) }; // explicit promotion
    println!("sum = {s}, mean = {m}, k = {k}");
}
```

```
rustc --edition 2024 sum.rs && ./sum
```

```
sum = 42, mean = 2.625, k = 5
```

Note `small as c_int`: when *Rust* calls a variadic function, it refuses to pass `u8` or `f32` and asks you to promote by hand (see *The trap*).

Case 2: a printf-style logger implemented in Rust, called from C

This is the case the feature exists for. A C program (or a C plugin host) expects `rlog(int level, const char *fmt, ...)`. The Rust implementation never parses the format string itself; it forwards the list to libc's `vsnprintf`, which is the only component that actually understands `%s`, `%.1f`, and `%llu`.

Two details matter:

- `vsnprintf` **consumes** the `va_list` it receives. To measure first and then render, pass `ap.clone()` (C `va_copy`) to the measuring call and the original to the rendering call.
- `VaList` can be passed **by value across FFI** — declare the C parameter as `ap: VaList` in the `extern` block.

```rust
// rlog.rs - printf-style logging and a sentinel-terminated API,
// implemented in Rust and exported with the C ABI.
use std::ffi::{c_char, c_int, CStr, VaList};
use std::ptr;

unsafe extern "C" {
    // int vsnprintf(char *str, size_t size, const char *format, va_list ap);
    fn vsnprintf(buf: *mut c_char, size: usize, fmt: *const c_char, ap: VaList) -> c_int;
}

/// Format `fmt` with `ap` into an owned Rust `String`.
///
/// # Safety
/// `fmt` must be a valid NUL-terminated C format string and `ap` must hold
/// arguments matching it exactly (same rules as C `vprintf`).
unsafe fn vformat(fmt: *const c_char, ap: VaList<'_>) -> Option<String> {
    // Pass 1: measure. vsnprintf consumes the list it is given, so hand it a
    // clone (C: va_copy) and keep the original for pass 2.
    let needed = unsafe { vsnprintf(ptr::null_mut(), 0, fmt, ap.clone()) };
    if needed < 0 {
        return None;
    }
    // Pass 2: render into a buffer with room for the trailing NUL.
    let mut buf = vec![0u8; needed as usize + 1];
    let written = unsafe { vsnprintf(buf.as_mut_ptr().cast(), buf.len(), fmt, ap) };
    if written != needed {
        return None;
    }
    buf.truncate(needed as usize);
    String::from_utf8(buf).ok()
}

/// C signature: `void rlog(int level, const char *fmt, ...);`
///
/// # Safety
/// Same contract as `printf`: the variadic arguments must match `fmt`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rlog(level: c_int, fmt: *const c_char, args: ...) {
    let tag = match level {
        0 => "DEBUG",
        1 => "INFO",
        2 => "WARN",
        _ => "ERROR",
    };
    match unsafe { vformat(fmt, args) } {
        Some(msg) => eprintln!("[{tag}] {msg}"),
        None => {
```

```
            let raw = unsafe { CStr::from_ptr(fmt) };
            eprintln!("[{tag}] <format error: {raw:?}>");
        }
    }
}

/// C signature: `size_t rjoin_len(const char *first, ...);`
/// Sum of strlen() of every string up to a terminating NULL pointer.
///
/// # Safety
/// Every argument must be a `const char *` to a NUL-terminated string,
/// and the list must end with `(char *)NULL`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rjoin_len(first: *const c_char, mut rest: ...) -> usize {
    let mut total = 0;
    let mut cur = first;
    while !cur.is_null() {
        total += unsafe { CStr::from_ptr(cur) }.count_bytes();
        cur = unsafe { rest.next_arg::<*const c_char>() };
    }
    total
}
```

The public C header is where you buy back type checking. `format(printf, 2, 3)` tells GCC and Clang that argument 2 is a printf format and checking starts at argument 3; `sentinel` demands a trailing null pointer.

```
/* rlog.h - public C header for the Rust-implemented logger. */
#ifndef RLOG_H
#define RLOG_H
#include <stddef.h>

/* format(printf, 2, 3): argument 2 is the format, checking starts at 3. */
void rlog(int level, const char *fmt, ...) __attribute__((format(printf, 2, 3)));

/* sentinel: the variadic list must end with a NULL pointer. */
size_t rjoin_len(const char *first, ...) __attribute__((sentinel));

#endif
```

```
/* main.c - C caller for the Rust-implemented variadic functions. */
#include <stdio.h>
#include "rlog.h"

int main(void) {
    rlog(1, "listening on %s:%d", "0.0.0.0", 8443);
```

```
    rlog(2, "disk %s at %.1f%% (%llu bytes free)", "/var", 91.5, 4294967296ULL);
    rlog(3, "%s", "plain string, no surprises");

    size_t n = rjoin_len("usr", "local", "bin", (char *)NULL);
    printf("rjoin_len = %zu\n", n);
    return 0;
}
```

Build the Rust side as a static library, then link it into the C program with the C compiler driver:

```
rustc --edition 2024 -O --crate-type=staticlib rlog.rs -o librlog.a
cc -std=c17 -Wall -Wextra main.c -L. -lrlog -o demo
./demo
```

```
[INFO] listening on 0.0.0.0:8443
[WARN] disk /var at 91.5% (4294967296 bytes free)
[ERROR] plain string, no surprises
rjoin_len = 11
```

If the link fails with undefined symbols from libc, pthreads, or libdl, ask `rustc` what the static library needs and append it to the `cc` line:

```
rustc --edition 2024 --crate-type=staticlib rlog.rs -o librlog.a --print native-static-libs
```

```
note: native-static-libs: -lgcc_s -lutil -lrt -lpthread -lm -ldl -lc
```

In a Cargo project the equivalent is `[lib] crate-type = ["staticlib"]` (or `"cdylib"`) in `Cargo.toml`; nothing else about the code changes.

Case 3: the header catches caller mistakes before they become UB

The Rust callee cannot detect a wrong argument type — it will happily read 4 bytes of an 8-byte `long long`. The defence lives at the call site, in the C compiler, driven by the attributes in `rlog.h`:

```
/* bad.c - mistakes the header attributes catch at compile time. */
#include "rlog.h"

int main(void) {
    long long bytes = 4294967296LL;
    rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
    return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
}
```

```
cc -std=c17 -Wall -Wextra -c bad.c -o /dev/null
```

```
bad.c: In function ‘main’:
bad.c:6:21: warning: format ‘%d’ expects argument of type ‘int’, but argument 3 has type ‘lo
    6 |     rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
      |                     ~^         ~~~~~
      |                      |         |
      |                      int       long long int
      |                     %lld
bad.c:7:5: warning: missing sentinel in function call [-Wformat=]
    7 |     return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
      |     ^~~~~~
```

Clang reports the same two problems (`-Wformat`, `-Wsentinel`) and even suggests the fix (`, NULL`). In CI, build C callers with `-Werror=format` (GCC files the missing-sentinel warning under `-Wformat`; Clang under `-Wsentinel`, so add `-Werror=sentinel` there) so these never ship.

Case 4: what the compiler refuses

Three mistakes are rejected at compile time on Rust 1.99. Seeing the errors once makes the rules stick.

A definition that is not `unsafe`:

```
// not_unsafe.rs
use std::ffi::c_int;
extern "C" fn count_args(_n: c_int, _args: ...) {}
fn main() {}
```

```
rustc --edition 2024 not_unsafe.rs
```

```
error: functions with a C variable argument list must be unsafe
 --> not_unsafe.rs:3:37
  |
3 | extern "C" fn count_args(_n: c_int, _args: ...) {}
  |                                     ^^^^^^^^^^
  |
help: add the `unsafe` keyword to this definition
  |
3 | unsafe extern "C" fn count_args(_n: c_int, _args: ...) {}
  | ++++++

error: aborting due to 1 previous error
```

Reading a type that C always promotes:

```
// read_f32.rs
unsafe extern "C" fn first_float(mut args: ...) -> f32 {
```

```
    unsafe { args.next_arg::<f32>() }
}
fn main() {}
```

```
rustc --edition 2024 read_f32.rs
```

```
error[E0277]: the trait bound `f32: VaArgSafe` is not satisfied
 --> read_f32.rs:3:30
  |
3 |     unsafe { args.next_arg::<f32>() }
  |                   --------   ^^^ the nightly-only, unstable trait `VaArgSafe` is not imp
  |                   |
  |                   required by a bound introduced by this call
  |
  = help: the following other types implement trait `VaArgSafe`:
            f64
            i128
            i32
            i64
            isize
            u128
            u32
            u64
            usize
note: required by a bound in `VaList::<'f>::next_arg`
```

(`i16`, `u8`, and `bool` fail the same way. Read `c_int` / `c_uint` / `c_double` and narrow afterwards with a checked conversion.)

Passing an unpromoted type *from Rust*:

```
// promote.rs
use std::ffi::c_int;

unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}

fn main() {
    let small: u8 = 7;
    let ratio: f32 = 0.5;
    let s = unsafe { sum_ints(2, small, ratio) };
```

```
    println!("{s}");
}
```

```
rustc --edition 2024 promote.rs
```

```
error[E0617]: can't pass `u8` to variadic function
  --> promote.rs:15:34
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                  ^^^^^
   |
help: cast the value to `c_uint`
   |
15 |     let s = unsafe { sum_ints(2, small as c_uint, ratio) };
   |                                         +++++++++

error[E0617]: can't pass `f32` to variadic function
  --> promote.rs:15:41
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                          ^^^^^
   |
help: cast the value to `c_double`
   |
15 |     let s = unsafe { sum_ints(2, small, ratio as c_double) };
   |                                                ++++++++++++

error: aborting due to 2 previous errors
```

Notice the second error also exposes a *logic* bug the compiler cannot see: `sum_ints` reads `c_int`s, but the caller is now about to send a `c_double`. Casting to silence E0617 is correct only if the cast type matches what the callee reads.

The trap

Treating `next_arg` as if it were type-checked. It is a raw read guided only by your protocol. Every one of these is undefined behaviour, and none produces a compile error in the callee:

- **Reading past the end.** There is no length; `next_arg` on an exhausted list reads whatever is in the next register or stack slot. A missing sentinel in Case 2 walks off into garbage pointers.
- **Reading the wrong type.** Caller passes `long long`, you read `c_int`; caller passes `double`, you read a pointer. The `VaList` docs allow only *compatible* types: identical types, same-size integers whose value fits both, compatible pointers, or `*const c_void` ↔ `*const u8`/`i8`.
- **Trusting integer widths across platforms.** `c_long` is 64-bit on Linux x86_64 and 32-bit on Windows x86_64. Read `c_long` when the C side passes `long`, not `i64`. The `VaArgSafe` docs say implementations for `i32`/`usize` "shouldn't be relied upon directly"; spell the C type.

- **Re-reading a consumed list.** After `vsnprintf(…, ap)` the list is spent; use `ap.clone()` *before* the first consumer if you need two passes.
- **Panicking out of an `extern "C"` function.** A panic that reaches an `extern "C"` boundary aborts the process. If unwinding through C is genuinely intended, use `"C-unwind"`; otherwise catch and convert errors to return codes, as `rlog` does with its `None` branch.
- **Calling your own variadic from Rust.** Rust callers get none of the C header's format checking. Give Rust code a normal, safe function (`fn log(level: Level, msg: &str)`) and keep the variadic entry point for C.

The boring rule

> A C-variadic function in Rust is a thin, `unsafe`, `#[unsafe(no_mangle)]` door for C callers. It reads arguments only under a documented protocol — a count, a format string handed to libc's `v*` functions, or a NULL sentinel — reads only C-spelled promoted types (`c_int`, `c_uint`, `c_long`, `c_double`, pointers), clones before any second pass, never unwinds through `extern "C"`, and ships with a C header carrying `format(printf, …)` or `sentinel` so the *caller's* compiler checks what the callee cannot.

Checklist before merging:

1. `# Safety` section states the exact protocol (how many, which types, how the list ends).
2. Real formatting is delegated to `vsnprintf`/`vfprintf`; you do not reimplement `%` parsing.
3. Every read type is a `core::ffi` C type or a raw pointer.
4. The C header has `__attribute__((format(printf, N, M)))` or `__attribute__((sentinel))`, and CI builds C callers with `-Werror=format`.
5. Rust-side callers use a safe non-variadic API instead.

Try this

1. Add `rlog_count(void)` to `rlog.rs` that returns how many messages have been logged (an `AtomicU32`), call it from `main.c`, and confirm it prints `3`.
2. Remove `(char *)NULL` from the `rjoin_len` call in `main.c`, rebuild with `-Wall`, and read the warning. Then delete `__attribute__((sentinel))` from the header and observe that the warning disappears while the bug remains — that is why the attribute is part of the API.
3. Write `vrlog(int level, const char *fmt, va_list ap)` in Rust taking `ap: VaList<'_>`, make `rlog` a one-line forwarder to it, and export both. Call `vrlog` from a C wrapper function that does its own `va_start`/`va_end`.
4. Change `vformat` to pass `ap` (not `ap.clone()`) to the measuring `vsnprintf` call. Explain why the second call is now undefined behaviour even if it appears to work on your machine.
5. Run `rustc --print native-static-libs` for your target and wire the result into a `Makefile` rule so the link line is never guessed.

Sources

- Announcing Rust 1.99.0 — extern "C" variadics (Rust Blog, 2026-10-01)
- `core::ffi::VaList` and `VaArgSafe` — std/core API docs
- The Rust Reference — Functions: C-variadic parameters and External blocks: variadic functions
- GCC manual, *Common Function Attributes* (`format`, `sentinel`)

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1)

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1)

Rust's `bool` is not a C `int`. Across the `extern "C"` boundary it must follow the platform ABI: on **x86-64 SysV**, a `_Bool` / `bool` return is a byte whose **bits 1–7 are zero** (bits above that are undefined). If the compiler forgets to zero-extend, a C caller that reads the return as `bool` can observe values like `16` instead of `0` or `1`—and then every `if (flag)` in C becomes a landmine.

Rust **1.99.0** shipped a regression that dropped that zero-extension for FFI `bool` returns on several x86_64 targets. **1.99.1** (2026-10-15) restored it. This chapter is the incident report you want next to the FFI chapter: what broke, how to see it in a register dump, who is affected, and what to do on a pinned toolchain.

Learning goals

By the end of this chapter you can:

- State the x86-64 ABI rule for returning `bool` / `_Bool`
- Reproduce the 1.99.0 miscompile with a tiny Rust cdylib + C main
- Recognize the missing `andb $0x1, %al` (or equivalent zero-extend) in disassembly
- Upgrade to **1.99.1** (or pin 1.98.x / avoid `bool` at the boundary) as mitigation

Thesis

> Safe Rust returning `bool` to C is only as trustworthy as codegen's ABI lowering. A missing zero-extend is a silent miscompile: the source looks right, the bit in `%al` does not.

Mental model

```
Rust: (flags & 1) == 1     →  i1 in LLVM


ABI: return bool in %al with bits 1-7 cleared

   1.98.0:  mov %edi,%eax ; and $1,%al ; ret
   1.99.0:  mov %edi,%eax ; ret                    (raw 0x10 survives)
```

```
      1.99.1:  zero-extend restored
```

Layer	What went wrong in 1.99.0
LLVM IR	`zeroext` on `i1` returns was dropped for x86_64 SysV (related fallout from broader scalar-extension changes)
Machine code	No mask/extend before `ret`
C caller	`(int)ret` can print `16` when input was `0x10`

Primary tracker: rust-lang/rust#163911. Fix release notes: Announcing Rust 1.99.1.

Case 1 — Minimal reproduction

Save the Rust side:

```rust
// lib.rs - cdylib exporting a C ABI bool
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rust_fn_returning_bool(flags: u8) -> bool {
    (flags & 0x1) == 0x1
}
```

Save the C caller:

```c
/* main.c */
#include <stdio.h>
#include <stdbool.h>

extern bool rust_fn_returning_bool(unsigned char);

int main(void) {
    bool ret = rust_fn_returning_bool(0x10);
    printf("ret = %d\n", (int)ret);
    return 0;
}
```

Build and run (Linux x86_64):

```
rustc --crate-type cdylib -O -o librust_bool.so lib.rs
gcc -O2 -o ffi main.c -L. -lrust_bool -Wl,-rpath,'$ORIGIN'
./ffi
```

Genuine output on this box (rustc 1.99.0)

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)

$ ./ffi
ret = 16
```

Expected on a fixed toolchain: `ret = 0` (because `0x10 & 1 == 0`).

Disassembly of the export on 1.99.0 (excerpt):

```
00000000000127cc <rust_fn_returning_bool>:
   127cc:   89 f8                   mov    %edi,%eax
   127ce:   c3                      ret
```

On 1.98.0 the same function included `andb $0x1, %al` before `ret` (see the issue). The logical compare was optimized assuming LLVM would still zero-extend the `i1` at the ABI boundary—and it no longer did.

Case 2 — Who was affected

Per the issue and write-ups circulating with 1.99.1:

- **Targets:** x86_64 SysV-style environments (Linux, musl, FreeBSD, Android, many bare-metal x86_64 configs). aarch64 / loongarch64 reproductions in the report masked correctly.
- **Surface:** Rust functions with `extern "C"` (or other C-like ABIs) that **return** `bool`, especially when compiled with optimizations so LLVM folds the compare into a raw register move.
- **Not required:** `unsafe` in the callee body beyond FFI—the export itself is the hazard. Callers in C/`bindgen` glue are where symptoms show up (wrong branch, wrong `printf`, corrupted flag packing).

If you ship a cdylib / staticlib consumed by C/C++/other languages, inventory `-> bool` at the boundary.

Case 3 — Mitigations

1. **Upgrade:** `rustup update stable` → **1.99.1** or newer, then **rebuild** all Rust artifacts that cross FFI (don't assume an old `.so` is fine).
2. **Temporary pin:** Stay on **1.98.x** until you can move, if 1.99.0 is already in CI.
3. **ABI hygiene:** Return `u8` (0/1) or `c_int` at the boundary and convert explicitly:

```rust
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rust_fn_returning_u8(flags: u8) -> u8 {
    u8::from((flags & 0x1) == 0x1)
}
```

4. **Test in CI:** Keep a C (or `cc` crate) smoke test that passes a non-0/1 pattern in the low byte's upper bits and asserts a clean 0/1 out—exactly the repro above.

The trap

- Assuming "it compiled and linked" means ABI correctness—miscompiles return successfully.
- Only testing `true`/`false` inputs that already clear high bits (`0` and `1`), which hide the bug.
- Upgrading `rustc` in the developer laptop but shipping libraries built with 1.99.0 in a Docker layer.

The boring rule

Treat `bool` across FFI as a **checked ABI contract**. Prefer explicit integer statuses at the boundary, pin a toolchain with a known-good 1.99.1+, and keep one C-linked regression test that would have printed `16`.

Try this

1. Run the repro on your machine; note `rustc --version` and `./ffi` output.
2. `rustup update stable` to 1.99.1+, rebuild, confirm `ret = 0`.
3. `objdump -d` the export before/after and look for the zero-extend.

Sources

- Announcing Rust 1.99.1 — official fix description
- rust-lang/rust#163911 — miscompilation report, IR/`zeroext` analysis, P-critical
- yeslogic/rust-bool-ffi — upstream minimal reproduction
- x86-64 psABI notes on `_Bool` return zeroing (discussed in the issue thread / LLVM history)

Inspiration

- Rapid response around Rust 1.99.0 FFI bool returns (issue filed 2026-10-07, point release 2026-10-15). Chapter cites the Rust blog and GitHub issue.

Verification

Genuine on this box with **rustc 1.99.0** + **gcc**: `./ffi` printed `ret = 16`; `objdump` showed `mov %edi,%eax; ret` without `and`. 1.99.1 was not installed here (scout runs do not upgrade toolchains); fixed behavior is taken from the official 1.99.1 announcement and the issue resolution.

Safe Rust UAF via covariant field assignment (rust#164250)

Safe Rust UAF via covariant field assignment (rust#164250)

Rust's type system is supposed to make use-after-free impossible in safe code. Issue rust-lang/rust#164250 (opened 2026-10-13, labels `I-unsound`, `P-high`, `A-variance`, `A-borrow-checker`) shows a counterexample: **no `unsafe` keyword**, yet a `Drop` impl can observe a dangling reference. A minimized form has compiled since **Rust 1.4** (2015). This chapter is the incident card next to the unsafe / FFI chapters: what breaks, how to reproduce on current stable, and what to audit in your own crates — not a claim that everyday application code is doomed.

Learning goals

By the end of this chapter you can:

- State the bug in one sentence: lifetime bounds on a type are not fully enforced when a **covariant field** is assigned after construction
- Reproduce the `MyType` / `Drop` smuggling pattern on **rustc 1.99.0**
- Separate "malicious `Drop`" exploitability from accidental UAF in ordinary code (per the Types Team note on the issue)
- Know where to watch for fixes (Types Team / Project Zero work; do not "patch" this in user code with `unsafe`)

Thesis

> If the compiler accepts a shorter-lived reference into a field whose type promised a longer lifetime, `Drop` runs with a lie. Safe code can then read freed stack memory — the soundness bug is in the type checker, not in your `unsafe` blocks.

Mental model

```
struct MyType<'a, 'b: 'a, 'c>(&'c mut &'a str, &'b str);

                                      field 1: covariant in 'b
       Drop assumes 'b: 'a (and related bounds) still hold

construction:  MyType(&mut me_out, "")     // '' is 'static-ish / long enough
later assign:  q.1 = &local_string;        // shorter 'b sneaks in
```

```
scope ends:    Drop runs → *me_out = dangling
after scope:   print(me_out) → UAF
```

Piece	Role
Covariance on a field	Allows treating a shorter lifetime as a longer one when unifying / assigning
Lifetime bounds on the struct (`'b: 'a`, `'a: 'static`, …)	Documented intent; **not** re-checked thoroughly on field assignment in this bug
`Drop`	Runs while the type system still believes the long lifetime
No `unsafe` in the repro	The unsoundness is accepting the program, not an explicit transmute

The Types Team note on the issue: this would be `P-critical` on novelty grounds, but an 11-year-old hole that stayed unnoticed is hard to hit by accident; exploiting it generally needs an **intentionally malicious `Drop`**. Treat production risk as real but not "rewrite everything tomorrow".

Worked examples

Case 1: Minimal Drop smuggling (reproduced here)

Verified on this box: **rustc 1.99.0** (`b940084d7 2026-09-28`). The program compiles; after the inner scope, `me_out` prints garbage (use-after-free of the `String` buffer).

```rust
// uaf_covariant_field.rs - safe Rust; reproduces rust#164250 shape (lcnr / theemathas)
// rustc 1.99.0: compiles and runs; output is nondeterministic garbage / asan would trap

struct MyType<'a: 'c, 'b: 'a, 'c>(&'c mut &'a str, &'b str);

impl<'a: 'c, 'b: 'a, 'c> Drop for MyType<'a, 'b, 'c> {
    fn drop(&mut self) {
        // Type system thinks self.1 lives at least as long as 'a
        *self.0 = self.1;
    }
}

fn main() {
    let mut me_out = "rarw";
    {
        let l = String::from("some cool string");
        let mut q = MyType(&mut me_out, "");
        q.1 = &l; // assigns a short-lived &str into a field that required 'b: 'a
    } // l dropped; Drop of q already wrote &l into me_out
    println!("me_out: {}", me_out);
```

```
}
```

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)
$ rustc uaf_covariant_field.rs -o uaf_covariant_field
$ ./uaf_covariant_field
me_out: <non-deterministic garbage bytes>
```

(Exact bytes vary; AddressSanitizer builds crash with `stack-use-after-scope` — see the Godbolt links on the issue.)

Case 2: `'static` bound bypass with a kept value

The original report uses a `Mutex<Vec<Box<dyn Any + Send>>>` to keep a `T: 'static` value that was actually a short reference. On 1.99.0 here it compiled and printed the **later** stack string that reused the slot:

```
// uaf_kept_static.rs - original-report shape; rustc 1.99.0 prints the filler String
use std::sync::Mutex;

static KEPT: Mutex<Vec<Box<dyn std::any::Any + Send>>> = Mutex::new(Vec::new());

struct Q<T: 'static + Send>(Option<T>);

impl<T: 'static + Send> Drop for Q<T> {
    fn drop(&mut self) {
        if let Some(t) = self.0.take() {
            KEPT.lock().unwrap().push(Box::new(t));
        }
    }
}

fn main() {
    {
        let l = String::from("hello world, this is long enough");
        let mut q = Q(None);
        q.0 = Some(&l); // T inferred as &'static String, but &l is not 'static
    }
    let _z = String::from("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX");
    let k = KEPT.lock().unwrap();
    println!("{}", k[0].downcast_ref::<&'static String>().unwrap());
}
```

```
$ rustc uaf_kept_static.rs -o uaf_kept_static && ./uaf_kept_static
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

That output is the smoking gun: the kept `&String` no longer points at `"hello world…"`.

Case 3: What to check in your code

You are looking for **custom `Drop` (or drop glue that runs user code)** combined with **post-init assignment into lifetime-carrying fields**, especially where the type constructor / bounds promised `'static` or a long lifetime. Ordinary `&T` fields without clever `Drop` are the common case and are not automatically vulnerable. Do not "fix" this by sprinkling `unsafe` — wait for a compiler fix; track #164250.

Miri (when available on nightly) reports UB on related reproducers (dangling reference in a spawned closure). This box did not run Miri; the stable `rustc` runs above are enough to show acceptance + observable UAF.

The trap

Hearing "safe Rust UAF" and concluding the borrow checker is worthless, or conversely dismissing #164250 because it needs a malicious `Drop`. The accurate posture: the invariant *failed* in a narrow variance/bounds corner; the ecosystem still relies on that invariant; `P-high` means the Types Team treats it as urgent work.

The boring rule

If you implement `Drop` and store references (or callbacks over references), construct the value with the **final** references in place when you can, and treat "assign a shorter borrow into a field that was typed longer" as a compiler bug to report — not a pattern to build on. Pin a toolchain; upgrade when the fix lands on stable.

Try this

```
rustc --version   # expect 1.99.x or newer until fixed
rustc uaf_covariant_field.rs -o /tmp/uaf && /tmp/uaf
# Optional: rustup + nightly miri on your machine (not installed here)
```

Watch rust#164250 for the fix PR; re-run the repro after upgrading — it should start failing to compile.

Sources

- rust-lang/rust#164250 — struct's lifetime requirements can be bypassed via covariant field assignment
- Types Team context note on the issue (`P-high`, malicious `Drop`, Project Zero direction)
- Godbolt links attached to the issue (with/without ASan)

Summary

Summary

Chapter Key Takeaways

- Declarative macros (`macro_rules!`) perform hygienic pattern-based code generation.
- Procedural macros transform token streams at compile time.
- `unsafe` isolates low-level pointer manipulation and FFI behind safe API boundaries.
- `#[repr(C)]` guarantees binary compatibility with standard C ABIs.

Testing, Documentation, Performance, and Deployment

Unit Tests, Integration Tests, and Doctests

Unit Tests, Integration Tests, and Doctests

Rust has first-class built-in testing support.

Unit Tests

Unit tests live in the same source files as the code they test inside a `#[cfg(test)]` module:

```rust
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }
}
```

Integration Tests

Integration tests live in the top-level `tests/` directory and test the public API as an external consumer.

Documentation Tests (Doctests)

Examples written in markdown doc-comments (`///`) are compiled and executed during `cargo test`.

Benchmarking with Criterion

Benchmarking with Criterion

Criterion provides robust statistical benchmarking with outlier detection and regression analysis:

```
[dev-dependencies]
criterion = "0.5"

[[bench]]
name = "compute_benchmark"
harness = false
```

Writing Great Documentation

Writing Great Documentation

Write documentation comments using triple slashes `///` supporting Markdown:

```
/// Calculates the checksum for a given network payload buffer.
///
/// # Examples
///
/// ```
/// let data = [1, 2, 3];
/// let sum: u32 = data.iter().map(|&x| x as u32).sum();
/// assert_eq!(sum, 6);
/// ```
pub fn calculate_checksum(data: &[u8]) -> u32 {
    data.iter().map(|&x| x as u32).sum()
}
```

Generate and open documentation:

```
cargo doc --open
```

Profiling and Optimization Techniques

Profiling and Optimization Techniques

Techniques for production profiling: - Use `perf` or `samply` on Linux to capture CPU flamegraphs. - Configure Link-Time Optimization (LTO) and codegen units in `Cargo.toml`:

```
[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true
```

Building and Deploying Rust Applications

Building and Deploying Rust Applications

Rust binaries can be statically compiled with musl libc for minimal, scratch Docker containers:

```
rustup target add x86_64-unknown-linux-musl
cargo build --release --target x86_64-unknown-linux-musl
```

MSRV in practice: rust-version and the MSRV-aware resolver

MSRV in practice: `rust-version` and the MSRV-aware resolver

Your crate's **minimum supported Rust version** (MSRV) is not just a property of your own code. It is a property of the whole dependency graph that `Cargo.lock` pins. One `cargo update` can pull in a transitive dependency, two or three levels down, that needs a newer compiler than you promised. Your source is unchanged, and the build breaks for everyone on the older toolchain. This chapter shows that break with real crates, then fixes it with the tools Cargo has shipped since 1.84: `package.rust-version`, resolver `"3"` (the MSRV-aware resolver, default in edition 2024), `cargo add`'s MSRV awareness, `--ignore-rust-version`, and `--locked`.

Learning goals

By the end of this chapter you can:

- Explain why a dependency with an old MSRV can still break your MSRV build
- Read Cargo's `requires Rust 1.xx` annotations and its "rustc … is not supported" error
- Switch a project to the MSRV-aware resolver and see what it downgrades and why
- Tell when the resolver *cannot* help (your own version requirement is too new)
- Make CI check the MSRV with the committed lockfile

Thesis

> `rust-version` is a promise about the build graph, not just your crate. Since Cargo 1.84 the resolver can keep that promise for you. With resolver `"3"` it prefers dependency versions whose own `rust-version` fits yours. It only does that if you opt in (edition 2024, `resolver = "3"`, or config), if dependencies declare `rust-version`, and if your version requirements leave it room.

Mental model

```
Cargo.toml requirement      →   resolver picks a version    →   Cargo.lock    →   rustc ch
url = "2.5"                     (newest that matches;           pins it           package'
                                 resolver 3: newest that                           at build
                                 ALSO fits your rust-version)
```

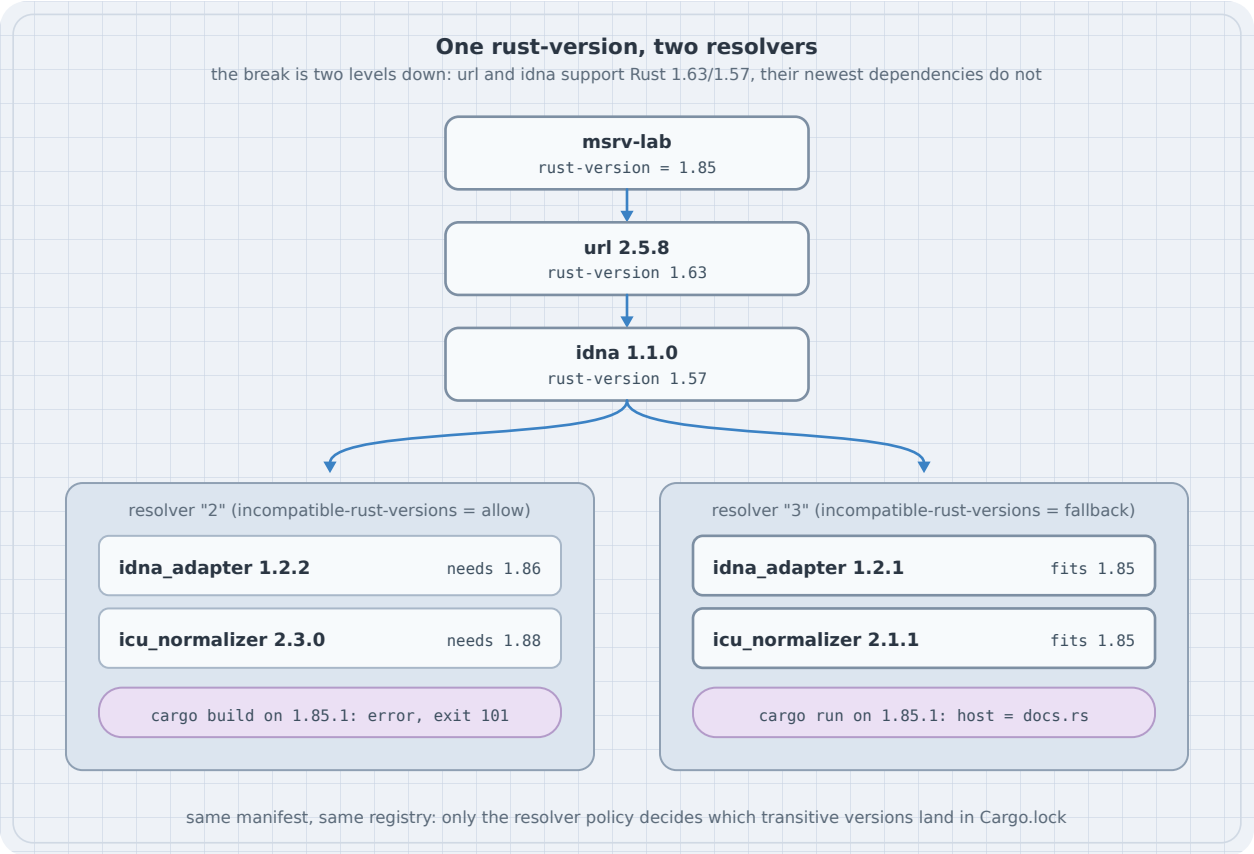


Figure 1: One package with rust-version 1.85; resolver 2 picks transitive versions needing Rust 1.86/1.88 and fails, resolver 3 picks older compatible ones and builds

Knob	Where	Effect
`package.rust-version = "1.85"`	`Cargo.toml`	Your MSRV. Cargo refuses to build on an older rustc. `cargo add` and resolver `"3"` use it to choose versions
`resolver = "3"`	`[package]` or `[workspace]`	Default for **edition 2024**. Sets `incompatible-rust-versions = "fallback"`
`resolver.incompatible-rust-versions`	`.cargo/config.toml` / `CARGO_RESOLVER_INCOMPATIBLE_RUST_VERSIONS`	`allow` (old behaviour) or `fallback` (prefer compatible, use incompatible only if nothing else matches). Respected since Cargo 1.84
`--ignore-rust-version`	CLI	Build or resolve as if no `rust-version` were set
`--locked`	CLI	Fail instead of changing `Cargo.lock`

"Your Rust version" for the resolver is the workspace's `rust-version`. If that is unset, Cargo uses the **toolchain you are running** (Case 5).

Worked examples

Run on Debian 13 x86_64 with the distribution toolchain **rustc 1.85.1 / cargo 1.85.1**. This box has no rustup toolchain installed and none was installed for this chapter. The current stable release is **Rust 1.99** (Cargo 1.99, 2026-10-01). Running an older compiler is the situation this chapter is about. One difference on a current Cargo, from the Cargo changelog: Cargo 1.92 reworded the rust-version incompatibility error, so your message text will differ. The behaviour shown here has been stable since resolver v3 was stabilised in Cargo 1.84. Crate versions are whatever crates.io served on 2026-10-17. Rerunning later may pick newer versions, but the mechanism is the same.

Run the labs in order. They share `/tmp/msrv-lab`. Each one is a bash script you can paste.

Case 1 — The break is two levels down

`url` declares `rust-version = "1.63"`. `idna` declares 1.57. Neither is the problem.

```
#!/usr/bin/env bash
# lab-01-break.sh - a direct dep with an old MSRV, a transitive dep with a new one
set -uo pipefail
rm -rf /tmp/msrv-lab && mkdir -p /tmp/msrv-lab/src && cd /tmp/msrv-lab
cat > Cargo.toml <<'TOML'
[package]
name = "msrv-lab"
```

```
version = "0.1.0"
edition = "2021"          # resolver "2": rust-version of deps is ignored
rust-version = "1.85"

[dependencies]
url = "2.5"               # url itself declares rust-version 1.63
TOML
cat > src/main.rs <<'RS'
// src/main.rs - parse one URL so the dependency is really used
fn main() {
    let u = url::Url::parse("https://docs.rs/url/latest/url/").expect("valid URL");
    println!("host = {}", u.host_str().unwrap_or("?"));
}
RS
cargo --version
cargo generate-lockfile 2>&1
cargo fetch -q              # download now so later output is only about resolution
echo '--- who pulls in the ICU crates?'
cargo tree -i icu_normalizer --edges normal 2>&1
echo '--- build on this toolchain'
cargo build -q 2>&1 | head -20
echo "[cargo build exit ${PIPESTATUS[0]}]"
```

```
cargo 1.85.1 (d73d2caf9 2024-12-31)
    Updating crates.io index
     Locking 35 packages to latest compatible versions
      Adding icu_collections v2.3.0 (requires Rust 1.88)
      Adding icu_locale_core v2.3.0 (requires Rust 1.88)
      Adding icu_normalizer v2.3.0 (requires Rust 1.88)
      Adding icu_normalizer_data v2.3.0 (requires Rust 1.88)
      Adding icu_properties v2.3.0 (requires Rust 1.88)
      Adding icu_properties_data v2.3.0 (requires Rust 1.88)
      Adding icu_provider v2.3.1 (requires Rust 1.88)
      Adding idna_adapter v1.2.2 (requires Rust 1.86)
--- who pulls in the ICU crates?
icu_normalizer v2.3.0
  idna_adapter v1.2.2
      idna v1.1.0
          url v2.5.8
              msrv-lab v0.1.0 (/tmp/msrv-lab)
--- build on this toolchain
error: rustc 1.85.1 is not supported by the following packages:
  icu_collections@2.3.0 requires rustc 1.88
  icu_locale_core@2.3.0 requires rustc 1.88
  icu_normalizer@2.3.0 requires rustc 1.88
  icu_normalizer_data@2.3.0 requires rustc 1.88
```

```
  icu_normalizer_data@2.3.0 requires rustc 1.88
  icu_normalizer_data@2.3.0 requires rustc 1.88
  icu_properties@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_provider@2.3.1 requires rustc 1.88
  idna_adapter@1.2.2 requires rustc 1.86
Either upgrade rustc or select compatible dependency versions with
`cargo update <name>@<current-ver> --precise <compatible-ver>`
where `<compatible-ver>` is the latest version supporting rustc 1.85.1

[cargo build exit 101]
```

Even with resolver `"2"` (the edition 2021 default), Cargo 1.85 *annotates* each pick with `(requires Rust 1.88)`, but it still locks those versions. The build then fails before compiling anything: rustc checks every package's `rust-version` against itself. `cargo tree -i` shows the path. `icu_normalizer` comes from `idna_adapter`, which comes from `idna`, which comes from `url`. None of those is in your `Cargo.toml`. This is the "one transitive update breaks the build" failure, and it can happen to a crate whose own code has not changed in years.

Case 2 — Let the resolver keep the promise

```
#!/usr/bin/env bash
# lab-02-fallback.sh - same manifest, MSRV-aware resolver (resolver "3")
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^rust-version = "1.85"/rust-version = "1.85"\nresolver = "3"            # default
grep -E '^(edition|rust-version|resolver)' Cargo.toml
echo '--- re-resolve against rust-version 1.85'
cargo update 2>&1
echo '--- build and run'
cargo run -q 2>&1
echo "[cargo run exit $?]"
```

```
edition = "2021"          # resolver "2": rust-version of deps is ignored
rust-version = "1.85"
resolver = "3"             # default for edition 2024; needs Cargo 1.84+
--- re-resolve against rust-version 1.85
    Updating crates.io index
     Locking 8 packages to latest Rust 1.85 compatible versions
 Downgrading icu_collections v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_locale_core v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_normalizer v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
```

```
 Downgrading icu_normalizer_data v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_properties v2.3.0 -> v2.1.2 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_properties_data v2.3.0 -> v2.1.2 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_provider v2.3.1 -> v2.1.1 (available: v2.3.1, requires Rust 1.88)
 Downgrading idna_adapter v1.2.2 -> v1.2.1 (available: v1.2.2, requires Rust 1.86)
--- build and run
host = docs.rs
[cargo run exit 0]
```

The same manifest and the same registry give a different lockfile. Each `Downgrading … (available: v2.3.0, requires Rust 1.88)` line is the resolver telling you what it held back and why. You do not have to add pins or `=` requirements by hand, and when you later raise `rust-version`, a plain `cargo update` brings the newer versions back. Here `resolver = "3"` is set explicitly while keeping edition 2021. Moving to `edition = "2024"` gets the same default.

Case 3 — `cargo add` is MSRV-aware; your requirement wins over the resolver

```
#!/usr/bin/env bash
# lab-03-direct.sh - cargo add is MSRV-aware; a hand-written requirement can still force a b
set -uo pipefail
cd /tmp/msrv-lab
echo '--- cargo add picks the newest uuid that fits rust-version'
cargo add uuid --features v4 2>&1 | grep -vE '^ +[-+] |Features:'   # drop the feature list
grep '^uuid' Cargo.toml
cargo build -q 2>&1 && echo '[build ok]'

echo '--- now ask for a version that only exists for newer Rust'
sed -i 's/^uuid = .*/uuid = { version = "1.28", features = ["v4"] }/' Cargo.toml
cargo update -p uuid 2>&1
cargo build -q 2>&1 | head -4
echo "[cargo build exit ${PIPESTATUS[0]}]"
```

```
--- cargo add picks the newest uuid that fits rust-version
    Updating crates.io index
warning: ignoring uuid@1.28.0 (which requires rustc 1.89.0) to maintain msrv-lab's rust-vers
      Adding uuid v1.26.1 to dependencies
    Updating crates.io index
     Locking 18 packages to latest Rust 1.85 compatible versions
      Adding bumpalo v3.20.3
      Adding cfg-if v1.0.5
      Adding futures-core v0.3.34
      Adding futures-task v0.3.34
      Adding futures-util v0.3.34
```

```
      Adding getrandom v0.4.3
      Adding js-sys v0.3.106
      Adding libc v0.2.191
      Adding once_cell v1.21.4
      Adding pin-project-lite v0.2.17
      Adding r-efi v6.0.0
      Adding rustversion v1.0.23
      Adding slab v0.4.12
      Adding uuid v1.26.1 (available: v1.28.0, requires Rust 1.89.0)
      Adding wasm-bindgen v0.2.129
      Adding wasm-bindgen-macro v0.2.129
      Adding wasm-bindgen-macro-support v0.2.129
      Adding wasm-bindgen-shared v0.2.129
uuid = { version = "1.26.1", features = ["v4"] }
[build ok]
--- now ask for a version that only exists for newer Rust
    Updating crates.io index
     Locking 1 package to latest Rust 1.85 compatible version
    Updating uuid v1.26.1 -> v1.28.0 (requires Rust 1.89.0)
note: pass `--verbose` to see 8 unchanged dependencies behind latest
error: rustc 1.85.1 is not supported by the following package:
  uuid@1.28.0 requires rustc 1.89.0
Either upgrade rustc or select compatible dependency versions with
`cargo update <name>@<current-ver> --precise <compatible-ver>`
[cargo build exit 101]
```

`cargo add` chose `1.26.1` and said why (`ignoring uuid@1.28.0 (which requires rustc 1.89.0)`). Once the manifest says `"1.28"`, no version that satisfies the requirement fits Rust 1.85. The resolver's `fallback` policy is documented to pick an incompatible version instead of failing resolution, so `cargo update` succeeded, and `cargo build` refused. The fix is in `Cargo.toml`: lower the requirement or raise `rust-version`. The resolver cannot fix it.

Case 4 — Your own `rust-version`, the escape hatch, and `--locked`

```
#!/usr/bin/env bash
# lab-04-own-msrv.sh - rust-version also guards your own package
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^uuid = .*/uuid = { version = "1.26", features = ["v4"] }/' Cargo.toml
cargo update -q -p uuid --precise 1.26.1 2>&1
sed -i 's/^rust-version = "1.85"/rust-version = "1.90"/' Cargo.toml
echo '--- claim a newer MSRV than this toolchain'
cargo build -q 2>&1 | head -4
echo "[cargo build exit ${PIPESTATUS[0]}]"
echo '--- the escape hatch (use when you know why)'
```

```
cargo build -q --ignore-rust-version 2>&1 && echo '[build ok with --ignore-rust-version]'
sed -i 's/^rust-version = "1.90"/rust-version = "1.85"/' Cargo.toml

echo '--- CI: the lockfile is the contract'
cargo build -q --locked 2>&1 && echo '[--locked build ok]'
sed -i 's/^url = "2.5"/url = "=2.5.4"/' Cargo.toml
cargo build -q --locked 2>&1 | head -3
echo "[--locked build exit ${PIPESTATUS[0]}]"
```

```
--- claim a newer MSRV than this toolchain
error: rustc 1.85.1 is not supported by the following package:
  msrv-lab@0.1.0 requires rustc 1.90

[cargo build exit 101]
--- the escape hatch (use when you know why)
[build ok with --ignore-rust-version]
--- CI: the lockfile is the contract
[--locked build ok]
error: the lock file /tmp/msrv-lab/Cargo.lock needs to be updated but --locked was passed to
If you want to try to generate the lock file without accessing the network, remove the --loc
[--locked build exit 101]
```

- Claiming `rust-version = "1.90"` makes Cargo reject your *own* package on 1.85.1 with a clear message, before rustc can fail on some syntax error deep in your code. That is what the field is for.
- `--ignore-rust-version` builds anyway. Use it when you know the newer feature is behind a cfg you do not hit, not as a habit.
- `--locked` makes the committed lockfile the contract. When the manifest and the lockfile disagree, CI fails instead of silently resolving a new graph. That is how you stop a CI run from picking up a transitive update that nobody reviewed.

Case 5 — No `rust-version`? Then your toolchain is the MSRV

```
#!/usr/bin/env bash
# lab-05-no-msrv.sh - no rust-version at all: what does resolver "3" compare against?
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^url = "=2.5.4"/url = "2.5"/; /^rust-version = /d' Cargo.toml
grep -E '^(edition|rust-version|resolver)' Cargo.toml
rm -f Cargo.lock
cargo generate-lockfile 2>&1 | grep -E 'Locking|uuid|icu_normalizer '
cargo build -q 2>&1 && echo '[build ok on rustc 1.85.1]'
```

```
edition = "2021"          # resolver "2": rust-version of deps is ignored
resolver = "3"             # default for edition 2024; needs Cargo 1.84+
     Locking 53 packages to latest Rust 1.85.1 compatible versions
      Adding icu_normalizer v2.1.1 (available: v2.3.0, requires Rust 1.88)
      Adding uuid v1.26.1 (available: v1.28.0, requires Rust 1.89.0)
[build ok on rustc 1.85.1]
```

With `rust-version` removed, resolver `"3"` resolved against **"Rust 1.85.1"**, the toolchain running `cargo`. That is convenient on one machine and a trap on a team. A developer on 1.99 and a CI runner on 1.85 generate *different* lockfiles from the same manifest. Declare `rust-version` and the result no longer depends on who ran `cargo update`.

Case 6 — Check the MSRV in CI (configuration, not run here)

This needs a second toolchain (`rustup toolchain install`), which this box does not have, so it was not run. It is the pattern the Cargo book's "Rust version" chapter and its CI guide describe: verify the MSRV with the lockfile you ship.

```yaml
# .github/workflows/msrv.yml - build the committed Cargo.lock on the declared MSRV
name: msrv
on: [push, pull_request]
jobs:
  msrv:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - run: rustup toolchain install 1.85 --profile minimal
      - run: cargo +1.85 check --locked --all-targets
```

Pair it with a normal job on the latest stable, so you also see breakage that a newer compiler would catch. For libraries, remember that downstream users do not use your `Cargo.lock`. Their resolver, their `rust-version` and their toolchain decide what they get. The `rust-version` field on every crate in the graph is what lets their resolver choose well.

The trap

The trap is reading MSRV as "the oldest compiler that compiles *my* code". You test once on the old toolchain, it works, and you ship. Weeks later a routine `cargo update`, or a fresh clone without a lockfile, pulls `icu_normalizer 2.3.0` three levels down, and the MSRV build fails with no change to your code. The two usual fixes make it worse. Pinning the transitive crate with `=` in your own `Cargo.toml` freezes it for every downstream user. Dropping `rust-version` makes Cargo resolve against whichever toolchain happens to run, so the lockfile changes from one machine to the next.

The boring rule

> Declare `package.rust-version`. Use resolver `"3"` (edition 2024 gives it to you). Let `cargo add` choose versions. Commit `Cargo.lock` for applications and build with `--locked` in CI. Keep one CI job on the declared MSRV and one on latest stable. When a dependency needs a newer compiler, decide deliberately: raise `rust-version` (a minor-version bump per the Cargo book) or keep the older dependency. Don't hand-pin transitive crates.

Try this

1. In `/tmp/msrv-lab`, set `resolver = "2"`, run `cargo update`, and confirm the ICU crates jump back to 2.3.0. Then set `CARGO_RESOLVER_INCOMPATIBLE_RUST_VERSIONS=fallback` for one `cargo update` and compare.
2. Raise `rust-version` to `"1.88"`, run `cargo update`, and read which `Downgrading` notes turn into `Updating`.
3. Use `cargo update icu_normalizer@2.1.1 --precise 2.3.0` with resolver `"3"`. What does `--precise` do to the MSRV policy?
4. Add a second workspace member with no `rust-version` that also depends on `url = "2.5"`. Which `idna_adapter` does the shared lockfile get, and why? (Hint: version unification in the resolver chapter.)
5. On a machine with rustup, run the Case 6 workflow steps locally (`cargo +1.85 check --locked --all-targets`) against one of your own crates.

Sources

- The Cargo Book, "Rust version" (`rust-version`, support expectations, policies): https://doc.rust-lang.org/cargo/reference/rust-version.html
- The Cargo Book, "Dependency resolution": resolver versions (`"3"` = edition 2024 default, Rust 1.84+) and the Rust-version section: https://doc.rust-lang.org/cargo/reference/resolver.html
- The Cargo Book, configuration: `resolver.incompatible-rust-versions` (`allow` / `fallback`, MSRV 1.84): https://doc.rust-lang.org/cargo/reference/config.html#resolverincompatible-rust-versions
- The Cargo Book, `cargo add`, `cargo update` (`--precise`), and the `--locked` / `--ignore-rust-version` flags: https://doc.rust-lang.org/cargo/commands/
- Cargo changelog (1.84 stabilises resolver v3; 1.92 "Improve error message for rust-version incompatibility diagnostics"; 1.99 is current, released 2026-10-01): https://doc.rust-lang.org/nightly/cargo/CHANGELOG.html
- Rust 2024 edition guide, "Cargo: Rust-version aware resolver": https://doc.rust-lang.org/edition-guide/rust-2024/cargo-resolver.html
- crates.io API (`rust_version` per published version) for the versions used here: `https://crates.io/api/v1/`

Summary

Summary

Chapter Key Takeaways

- Unit tests, integration tests, and doctests are integrated into `cargo test`.
- Criterion provides statistical benchmarking.
- Triple-slash doc comments generate standard HTML documentation.
- Release profiles with LTO produce compact, high-performance static binaries.

The Rust Ecosystem: Essential Crates for Production Development

serde: Serialization and Deserialization

serde: Serialization and Deserialization

Serde is the framework for serializing and deserializing Rust data structures efficiently and generically.

```
// serde derives Serialize and Deserialize for zero-allocation JSON parsing
// #[derive(Serialize, Deserialize, Debug)]
// struct User { id: u64, username: String }
```

clap: Command-Line Argument Parsing

clap: Command-Line Argument Parsing

Clap provides declarative, type-safe command-line argument parsing via procedural derive macros:

```
// #[derive(Parser)]
// #[command(author, version, about)]
// struct Cli { port: u16 }
```

reqwest: HTTP Client

reqwest: HTTP Client

Reqwest provides ergonomic asynchronous and synchronous HTTP clients with connection pooling, TLS, and JSON decoding.

axum: Web Framework

axum: Web Framework

Axum is a modular, ergonomic web application framework built on top of Tokio, Tower, and Hyper.

actix-web: High-Performance Web Framework

actix-web: High-Performance Web Framework

Actix-web is a high-performance, actor-inspired web framework frequently ranking among the fastest web servers in industry benchmarks.

Project 2: Async HTTP API with axum, sqlx, and tracing

Project 2: Async HTTP API with axum, sqlx, and tracing

In Project 2, we build an asynchronous microservice handling user registrations and order audits backed by SQLite/PostgreSQL with structured logging:

```
// Architecture of the User Service API
// Router binds routes: POST /users, GET /users/:id
// Handlers extract typed JSON, execute SQL queries via sqlx, and emit tracing spans
```

Testing the User API

Testing the User API

End-to-end integration tests using Tower `ServiceExt::oneshot` to simulate HTTP requests without binding network sockets.

Benchmarking the User API

Benchmarking the User API

Load-testing the async API with `oha` or `wrk` measuring request latencies under sustained concurrent connections.

Deploying the User API

Deploying the User API

Containerizing the API into a multi-stage Docker build producing a single 15MB container running as an unprivileged user.

sqlx: Async Database Access

sqlx: Async Database Access

SQLx provides compile-time verified SQL queries without an ORM, checking query syntax against live database schemas during `cargo build`.

rayon: Data Parallelism

rayon: Data Parallelism

Rayon converts sequential iterator chains into parallel work-stealing pipelines with a simple `.par_iter()` drop-in replacement.

tracing: Structured Logging and Observability

tracing: Structured Logging and Observability

Tracing instruments Rust applications with contextual, structured events and spans compatible with OpenTelemetry and Jaeger.

chrono: Date and Time

chrono: Date and Time

Chrono handles ISO 8601 timestamps, UTC conversions, duration arithmetic, and timezone formatting.

uuid: Universally Unique Identifiers

uuid: Universally Unique Identifiers

Fast, cryptographically secure generation and parsing of UUID v4 and v7 identifiers.

rand: Random Number Generation

rand: Random Number Generation

Generates cryptographically secure pseudorandom numbers and provides distribution sampling.

regex: Regular Expressions

regex: Regular Expressions

Linear-time regular expression matching guaranteeing no catastrophic backtracking.

bytes: Efficient Binary Buffers

bytes: Efficient Binary Buffers

`Bytes` and `BytesMut` provide reference-counted zero-copy slicing for network packet processing.

futures: Async Primitives

futures: Async Primitives

Provides channels, select combinators, and join mechanisms for working with asynchronous streams.

Summary

Summary

Chapter Key Takeaways

- Serde, Clap, Reqwest, Axum, and SQLx form the production backbone for web backends and CLIs.
- SQLx enables compile-time verified database operations.
- Tracing provides production-ready observability.
- The Rust crate ecosystem provides zero-cost modular tools for systems engineering.

Servo 0.1: Embeddable Engine on crates.io

Servo 0.1: Embeddable Engine on crates.io

Thesis

On **2026-04-13** the Servo project published **`servo` 0.1.0** on crates.io — the first release of the engine **as a library**, not only as the `servoshell` demo browser. The `0.1` version number is deliberate: the team has not defined what **1.0** means yet, but the bump signals growing confidence that the **embedding API** is usable for real embedders. Monthly crates keep moving (by late 2026 the crate line includes **0.7.x** regular releases plus a parallel **0.1.x LTS** line). This chapter teaches the mental model of embedding Servo: what the crate is for, what `0.1` vs LTS vs monthly means, and which build/feature knobs matter — without downloading the engine's enormous dependency graph in the book lab.

Toolchain note: examples assume a recent stable Rust. This authoring host has **rustc 1.99.0**. Current `servo` releases advertise **MSRV 1.88.0** (see crates.io); LTS migration notes may raise clang and Rust floors when you jump major LTS lines.

Mental model

```
Your application (windowing, input, IPC, privileges)

            embedding API (servo crate + embedder traits)


  Servo library
  constellation · script · layout · paint · net · …
  optional: webgl / webgpu / webxr / media-gstreamer



OS graphics / audio / network backends (surfman, etc.)
```

Concept	Meaning
`servo` crate	The embeddable engine on crates.io (MPL-2.0)
`servoshell`	Demo browser — **not** published to crates.io; build from the GitHub tree
0.1.0 milestone	First library release; embedding API "good enough for some users," not a stability covenant

Concept	Meaning
Monthly / regular line	Fast-moving; breaking changes expected between minors
LTS line	Best-effort ~6-month cadence, ~9-month support, **security fixes only**; may need `git` deps for CVE backports
`servo-embedder-traits`	Hooks embedders implement (e.g. custom resource readers)
`baked-in-resources` / `bundled`	Default path that embeds default resources in the binary; disable and supply your own reader if you need that

Risk profile (from the Servo book): rendering **known, trusted** content inside your app is a different threat model from a general-purpose browser that loads the open web. Evaluate accordingly — Servo is still pre-1.0.

Worked examples

Case 1: Read the milestone announcement, then pin deliberately

The 0.1.0 blog post is the historical milestone. For new work in late 2026 you typically choose:

Goal	Pinning strategy
Track features / file upstream bugs easily	Latest regular release on crates.io (e.g. `0.7.x`)
Ship a product with scheduled upgrades	Current **LTS** (see LTS release in the Servo book)
Reproduce a 0.1-era tutorial	`servo = "=0.1.0"` (expect migration work later)

```
# Cargo.toml - illustrative pins (check crates.io before locking)
[dependencies]
# Regular line (example version as of 2026-10; bump intentionally):
servo = "0.7"

# Or an LTS / milestone pin when you need a slower moving API:
# servo = "=0.1.4"
```

Do **not** treat `0.1` as "more stable than 0.7" merely because the number is smaller — on crates.io, `0.1.x` after the LTS decision is the **long-term** branch family, while `0.2`–`0.7` are the regular train. Read the Servo book's LTS page before you pick.

Case 2: Embedding API shape (no full build in this chapter)

Documentation for embedding lives under The Servo Book → Embedding. The stable ideas:

1. Depend on the **`servo`** library crate (not servoshell).
2. Drive the engine from your event loop: create a `Servo` / builder, feed input, composite frames into your windowing stack.
3. Decide **resources**: keep default `bundled` / `baked-in-resources`, or disable default features and call `servo_embedder_traits::submit_resource_reader` with your own `ResourceReader` (OpenHarmony's filesystem reader is the cited pattern upstream).
4. Select **feature flags** deliberately. Modularization moved many web features behind flags; LTS migration notes recommend considering `default_web_features` when you want the broad web surface.

```
// embed_sketch.rs - API orientation only (will not compile without the servo crate
// and platform deps). Do not cargo-add servo in the book lab: the dependency
// tree and system libraries (e.g. gstreamer for media) are large.
//
// use servo::ServoBuilder; // names evolve - follow the book for your pin
//
// fn main() {
//     let mut servo = ServoBuilder::new(/* embedder handles */)
//         // .prefs(...)
//         .build();
//     // pump events → servo.handle_events(...) → present frames
// }
fn main() {
    println!("See https://book.servo.org/embedding/overview.html for current builders");
}
```

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)
$ rustc embed_sketch.rs -o /tmp/embed_sketch && /tmp/embed_sketch
See https://book.servo.org/embedding/overview.html for current builders
```

Case 3: Building without `./mach` (embedder perspective)

Upstream notes that embedding builds often go through **`cargo`** directly, but `./mach` normally sets environment variables and enables **`media-gstreamer`** by default. If you bypass mach:

- Run `./mach print-env` on a Servo checkout when cross-compiling (Android / OpenHarmony) and mirror those variables.
- Expect to install platform media/graphics dependencies yourself if you enable gstreamer features.
- Prefer feature flags over "copy servoshell's Cargo.toml blindly."

This book chapter **does not** run `cargo add servo` or `./mach bootstrap` — both pull toolchains and system packages beyond the no-install lab policy. Use the Servo book's "Building servo without mach" section on a dedicated machine.

Case 4: Migrating off 0.1 LTS (preview)

The Servo book's "Migrating from 0.1 to 0.6 LTS" list is the boring checklist when you finally jump:

- Enable `default_web_features` (or an explicit subset) after modularization.
- Re-validate **prefs** — unknown preference names can **panic**.
- Update mouse button enums (`Left/Middle/Right` → `Primary/Auxiliary/Secondary`).
- Relocate WebXR registry registration as documented for your target version.
- Budget for **MSRV** and **clang** bumps on the new LTS.

Read the full list for your target LTS; do not crochet a partial migrate.

The trap

1. **Publishing or depending on servoshell.** The demo browser is not the embeddable product line on crates.io.
2. **Assuming crates.io LTS patches always exist.** Security backports may force **`git` dependencies** when MSRV blocks a crates.io bump — plan for that in corporate allow-lists.
3. **Leaving default features on "for convenience" on constrained devices.** Media, WebGL, WebGPU, and WebXR are heavy; turn on what you need.
4. **Treating 0.1 as production-complete.** The project is explicit: no 1.0 definition yet. Trusted-content widgets full browser replacement.
5. **Skipping resource ownership.** If you disable baked-in resources and forget a `ResourceReader`, startup fails in confusing ways. The OpenHarmony example is the reference pattern.

The boring rule

Depend on the **`servo` library crate**, pick **regular vs LTS** on purpose, keep **servoshell** out of your `Cargo.toml`, and follow book.servo.org/embedding for the builder/event-loop API on the version you pinned. Evaluate threat model before you point the engine at untrusted URLs.

Try this

1. Open crates.io/crates/servo and list the latest regular version vs the latest `0.1.x`. Write one sentence on which you would pin for a shipping app and why.
2. Read the embedding overview and LTS pages in the Servo book; sketch your app's event-loop ownership on paper (who owns the window, who calls into Servo).

3. On a disposable machine (not this no-install lab), create a hello-embed crate against an LTS pin and run *only* what the book's embedding chapter shows — then stop before enabling every feature flag.
4. Skim the 0.1→0.6 migration notes and mark which items would touch your sketch.

Sources

- Servo is now available on crates.io (2026-04-13)
- crates.io: servo (version history and features)
- Servo Book — Embedding overview
- Servo Book — LTS releases

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns

The Rust Compilation Pipeline

The Rust Compilation Pipeline

Understanding how `rustc` compiles code: 1. Parsing & Macro Expansion -> AST 2. HIR (High-level Intermediate Representation) -> Type checking 3. MIR (Mid-level Intermediate Representation) -> Borrow checker & optimizations 4. LLVM IR -> Native machine code generation

Debugging Rust Programs

Debugging Rust Programs

Debugging Rust applications using `gdb`, `lldb`, and Rust backtraces:

```
RUST_BACKTRACE=1 cargo run
```

Security Considerations in Rust

Security Considerations in Rust

- Audit dependencies using `cargo-audit` and `cargo-vet`.
- Avoid integer overflow panics in production using checked/saturating arithmetic.
- Sanitize memory buffers holding sensitive cryptographic keys with crates like `zeroize`.

Common Anti-Patterns and How to Avoid Them

Common Anti-Patterns and How to Avoid Them

- **Anti-pattern:** Excessive `.clone()` calls to appease the borrow checker. *Remedy:* Restructure ownership or pass references.
- **Anti-pattern:** Unnecessary `Arc<Mutex<T>>` when local message passing suffices. *Remedy:* Use channels or scoped threads.
- **Anti-pattern:** Overusing `Box<dyn Error>` in public library crates. *Remedy:* Expose custom structured error enums.

Summary

Summary

Chapter Key Takeaways

- Understanding the AST -> HIR -> MIR -> LLVM pipeline clarifies compiler errors and optimizations.
- `RUST_BACKTRACE=1` and debuggers demystify runtime failures.
- Regular security audits with `cargo-audit` prevent supply-chain vulnerabilities.
- Writing idiomatic Rust eliminates defensive over-cloning and unnecessary synchronization locks.

WebAssembly with Rust

Why Rust for WebAssembly?

Why Rust for WebAssembly?

WebAssembly (Wasm) is a binary instruction format for a stack-based virtual machine designed for near-native execution speed on the web and edge servers.

Rust is uniquely positioned as the premier language for WebAssembly: - **No Garbage Collector:** Wasm has no built-in GC; Rust's compile-time memory management avoids bundling heavy GC runtimes into the `.wasm` binary. - **Compact Binaries:** Tiny output sizes optimize load times across web browsers and serverless functions. - **Rich Tooling:** `wasm-pack` and `wasm-bindgen` provide seamless high-level JavaScript interoperability.

WebAssembly Fundamentals

WebAssembly Fundamentals

Wasm executes inside a sandboxed linear memory space with strict type checking and deterministic execution semantics.

Setting Up: wasm-pack and wasm-bindgen

Setting Up: wasm-pack and wasm-bindgen

Install `wasm-pack`:

```
cargo install wasm-pack
```

Configure `Cargo.toml`:

```
[lib]
crate-type = ["cdylib", "rlib"]

[dependencies]
wasm-bindgen = "0.2"
```

Building Your First WebAssembly Module

Building Your First WebAssembly Module

```
// In src/lib.rs
// use wasm_bindgen::prelude::*;
//
// #[wasm_bindgen]
// pub fn add(a: i32, b: i32) -> i32 {
//     a + b
// }
```

Build the npm package:

```
wasm-pack build --target web
```

Interacting with the DOM

Interacting with the DOM

Using `web-sys` to interact directly with DOM nodes, canvases, and local storage from Rust.

Handling Events and Async

Handling Events and Async

Attaching browser event listeners with closures and running async promises with `wasm-bindgen-futures`.

Performance Considerations

Performance Considerations

Optimizing Wasm size and throughput using `wasm-opt` from binaryen and disabling panic formatting.

Server-Side WebAssembly

Server-Side WebAssembly

Running Wasm outside the browser using WASI (WebAssembly System Interface) and runtimes like Wasmtime, Wasmer, and Cloudflare Workers.

Publishing and Distribution

Publishing and Distribution

Distributing Wasm modules via npm registries or OCI container registries for serverless edge deployments.

Summary

Summary

Chapter Key Takeaways

- Rust's lack of a GC runtime makes it ideal for compact, high-speed WebAssembly modules.
- `wasm-pack` packages Rust code into web-ready JavaScript/Wasm bundles.
- WASI enables secure, sandboxed server-side WebAssembly execution.

Embedded Rust and no_std Development

Understanding no_std

Understanding no_std

Embedded microcontrollers lack operating system services like virtual memory, filesystems, and threads. In embedded Rust, programs declare `#![no_std]`, compiling without the standard library while retaining the core language and primitives.

```
// #![no_std]
// #![no_main]
//
// use core::panic::PanicInfo;
//
// #[panic_handler]
// fn panic(_info: &PanicInfo) -> ! {
//     loop {}
// }
```

Choosing a Target: Cortex-M

Choosing a Target: Cortex-M

Targeting ARM Cortex-M microcontrollers using `rustup target add thumbv7em-none-eabihf`.

Writing an Embedded Application: Blinking an LED

Writing an Embedded Application: Blinking an LED

The classic embedded "Hello World": toggling a GPIO pin tied to an LED using Peripheral Access Crates (PAC) and Hardware Abstraction Layers (HAL).

Memory Management Without a Heap

Memory Management Without a Heap

Managing state using static memory and stack buffers with zero dynamic heap allocation.

Interrupts and Concurrency

Interrupts and Concurrency

Handling hardware interrupt service routines (ISRs) safely using critical sections and interrupt-safe atomics.

Choosing an Executor or RTOS

Choosing an Executor or RTOS

Comparing bare-metal superloops, RTIC (Real-Time Interrupt-driven Concurrency), and Embassy (async embedded executor).

Testing Embedded Code

Testing Embedded Code

Unit testing business logic on host architectures and running on-target tests with `probe-rs`.

Toolchain and Debugging

Toolchain and Debugging

Hardware flashing and in-circuit debugging via SWD/JTAG with `probe-rs` and `gdb`.

Summary

Summary

Chapter Key Takeaways

- `#![no_std]` strips OS dependencies for bare-metal firmware development.
- Hardware Abstraction Layers (HAL) provide safe, type-checked register manipulation.
- Critical sections and atomics guarantee interrupt safety without data corruption.

Conclusion: The Rust Mindset

Core Principles

Core Principles

Looking back over our journey, three fundamental engineering principles define idiomatic Rust:

1. **Safety is an Asset, Not a Constraint:** The borrow checker is an automated proof assistant ensuring thread and memory safety.
2. **Explicit over Implicit:** Errors are values, allocations are intentional, and data flows are visible.
3. **Zero-Cost Abstractions:** High-level expressiveness compiles down to optimal low-level machine instructions.

Where Rust Shines

Where Rust Shines

Rust delivers unmatched value in: - High-performance network proxies, database engines, and storage systems. - Operating system kernels, drivers, and hypervisors. - Low-latency financial trading platforms. - WebAssembly client-side computational modules and edge serverless workers. - Embedded firmware and safety-critical medical and automotive controllers.

Where Rust May Not Be the Best Fit

Where Rust May Not Be the Best Fit

Rust may introduce unnecessary friction for: - Disposable one-off scripts where Python or Bash finishes the job in minutes. - Rapidly churning early-stage UI prototypes where types and domain models change hourly. - Systems where memory footprint and latency do not matter and teams already possess deep expertise in managed languages.

Continuing Your Journey

Continuing Your Journey

To continue deepening your Rust mastery: - Read *The Rust Reference* and *The Rustonomicon* for deep language semantics and unsafe invariants. - Explore open-source projects on crates.io to observe production architectures. - Contribute bug fixes and documentation improvements to the community.

Final Thoughts

Final Thoughts

Rust changes the way you think about programming. Even when writing Go, C, or Python, the habits ingrained by Rust—thinking carefully about ownership, data sharing, mutability, and error handling—will make you a vastly more rigorous systems engineer.

Welcome to the community, and build fearless software.